



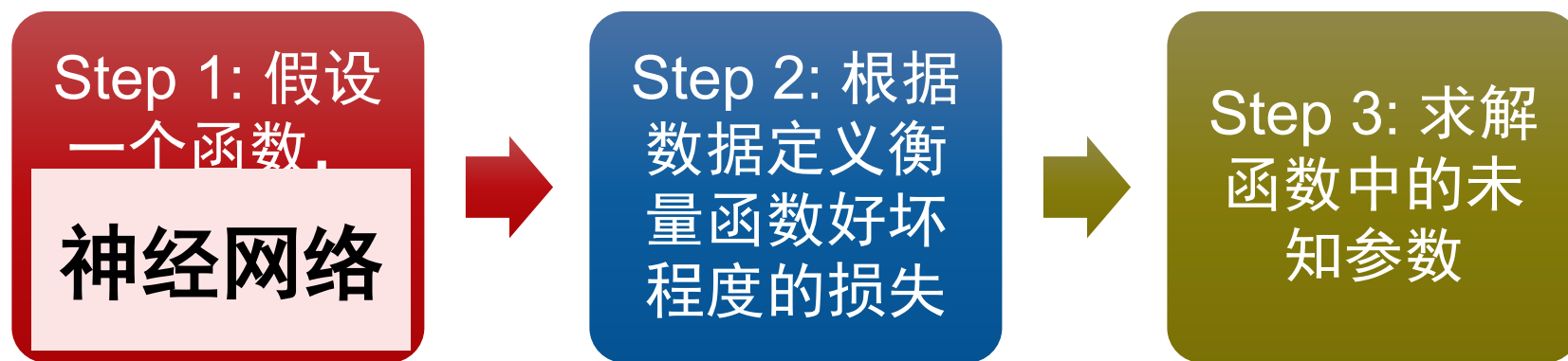
第五章：深度学习

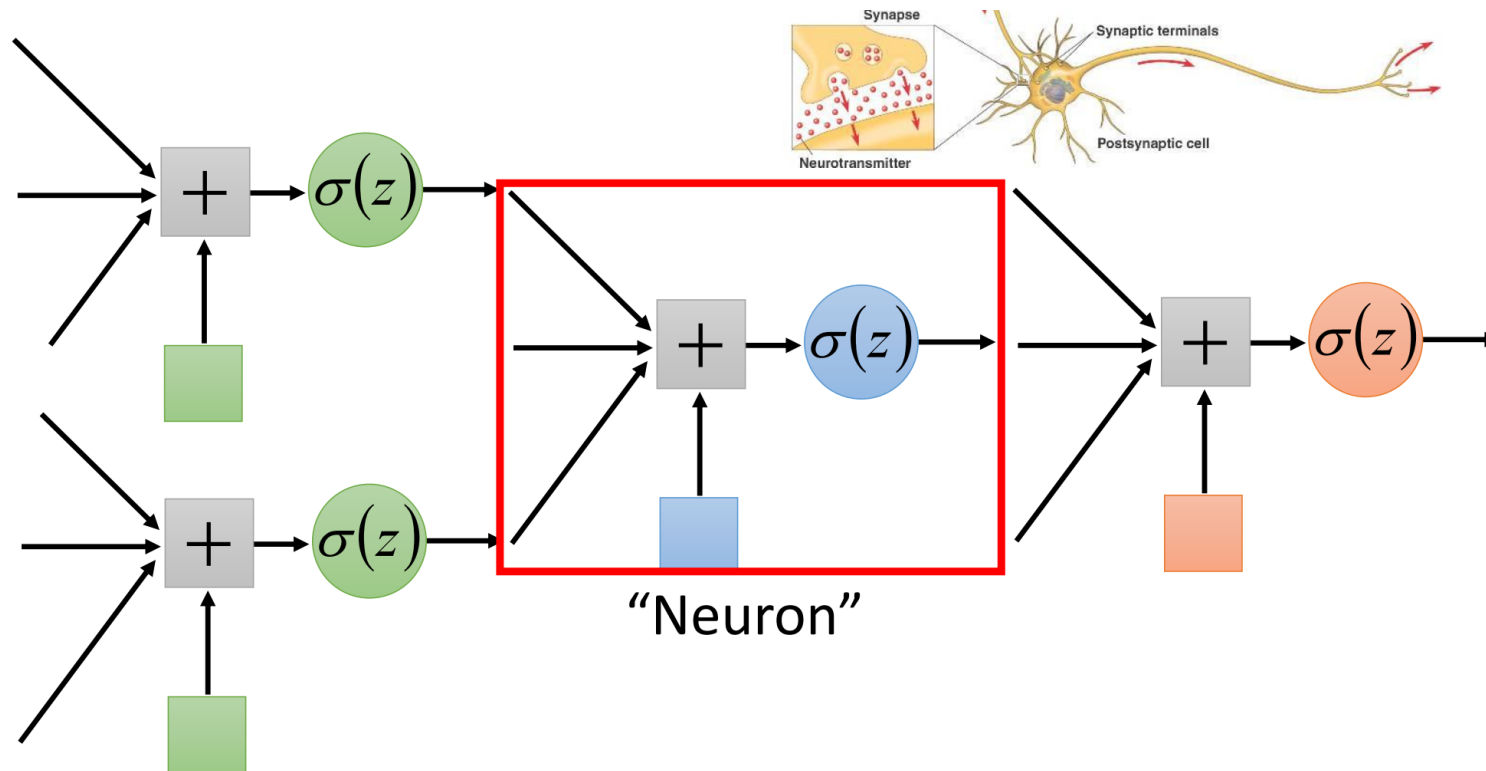
目录

- 神经网络基本概念
- 卷积神经网络
- 循环神经网络
- Transformer

目录

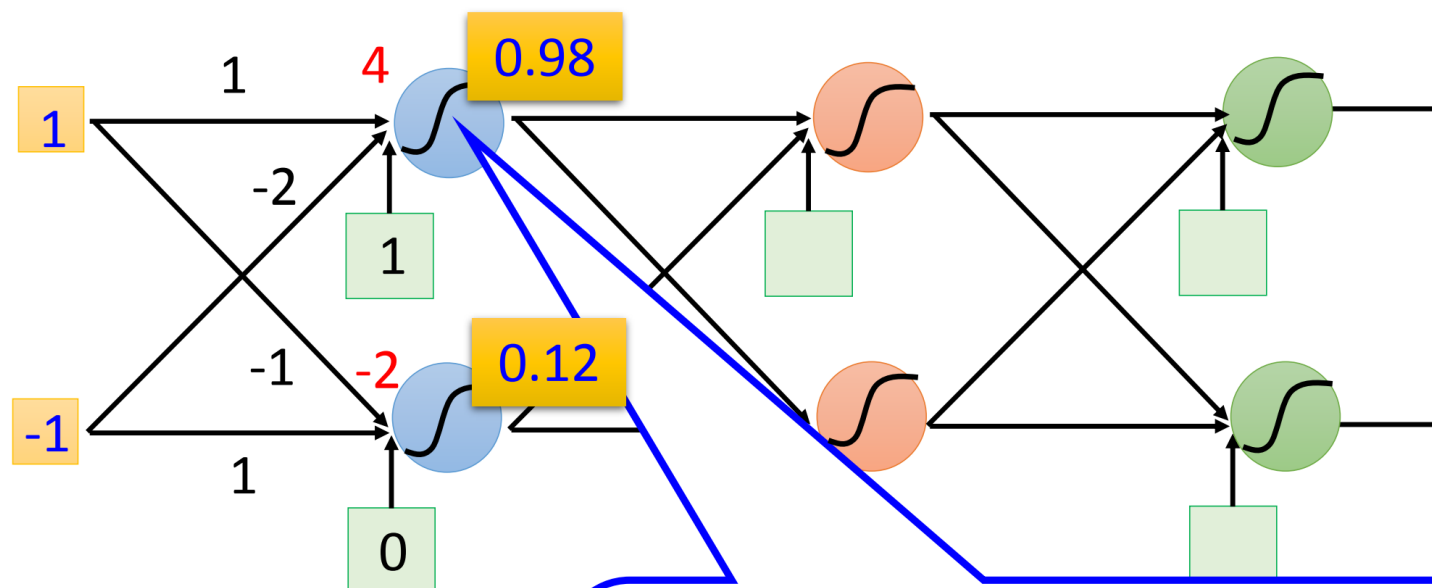
- 神经网络基本概念
- 卷积神经网络
- 循环神经网络
- Transformer





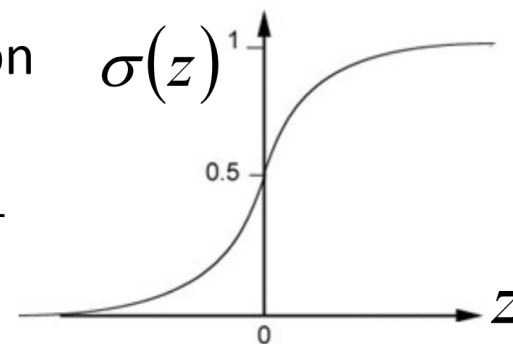
- 不同的连接方式会导致不同的网络结构
- 网络参数 θ : 所有神经元中的权重 w 和偏置 b

全连接前馈网络

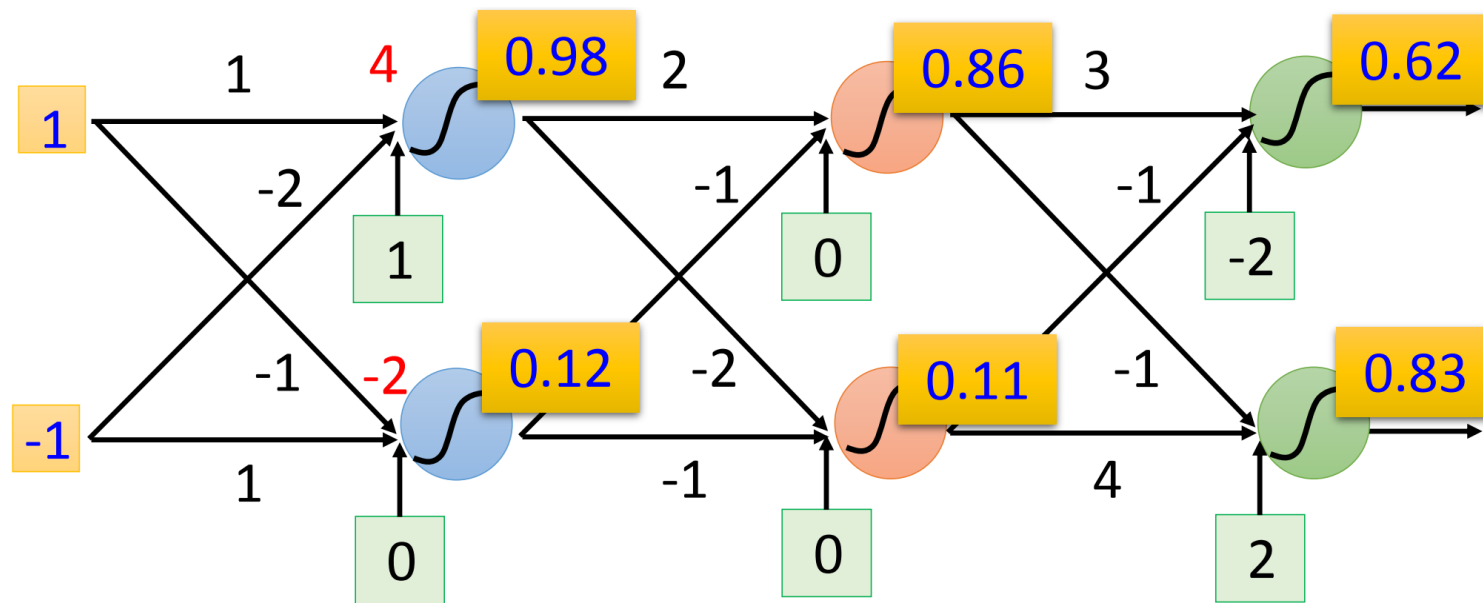


Sigmoid Function

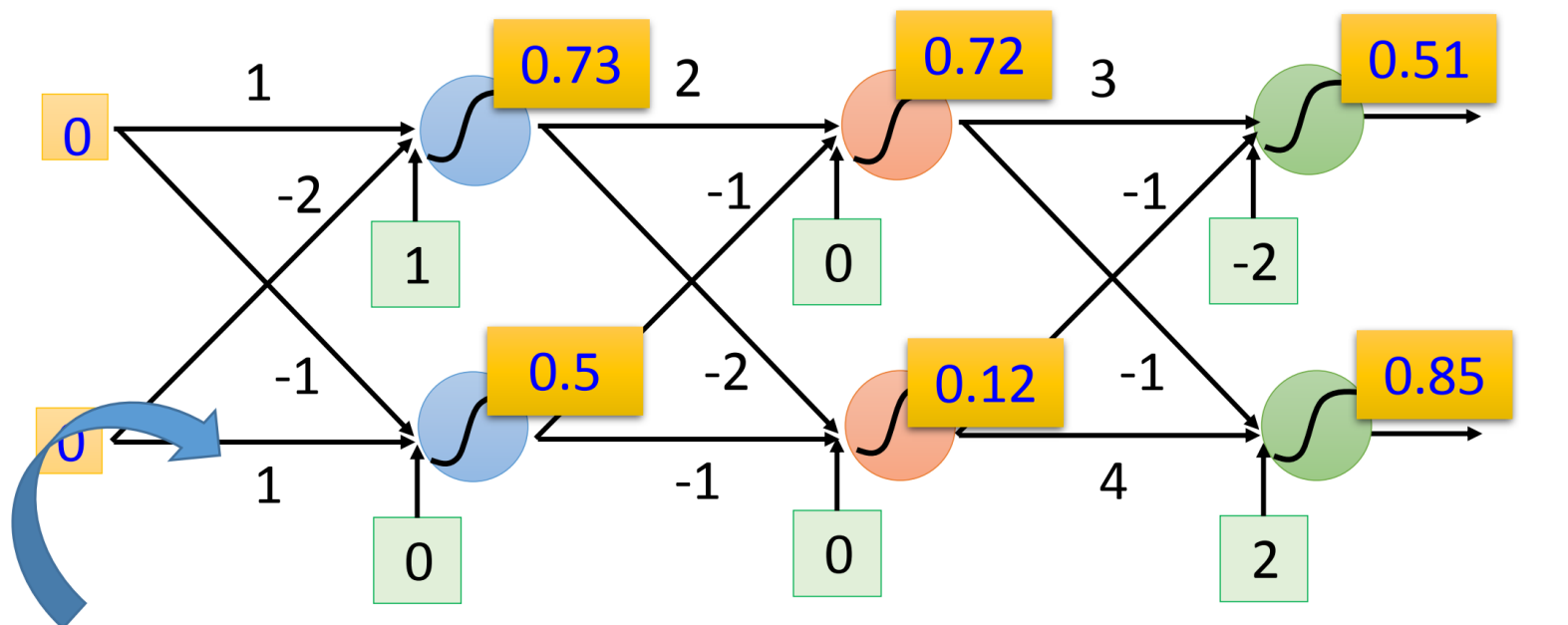
$$\sigma(z) = \frac{1}{1 + e^{-z}}$$



全连接前馈网络



全连接前馈网络

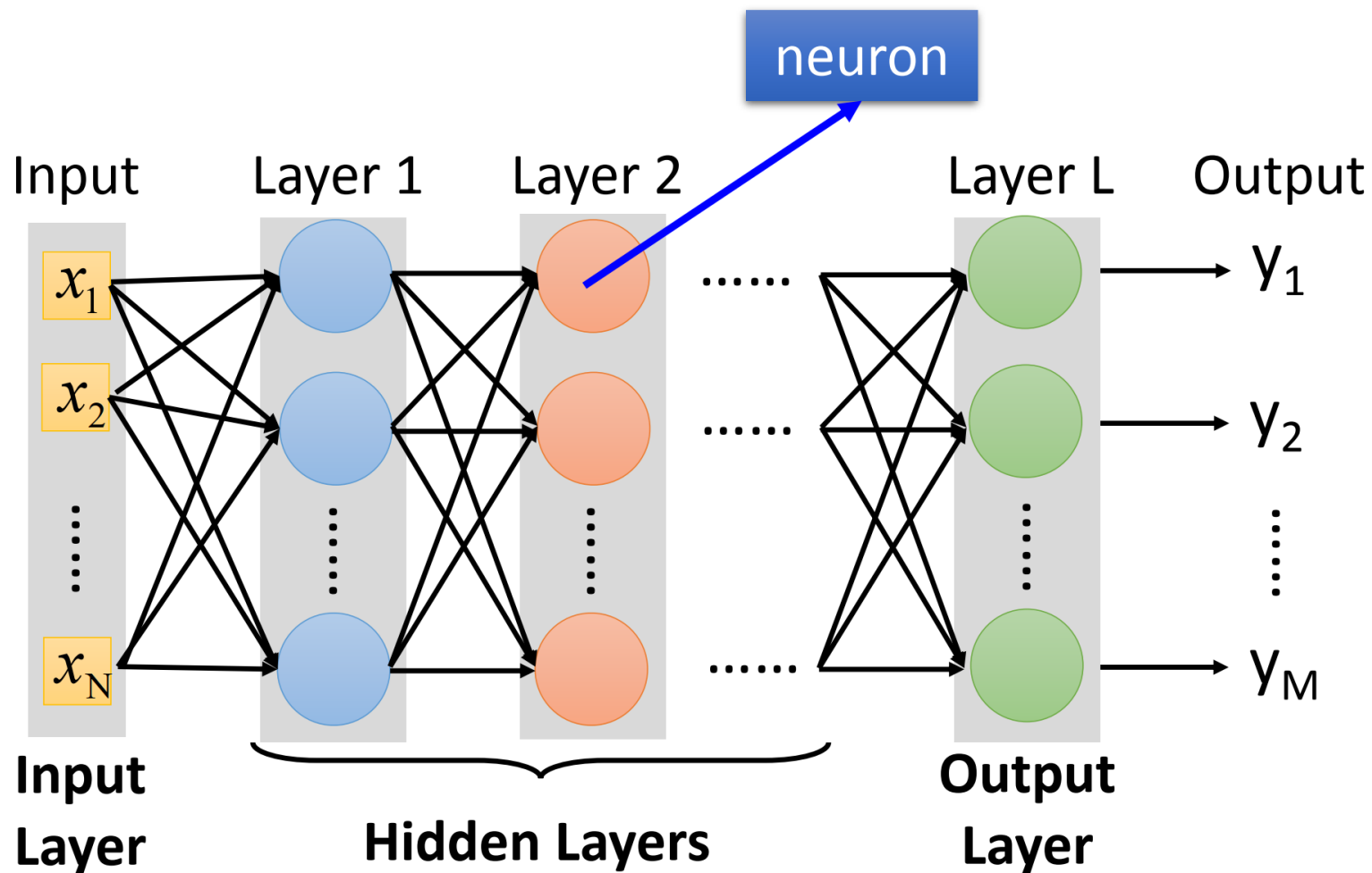


这是一个函数

$$f\left(\begin{bmatrix} 1 \\ -1 \end{bmatrix}\right) = \begin{bmatrix} 0.62 \\ 0.83 \end{bmatrix} \quad f\left(\begin{bmatrix} 0 \\ 0 \end{bmatrix}\right) = \begin{bmatrix} 0.51 \\ 0.85 \end{bmatrix}$$

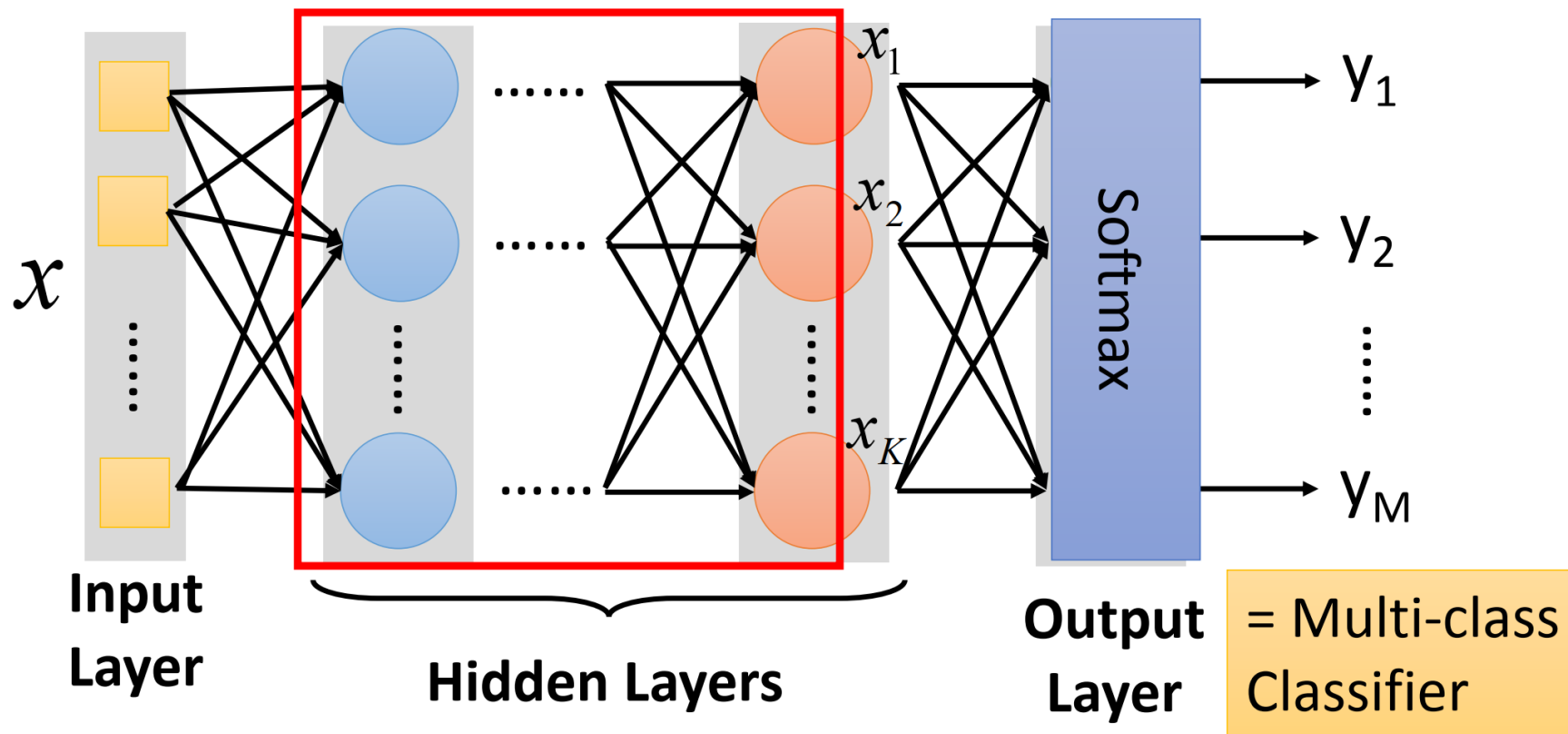
- 给定一个网络结构就定义了一个函数族

全连接前馈网络



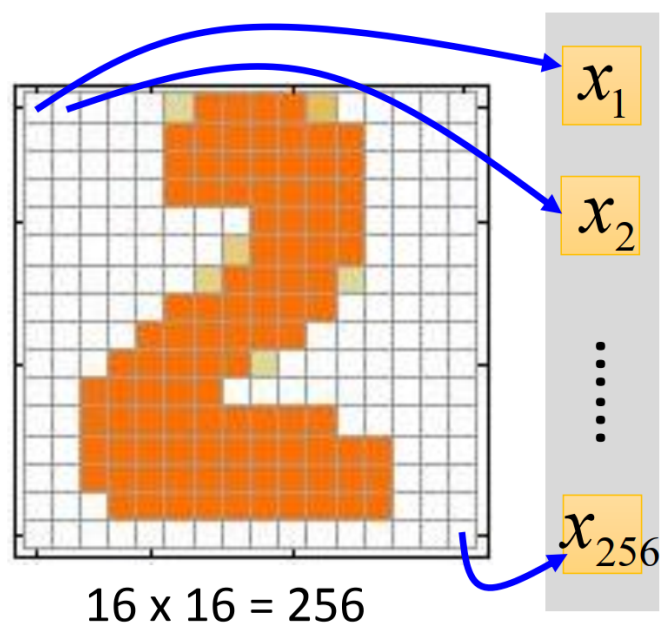
- 深度=很多隐藏层

特征抽取，代替特征工程

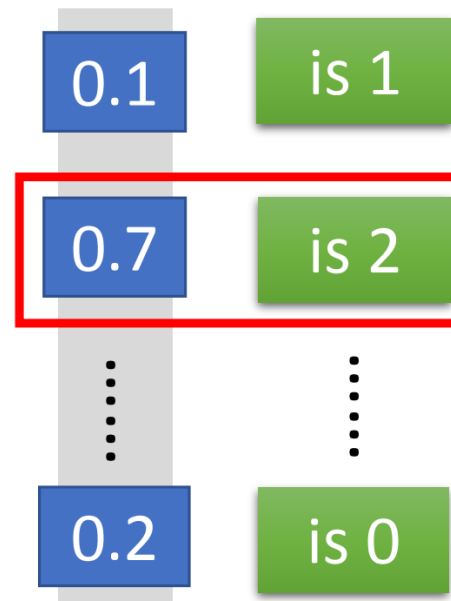


神经网络

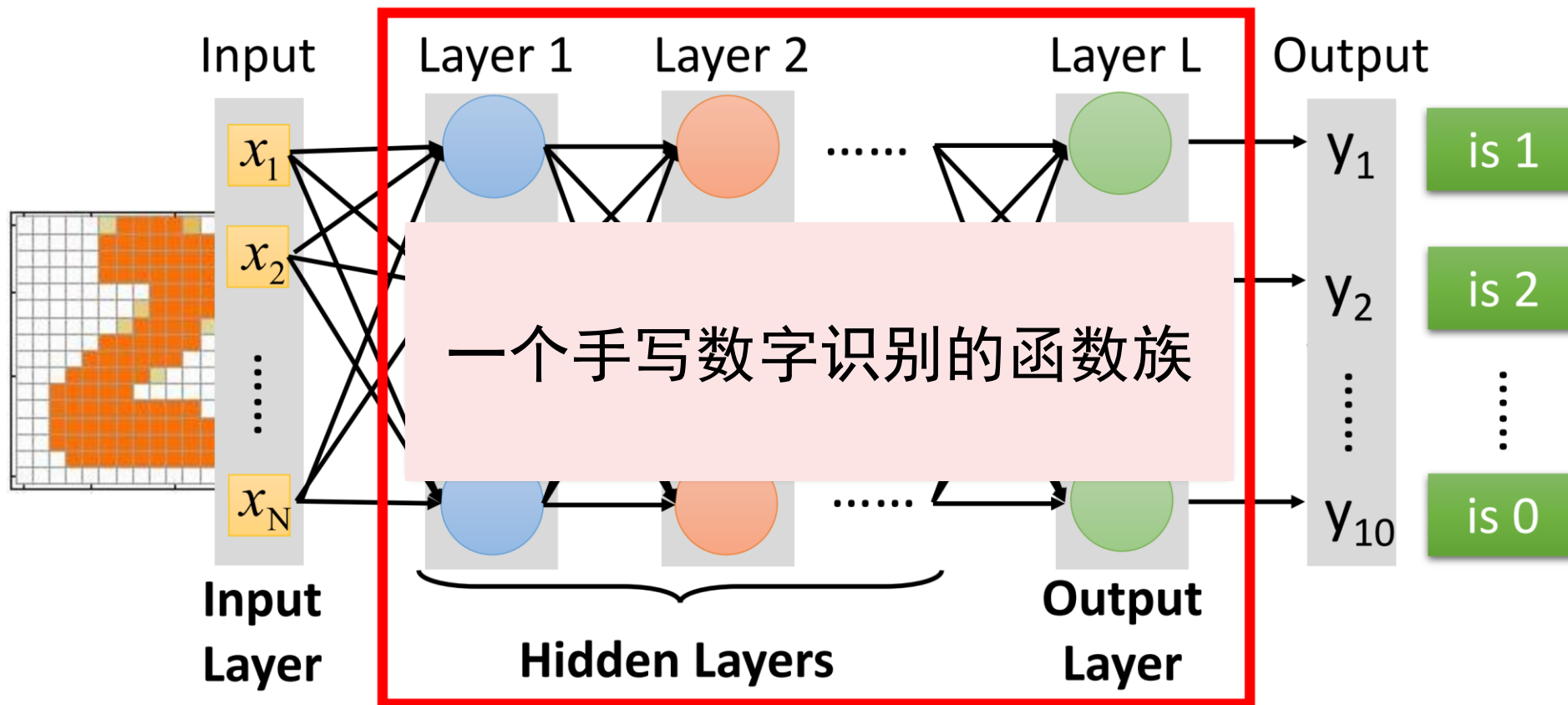
输入



输出：每个维度代表对图片输出这个数字的置信度

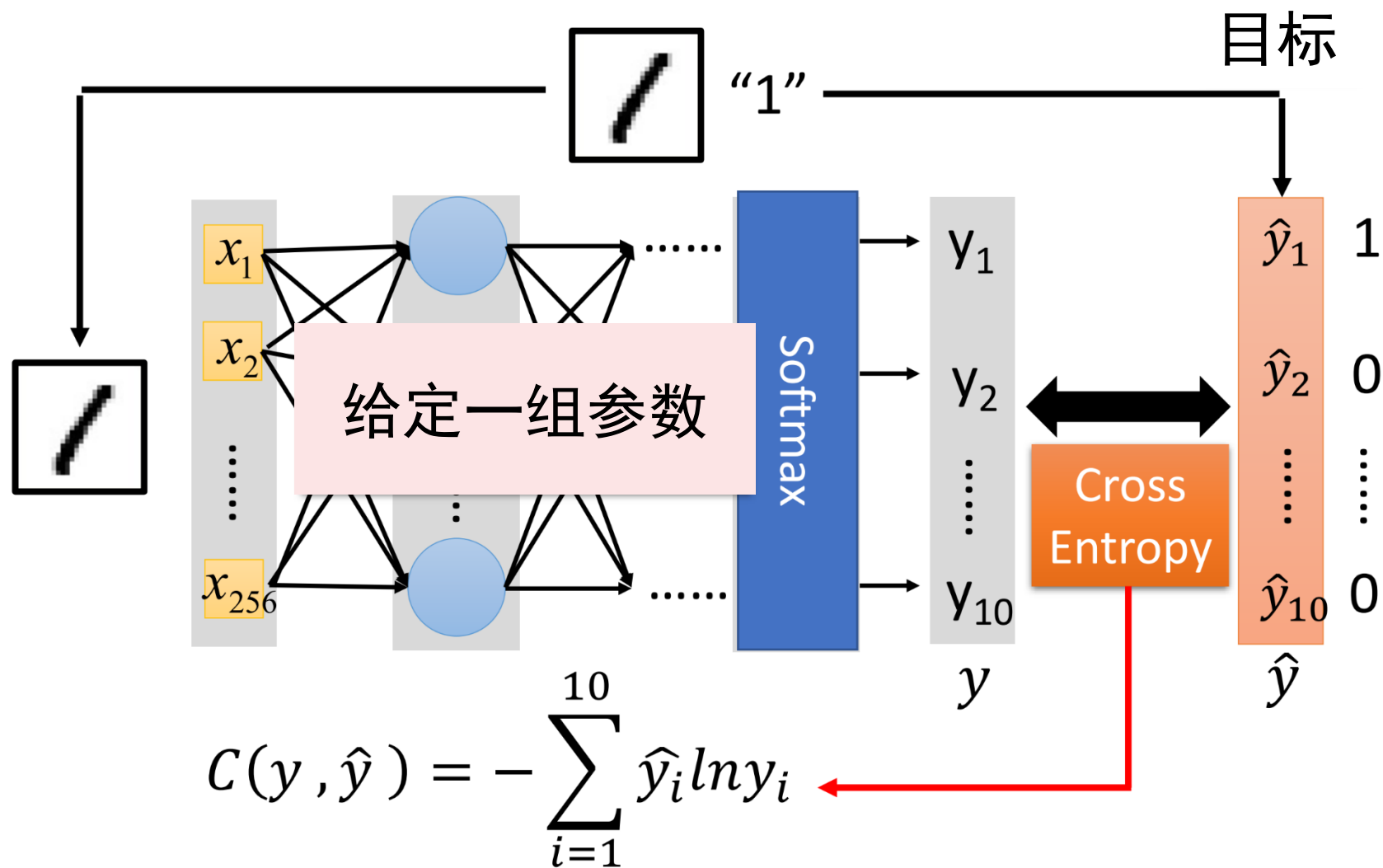


图里是2

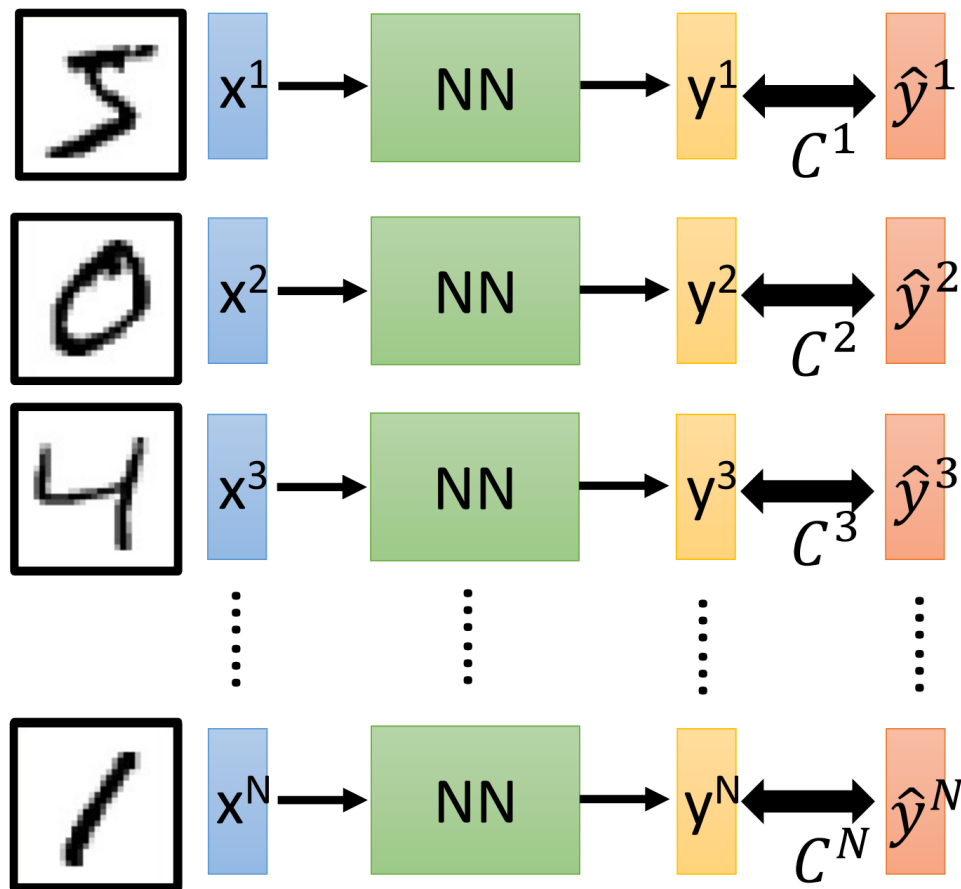


- 需要确定网络结构，以便在定义的函数族中实现一个表现良好的函数

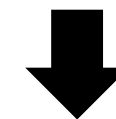
损失函数



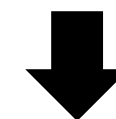
损失函数



$$L = \sum_{n=1}^N C^n$$

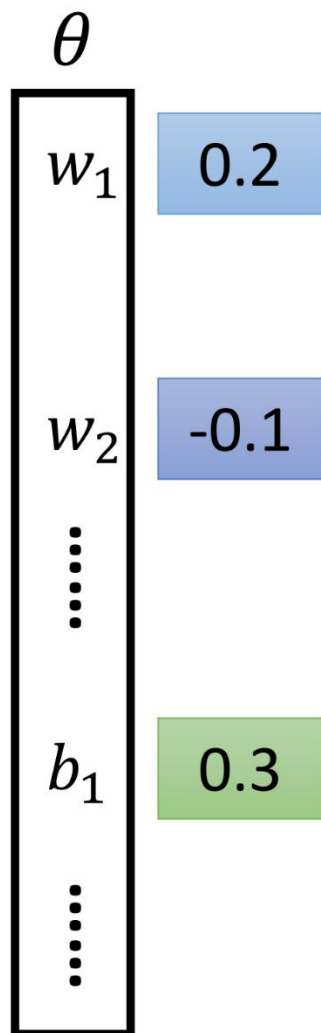


找可以最小化总损失的函数族



找可以最小化总损失的网络参数 θ

梯度下降



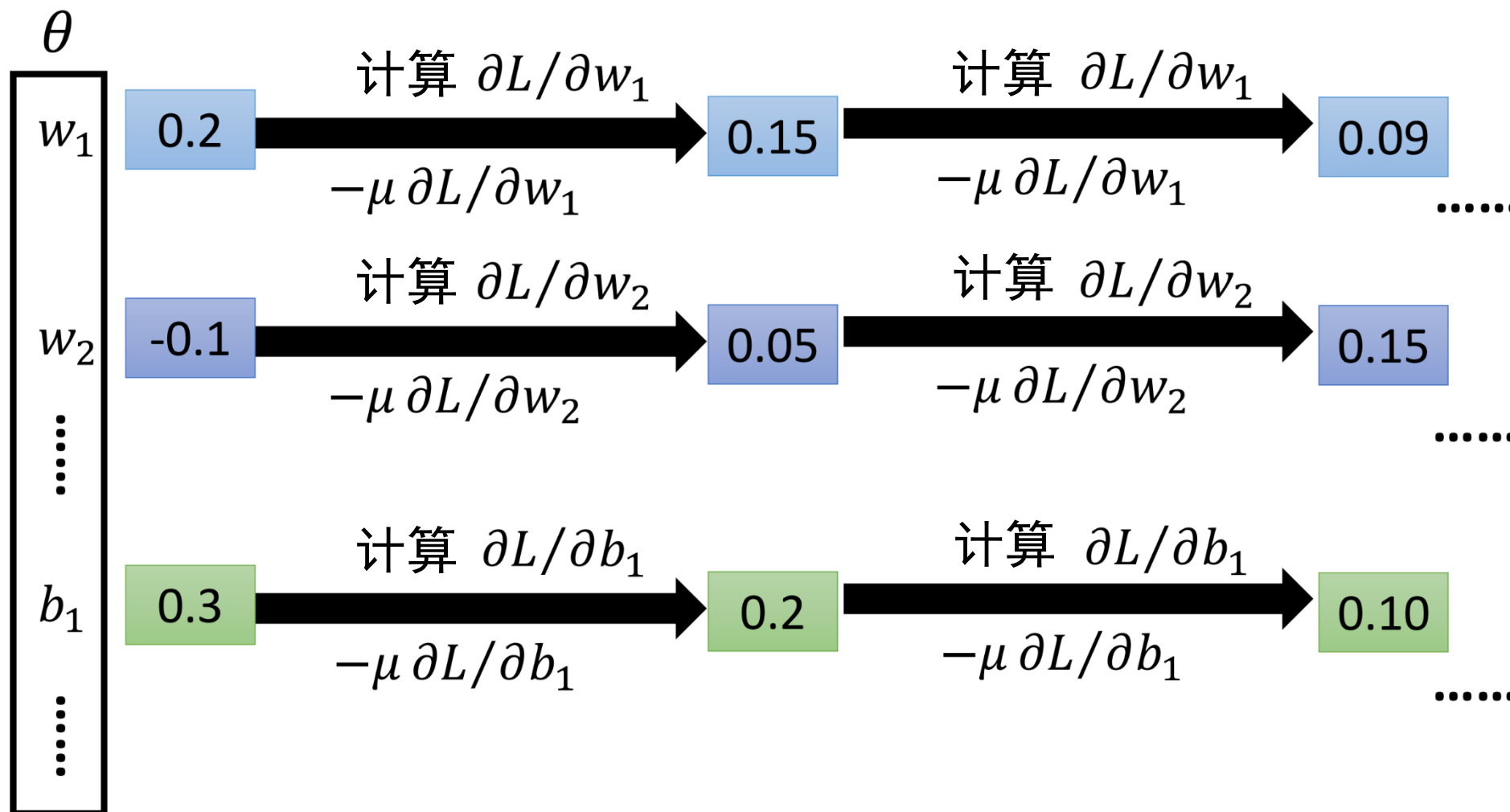
计算 $\partial L / \partial w_1$

计算 $\partial L / \partial w_2$

计算 $\partial L / \partial b_1$

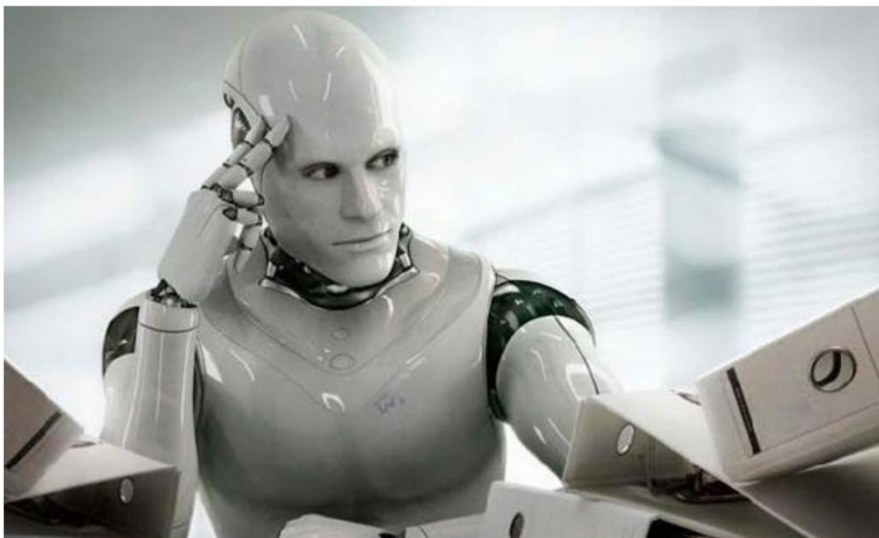
$$\nabla L = \begin{bmatrix} \frac{\partial L}{\partial w_1} \\ \frac{\partial L}{\partial w_2} \\ \vdots \\ \frac{\partial L}{\partial b_1} \\ \vdots \end{bmatrix}$$

梯度下降



机器“学习”

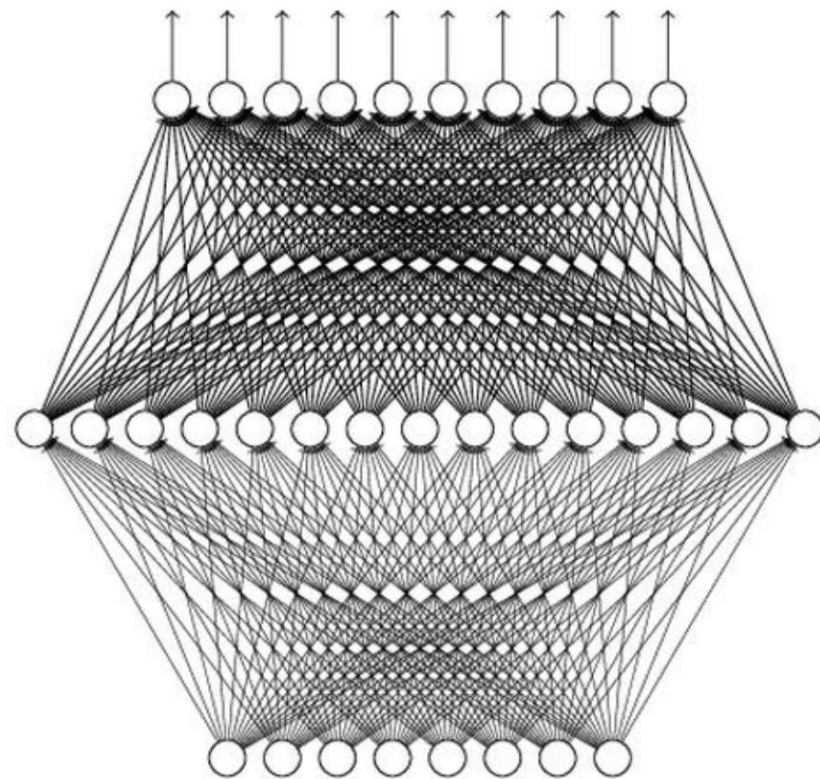
人们以为是



越深越好吗

若隐藏层神经元数量足够多，
任何连续函数 $f: \mathbb{R}^N \rightarrow \mathbb{R}^M$ 都可以通过
具有一个隐藏层的网络实现
—— Universality Theorem

那为什么是“深度”神经网络而非“肥胖”神经网络呢？



越深越好吗

Layer X Size	Word Error Rate (%)
1 X 2k	24.2
2 X 2k	20.4
3 X 2k	18.4
4 X 2k	17.8
5 X 2k	17.2
7 X 2k	17.1

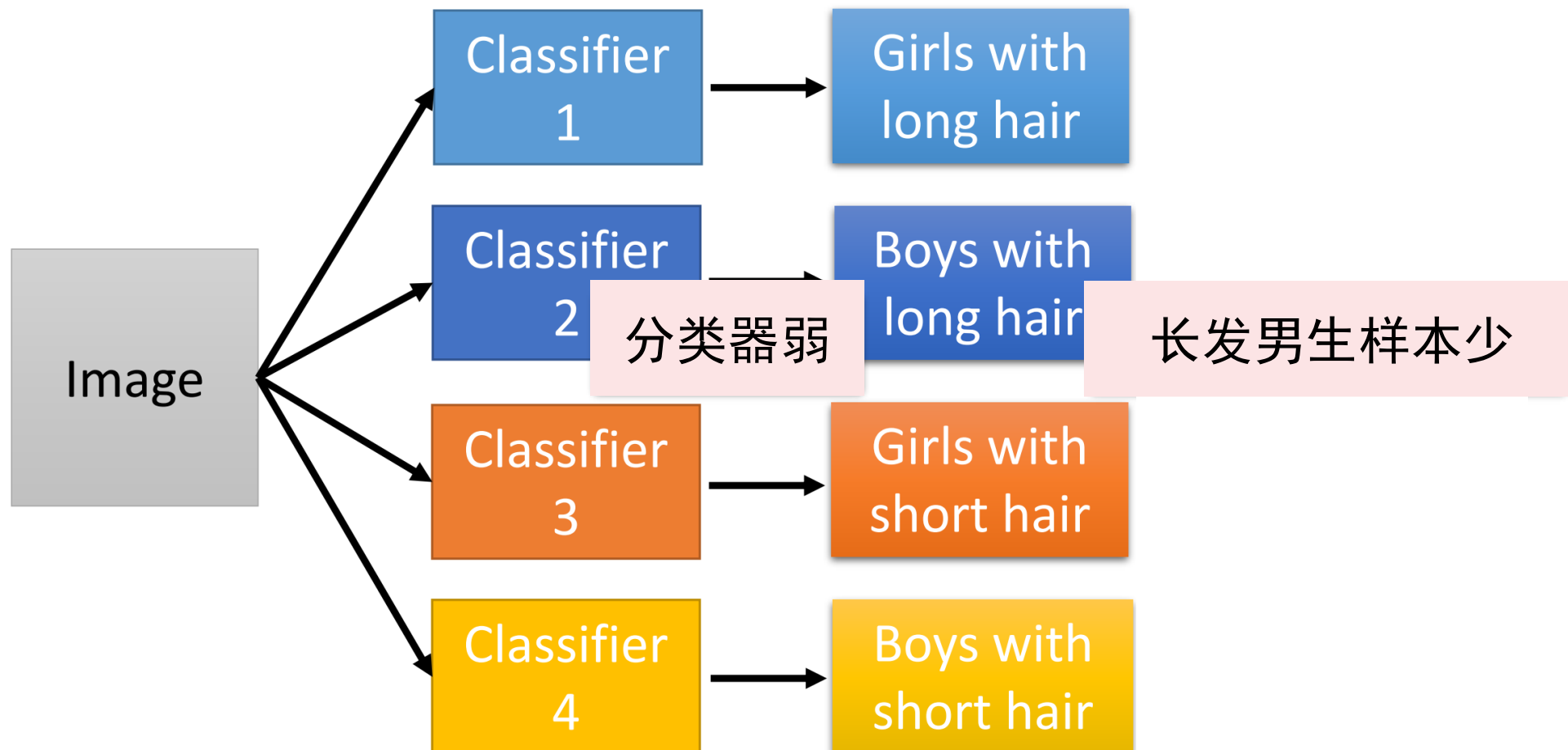
毫不意外 越多参数 效果越好

胖矮 vs. 瘦高

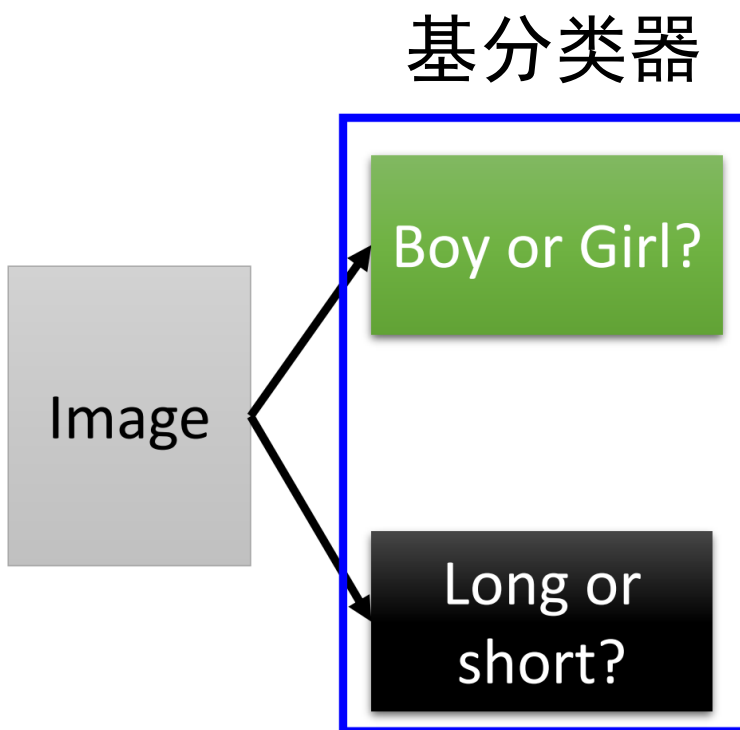
Layer X Size	Word Error Rate (%)	Layer X Size	Word Error Rate (%)
1 X 2k	24.2		
2 X 2k	20.4		
3 X 2k	18.4		
4 X 2k	17.8		
5 X 2k	17.2	1 X 3772	22.5
7 X 2k	17.1	1 X 4634	22.6
		1 X 16k	22.1

为什么?

模块化

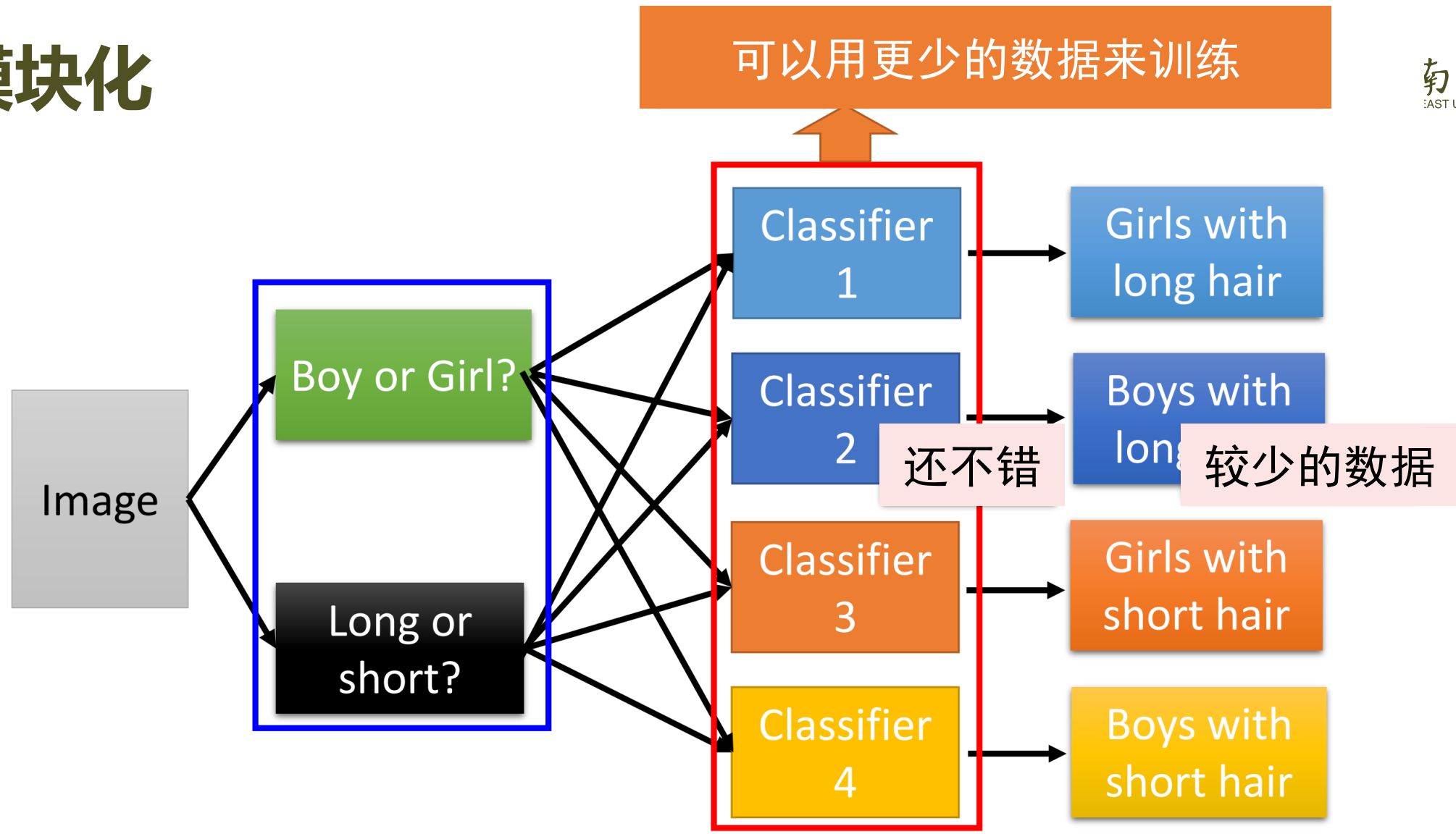


模块化

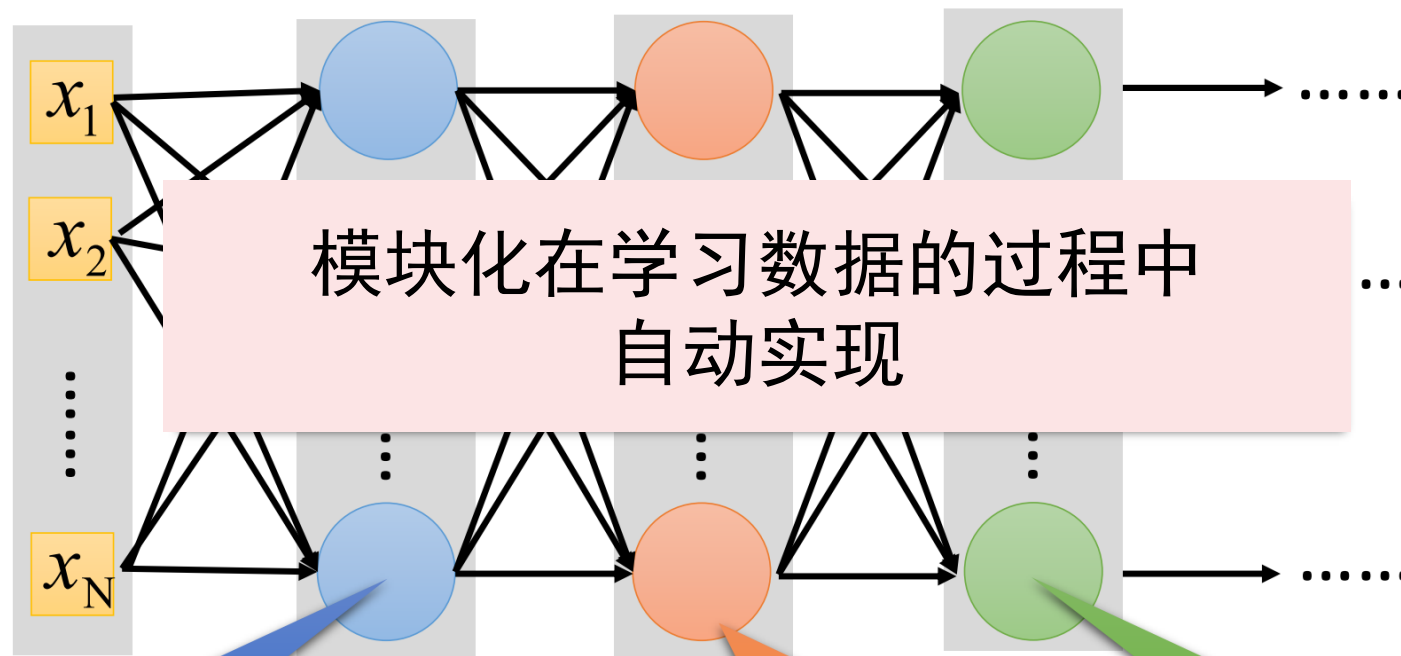


每个基分类器都有足够多的样本

模块化



模块化

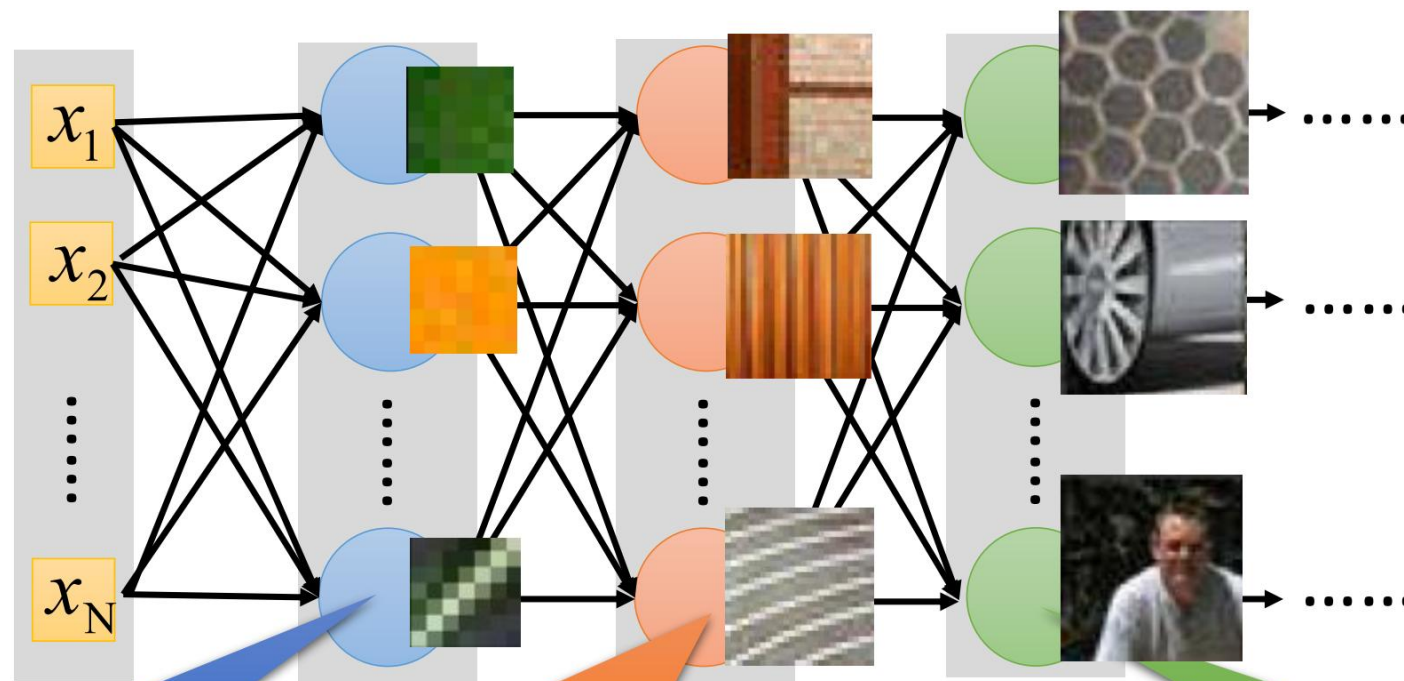


最基础的
分类器

用第一层作为模块来
构建分类器

用第二层作为
模块...

模块化

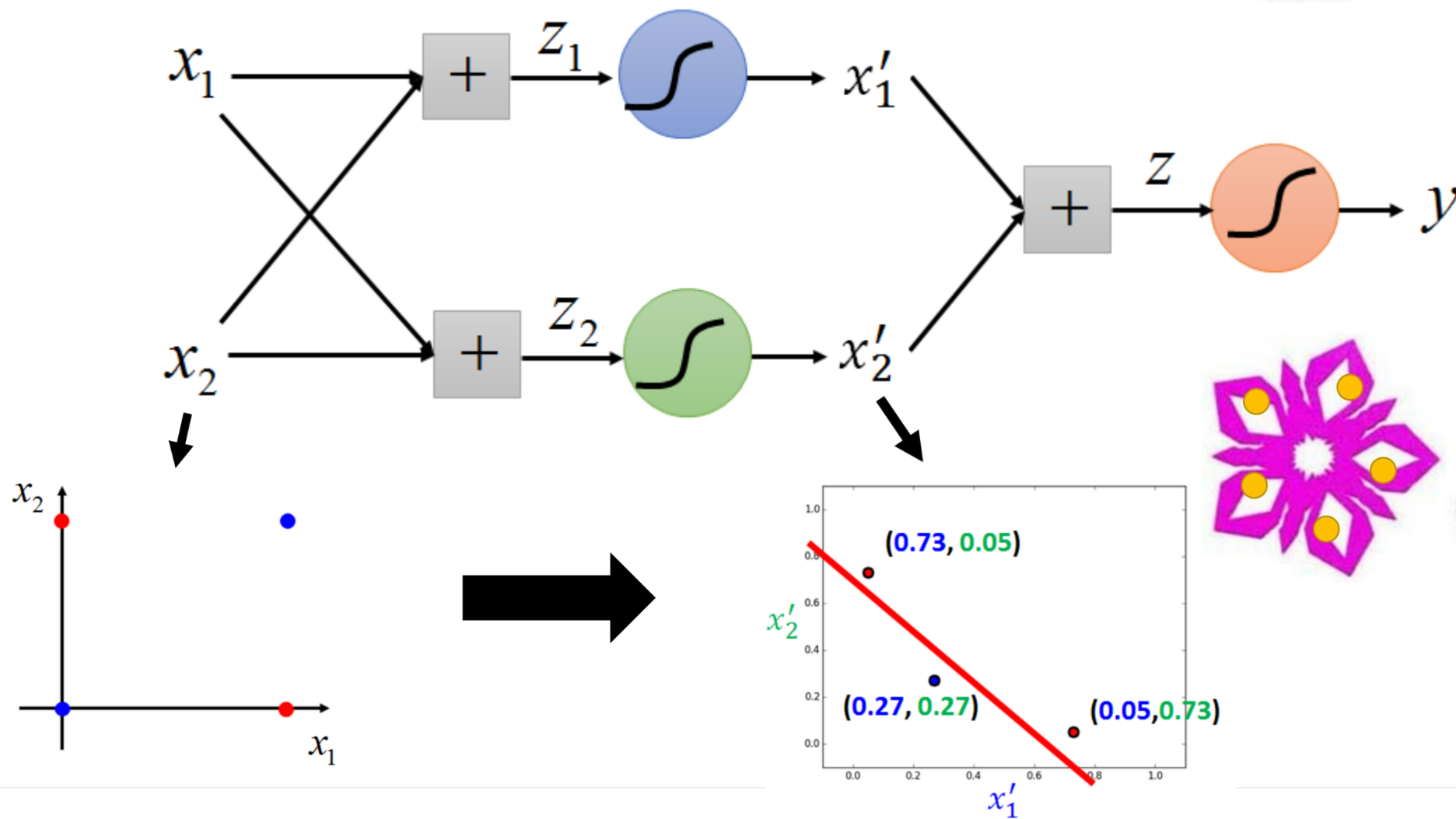


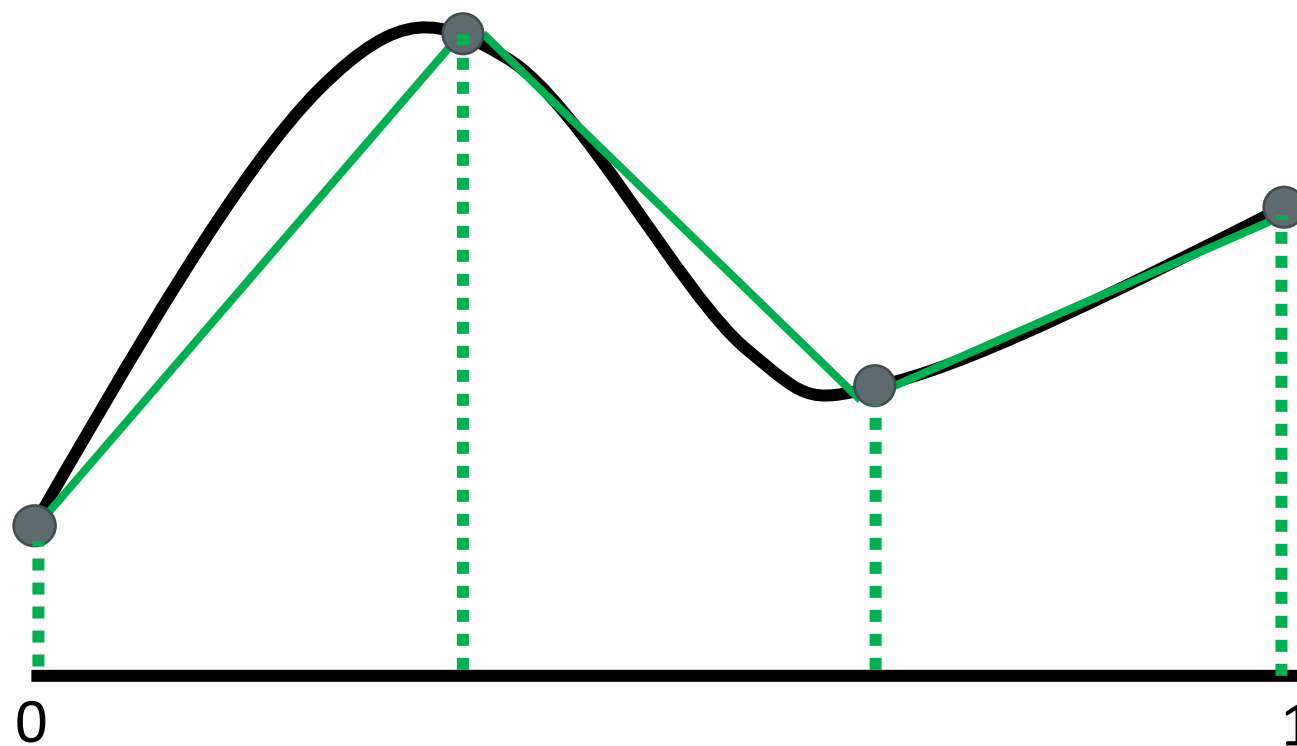
最基础的
分类器

用第一层作为模块来
构建分类器

用第二层作为
模块...

模块化





- 用足够多的分段就能得到任意曲线很好的近似

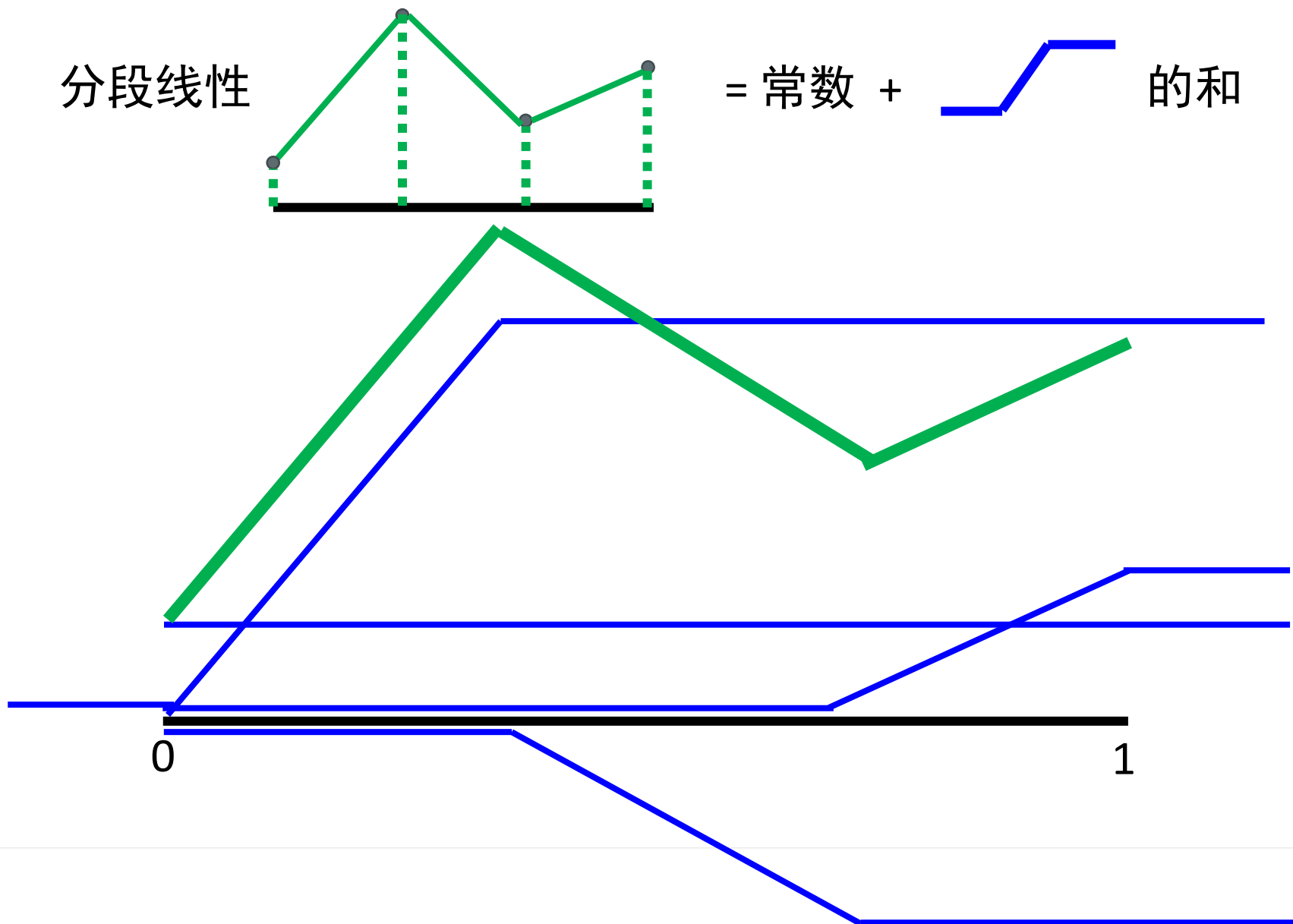
激活函数

分段线性

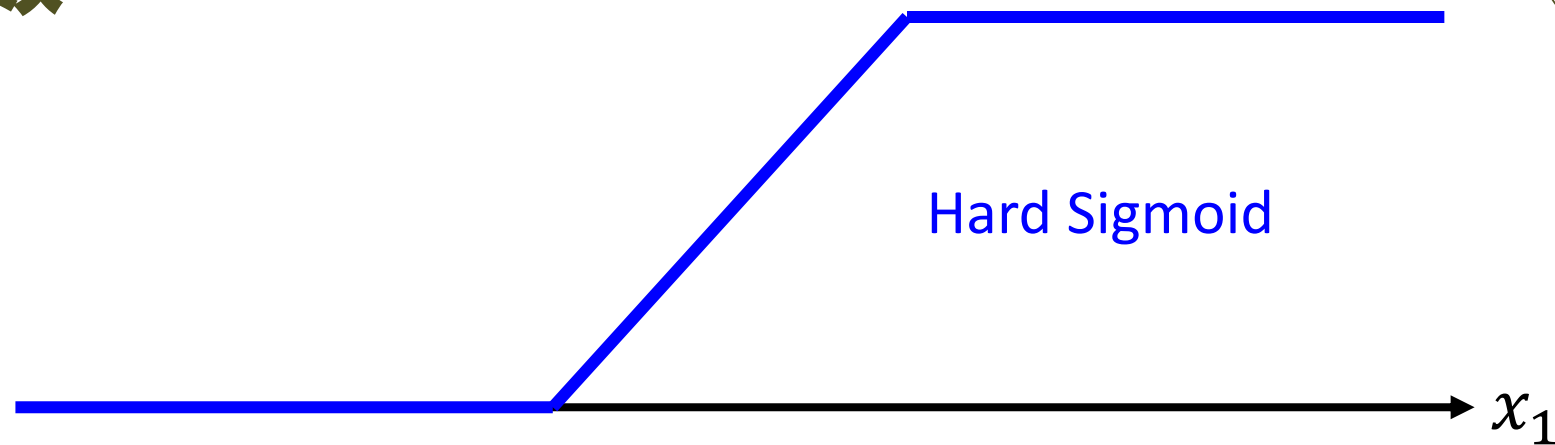
= 常数 +



的和

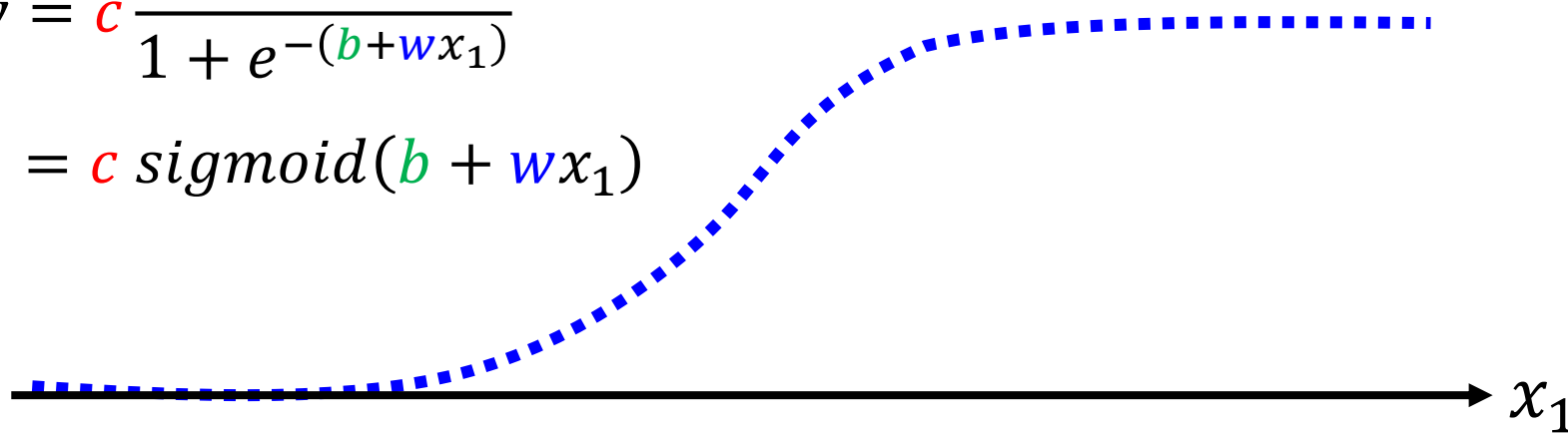


激活函数

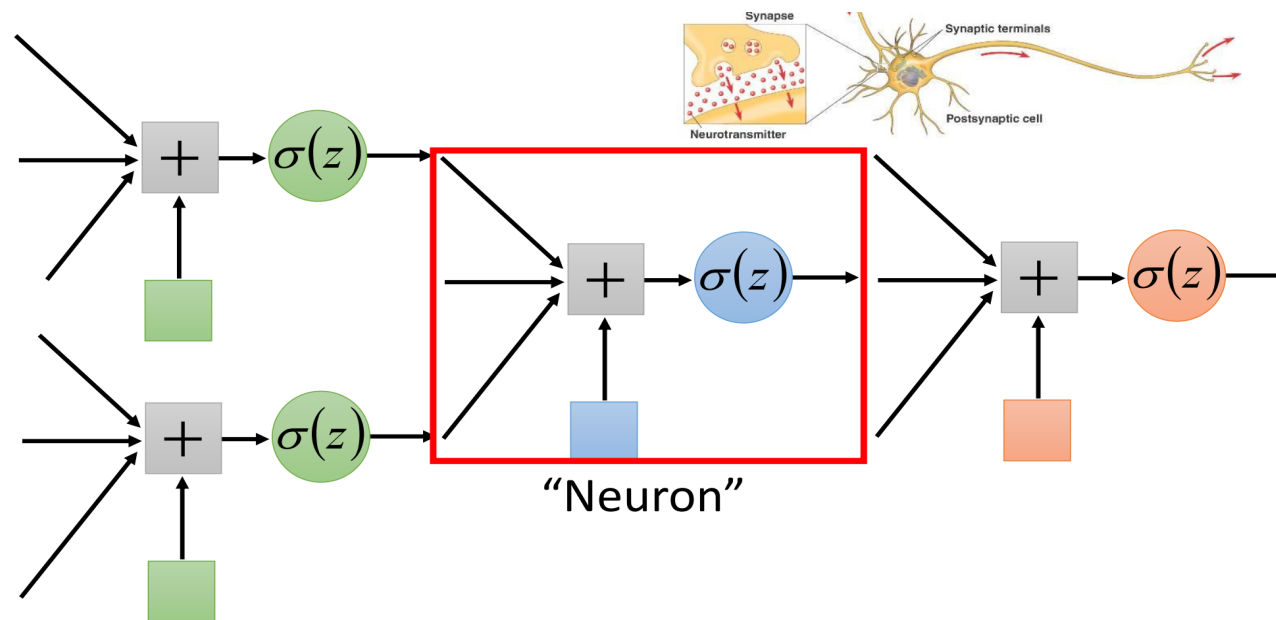


Sigmoid Function

$$y = c \frac{1}{1 + e^{-(b + wx_1)}}$$
$$= c \operatorname{sigmoid}(b + wx_1)$$

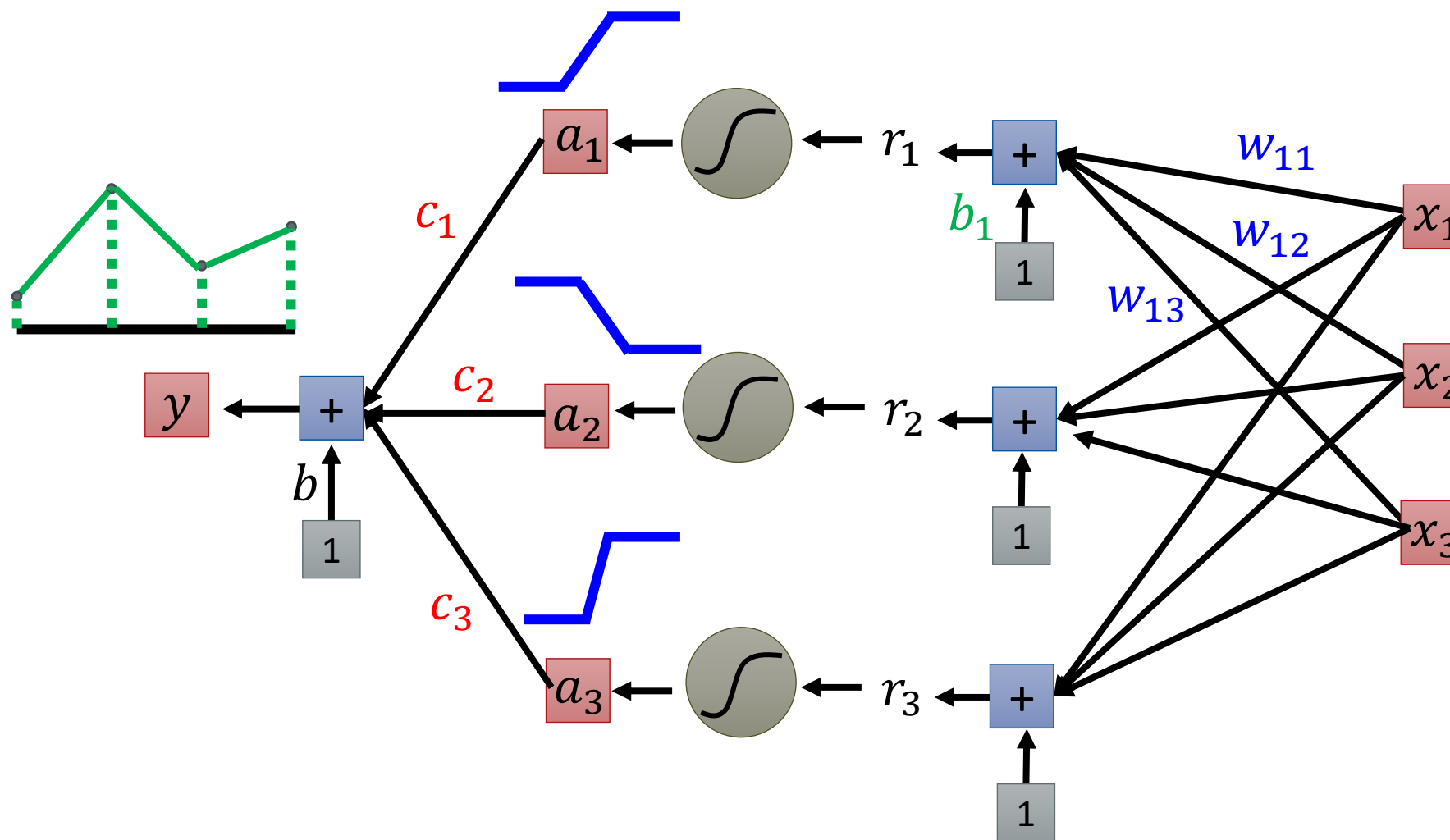


Review



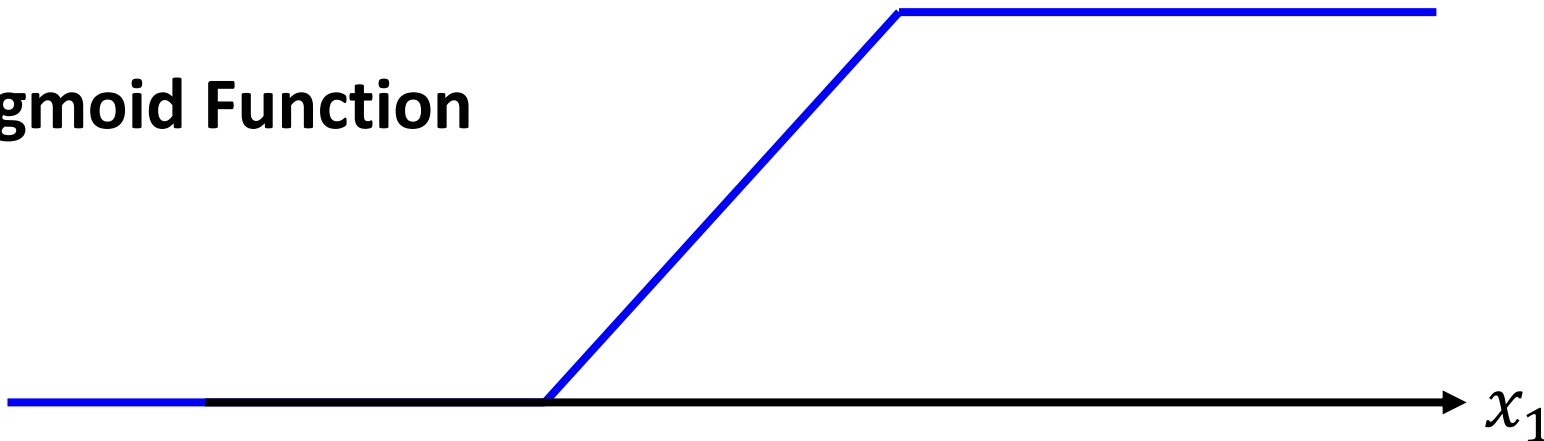
- 深度学习是机器学习的一种技术，即假设输入和输出间的映射是神经网络
- 不同的连接方式会导致不同的网络结构，给定一个网络结构就定义了一个函数族
- 网络参数 θ ：所有神经元中的权重 w 和偏置 b

激活函数

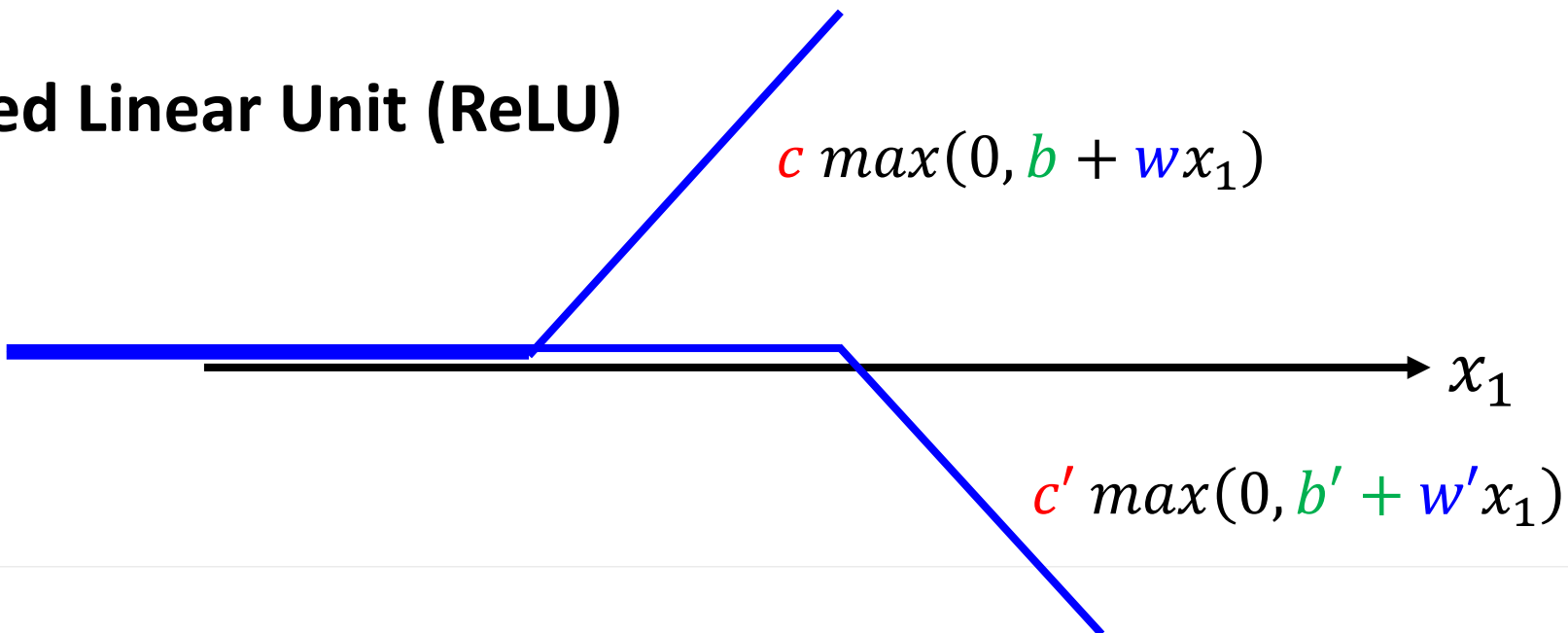


激活函数

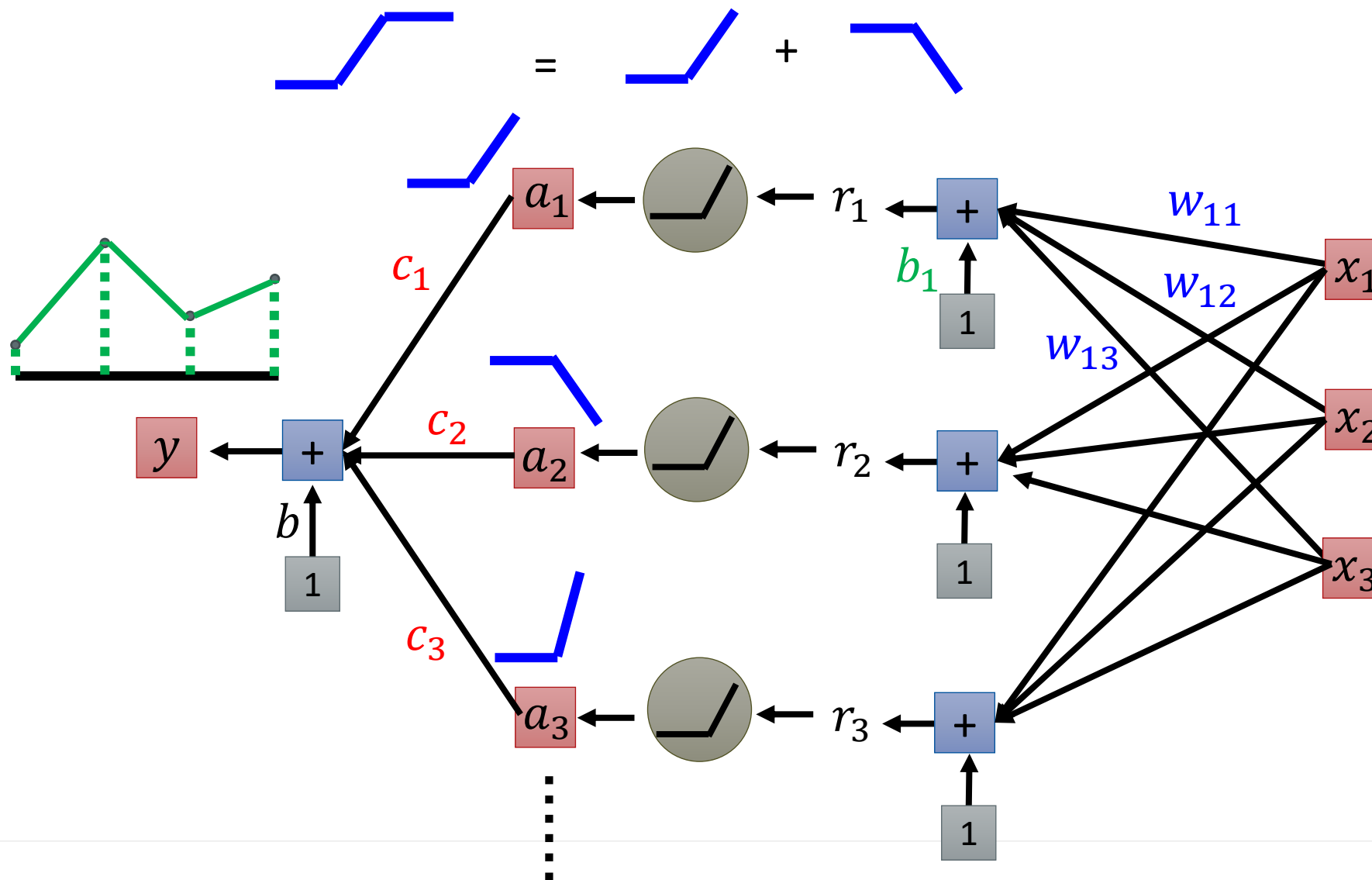
Sigmoid Function



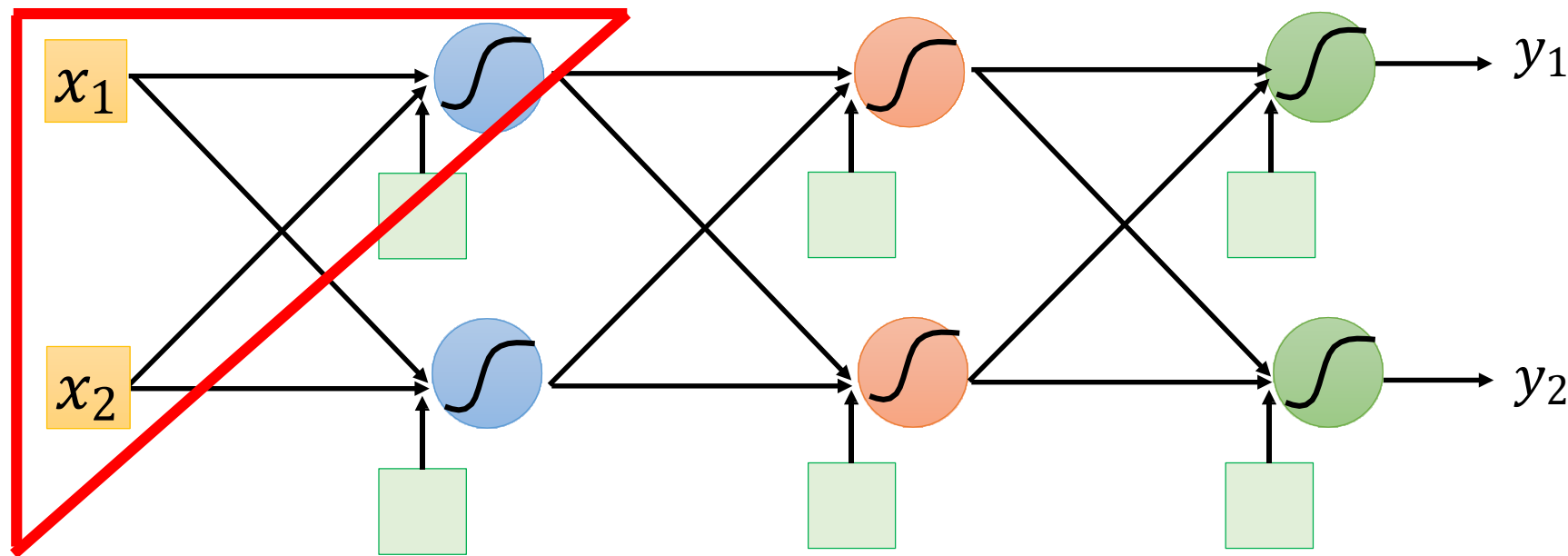
Rectified Linear Unit (ReLU)



激活函数



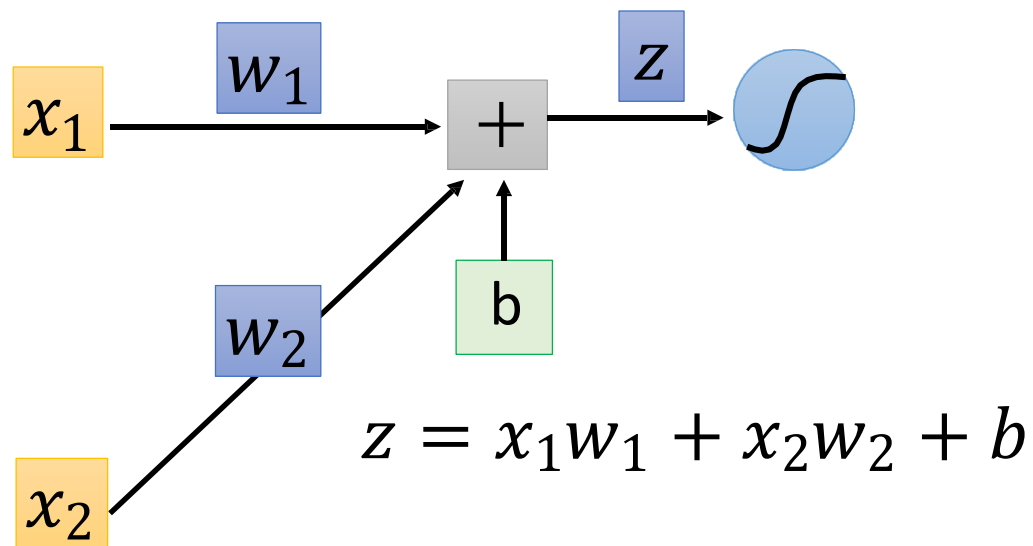
反向传播



$$L = \sum_{n=1}^N C^n$$

$$\frac{\partial L(\theta)}{\partial w} = \sum_{n=1}^N \frac{\partial C^n(\theta)}{\partial w}$$

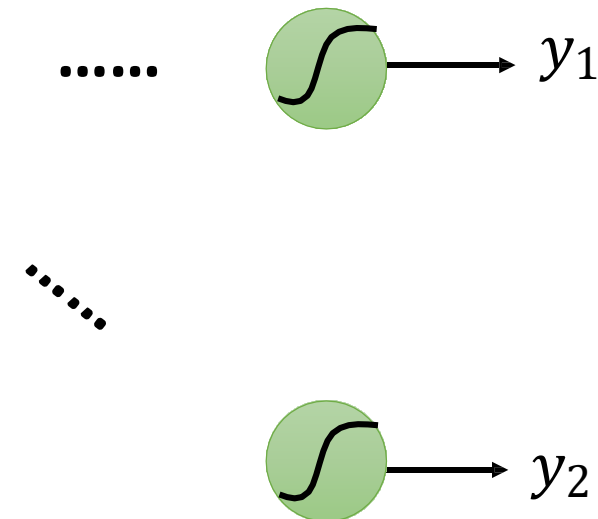
反向传播



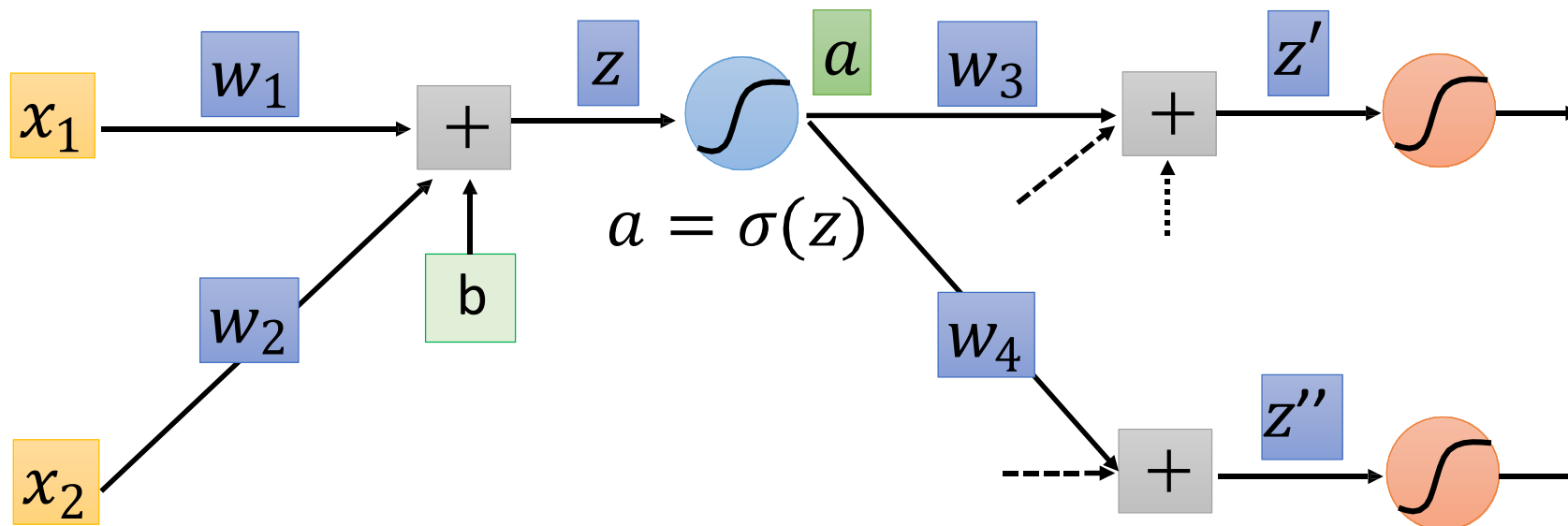
$$\frac{\partial C}{\partial w} = \frac{\partial z}{\partial w} \frac{\partial C}{\partial z}$$

$$\frac{\partial z}{\partial w_1} = x_1$$

$$\frac{\partial z}{\partial w_2} = x_2$$

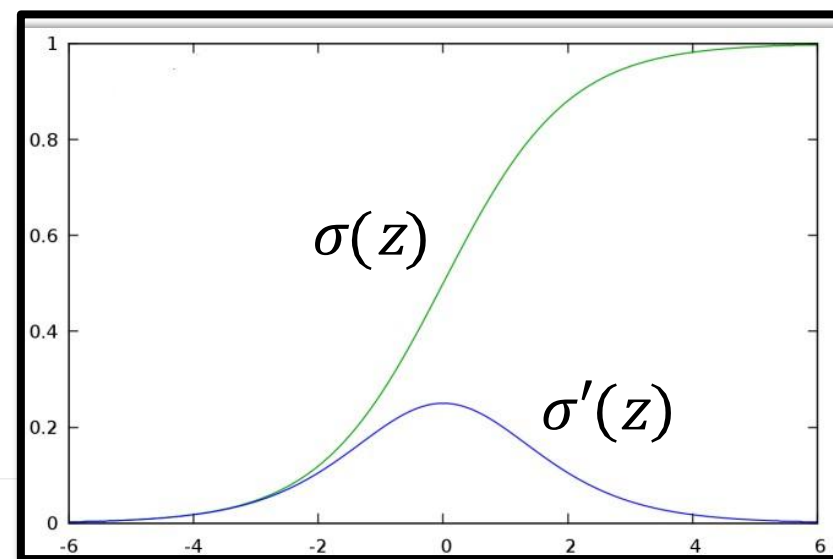


反向传播

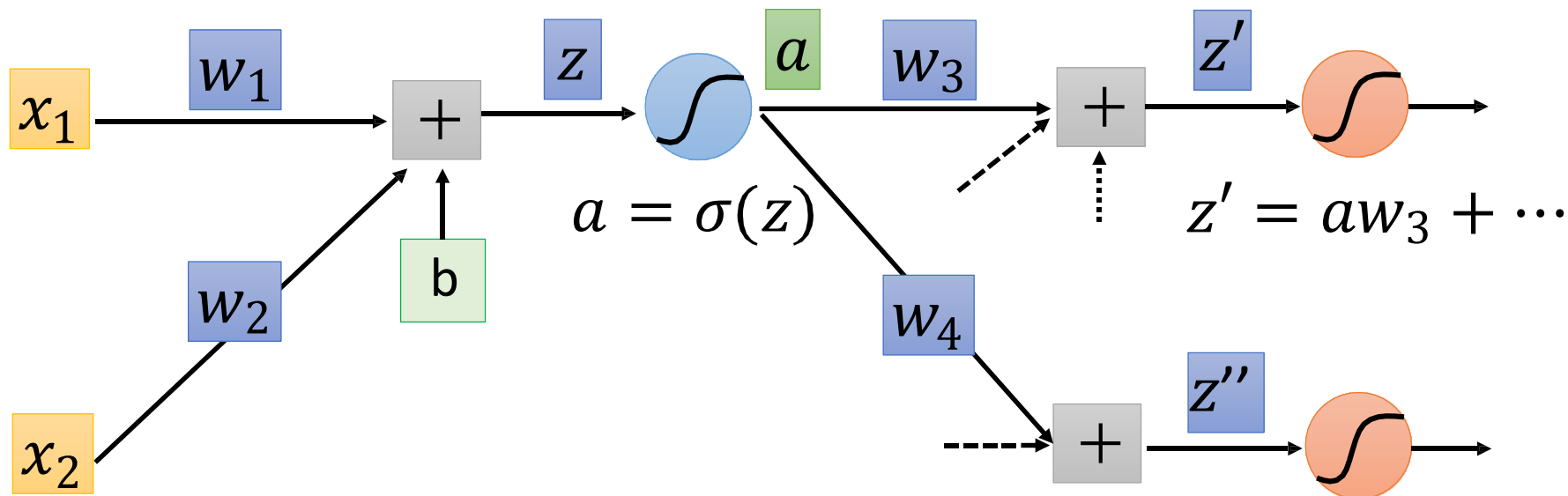


$$\frac{\partial C}{\partial z} = \frac{\partial a}{\partial z} \frac{\partial C}{\partial a}$$

➡ $\sigma'(z)$



反向传播



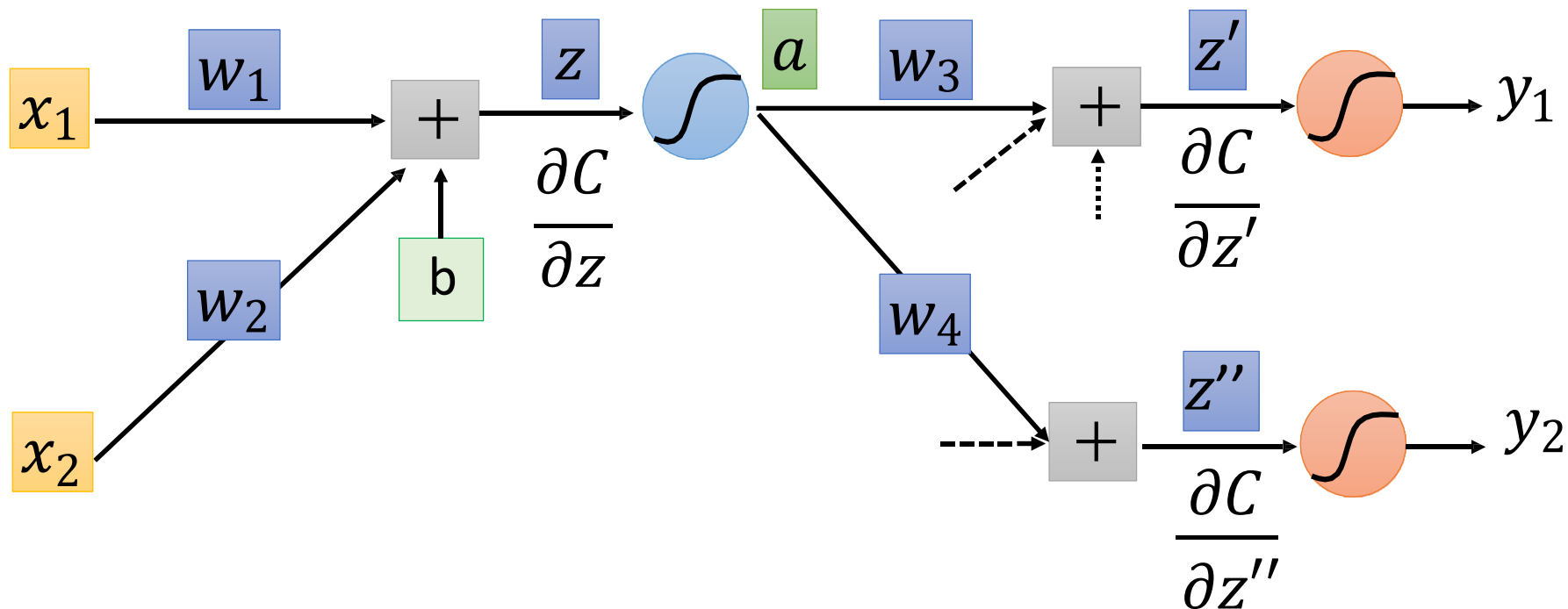
$$\frac{\partial C}{\partial z} = \frac{\partial a}{\partial z} \frac{\partial C}{\partial a}$$

$$\Rightarrow = \frac{\partial z'}{\partial a} \frac{\partial C}{\partial z'} + \frac{\partial z''}{\partial a} \frac{\partial C}{\partial z''}$$

w_3 w_4

$$\frac{\partial C}{\partial z} = \sigma'(z) \left[w_3 \frac{\partial C}{\partial z'} + w_4 \frac{\partial C}{\partial z''} \right]$$

反向传播



输出层:

$$\frac{\partial C}{\partial z'} = \frac{\partial y_1}{\partial z'} \frac{\partial C}{\partial y_1} \quad \frac{\partial C}{\partial z''} = \frac{\partial y_2}{\partial z''} \frac{\partial C}{\partial y_2}$$

神经网络训练技巧

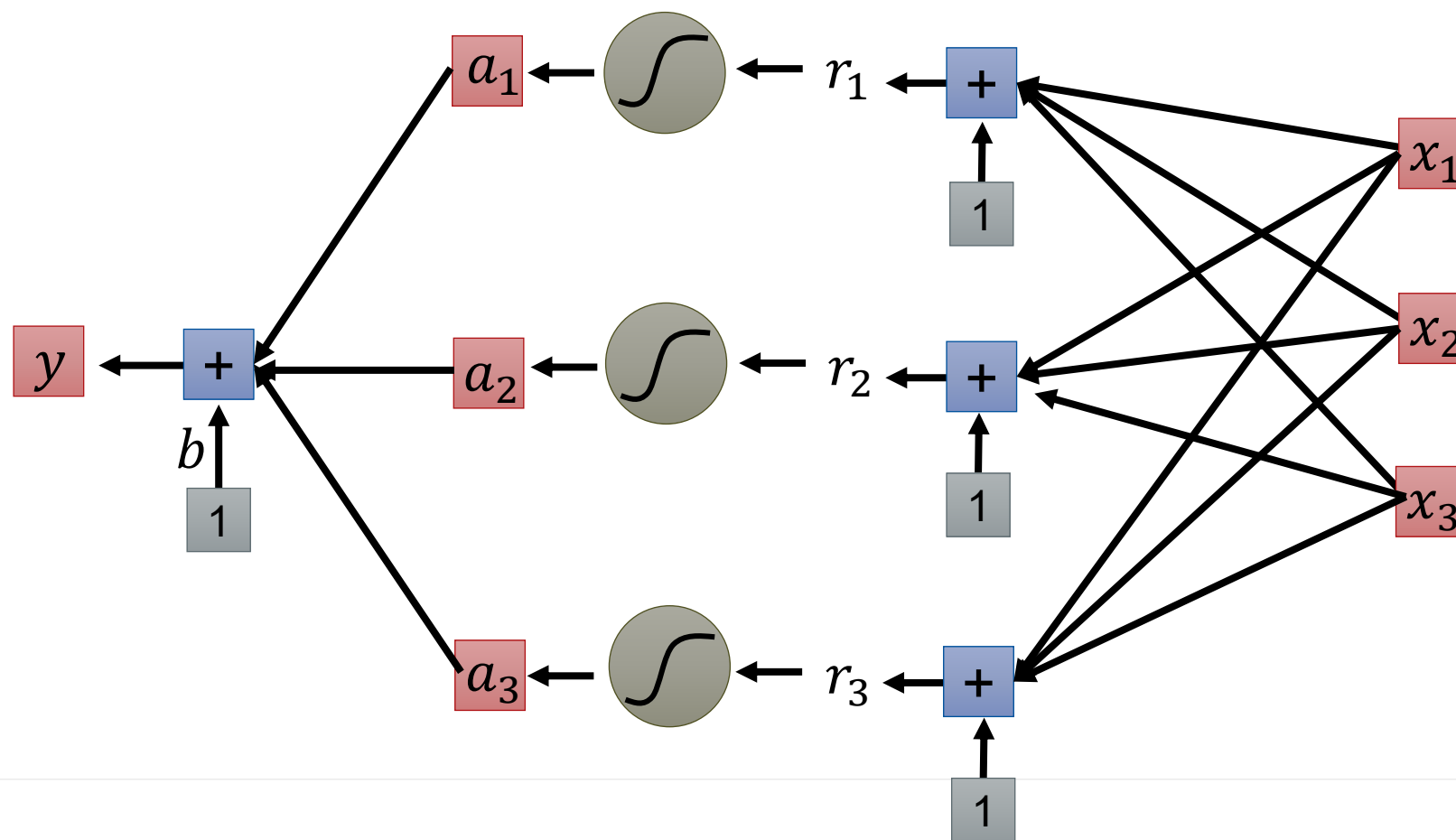


分类损失函数

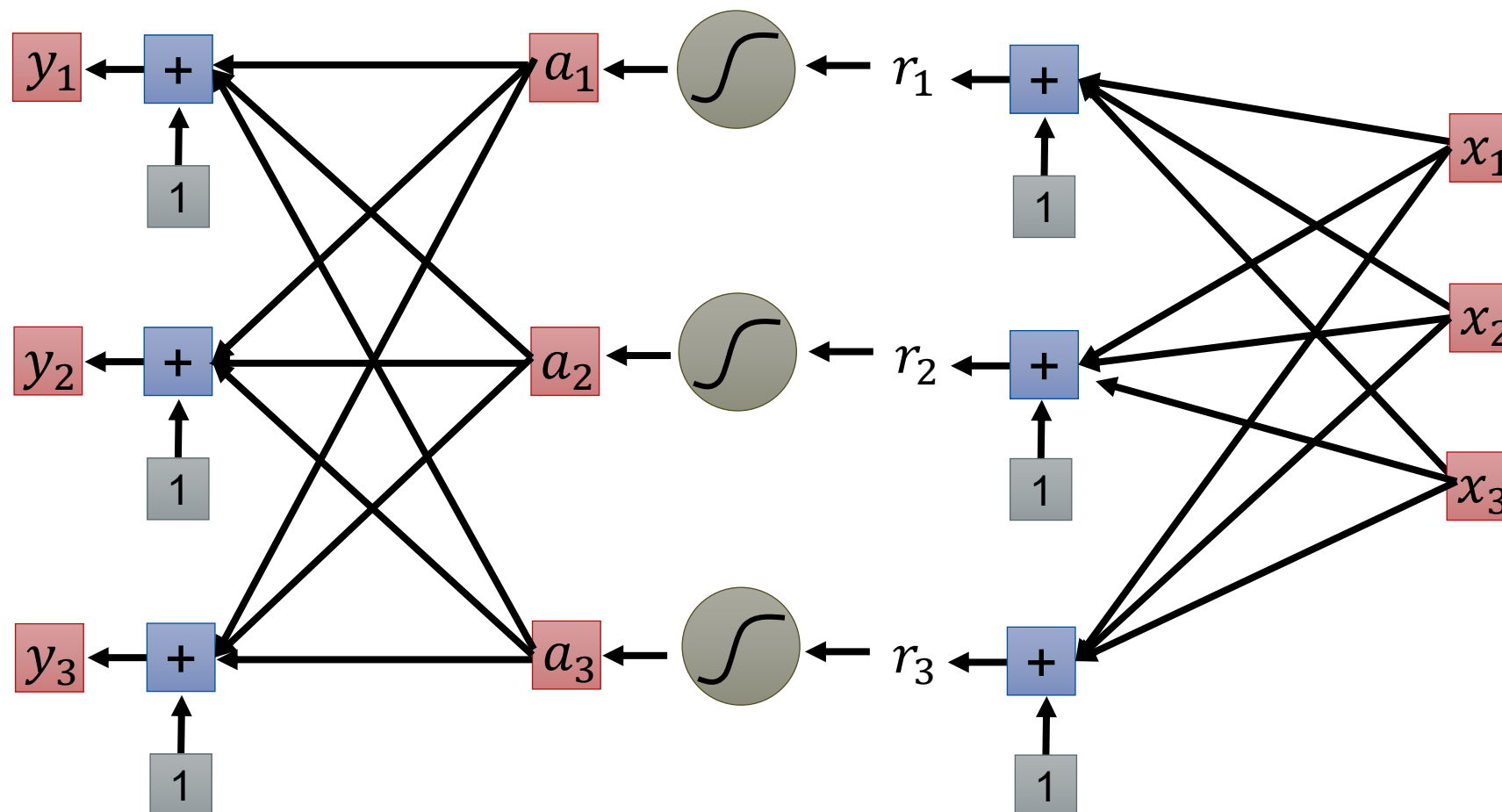
One-hot vector:

$$\hat{y} = \begin{bmatrix} 1 \\ 0 \\ 0 \end{bmatrix} \quad \text{or} \quad \begin{bmatrix} 0 \\ 1 \\ 0 \end{bmatrix} \quad \text{or} \quad \begin{bmatrix} 0 \\ 0 \\ 1 \end{bmatrix}$$

Class 1 Class 2 Class 3



分类损失函数



分类损失函数

$$\mathbf{y} = \mathbf{b}' + \mathbf{W}' \sigma(\mathbf{b} + \mathbf{W} \mathbf{x})$$

label $\hat{\mathbf{y}}$ $\longleftrightarrow$ $\mathbf{y}' = \text{softmax}(\mathbf{y})$

0或1

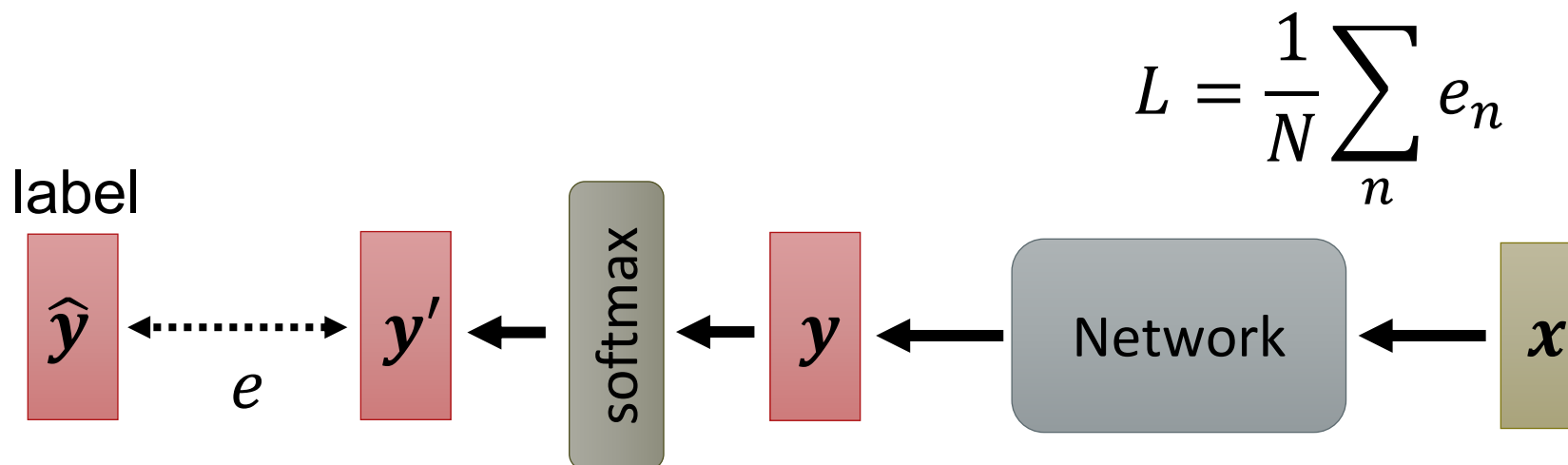
把所有值变到
0到1之间

任何值

$$y'_i = \frac{\exp(y_i)}{\sum_j \exp(y_j)}$$

- $0 < y'_i < 1$
- $\sum_i y'_i = 1$

分类损失函数



Mean Square Error (MSE)

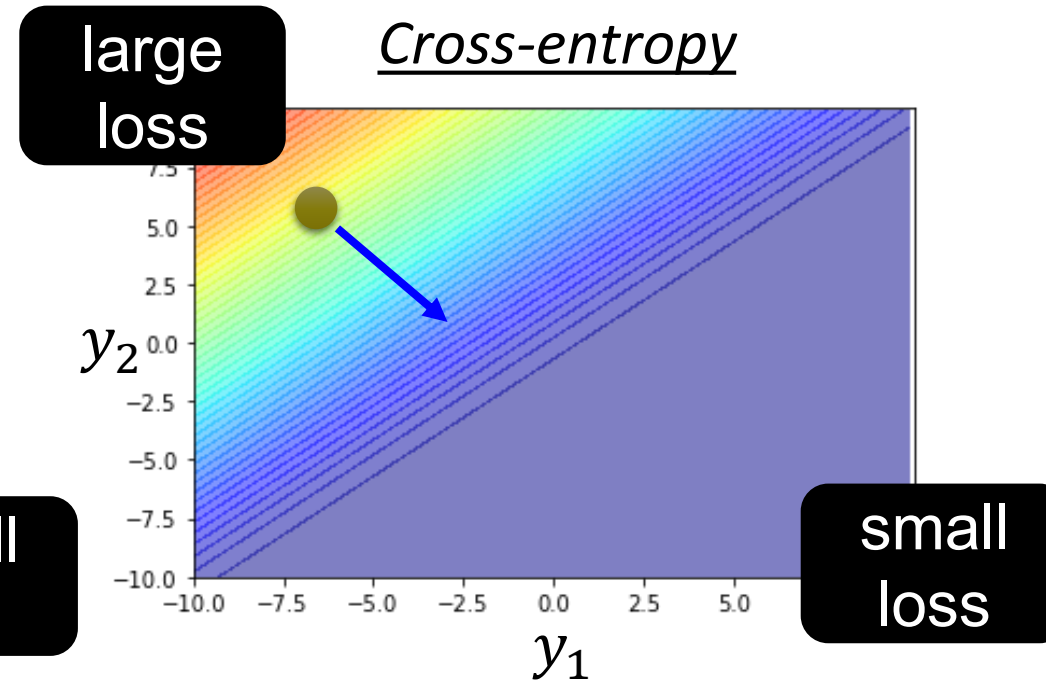
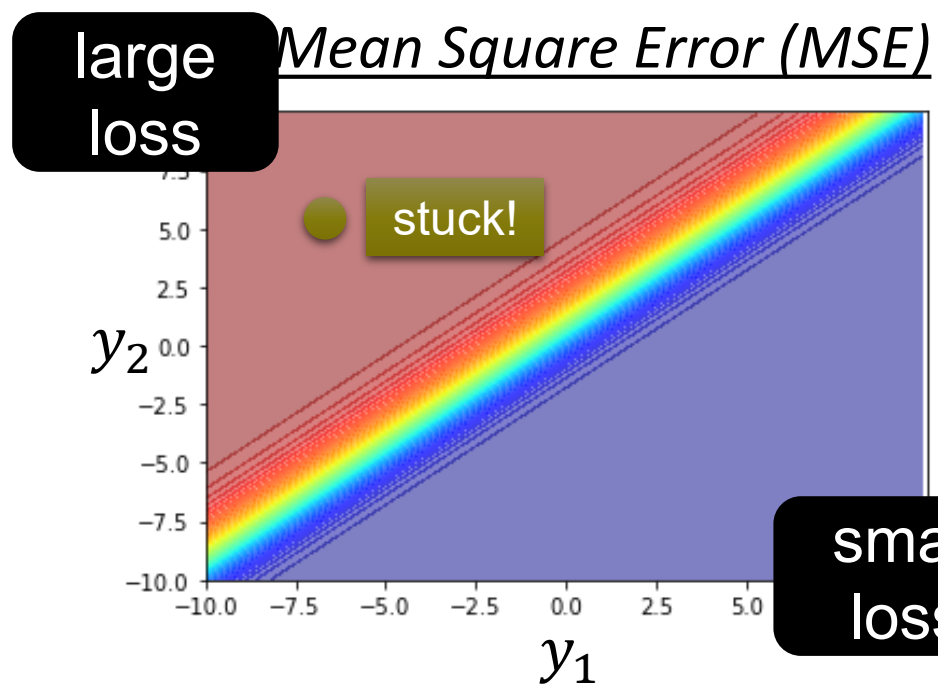
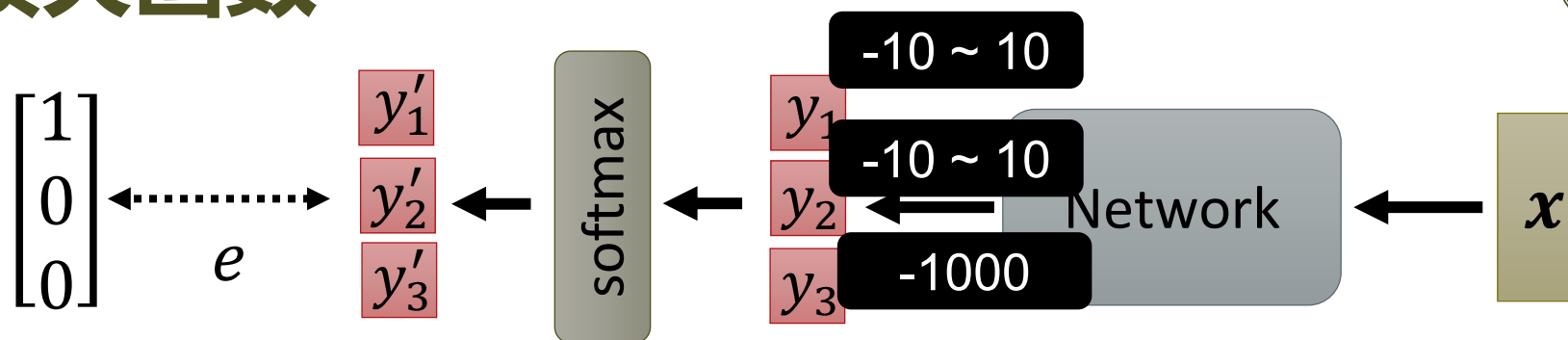
$$e = \sum_i (\hat{y}_i - y'_i)^2$$

Cross-entropy



$$e = - \sum_i \hat{y}_i \ln y'_i$$

分类损失函数



复习：梯度下降

形式化地说，在步骤3中，我们需要解决以下优化问题：

$$\theta^* = \arg \min_{\theta} L(\theta) \quad L: \text{loss function} \quad \theta: \text{parameters}$$

假设 θ 有两个值 $\{\theta_1 \theta_2\}$

从一个随机的初始值 $\theta^0 = \begin{bmatrix} \theta_1^0 \\ \theta_2^0 \end{bmatrix}$ 开始

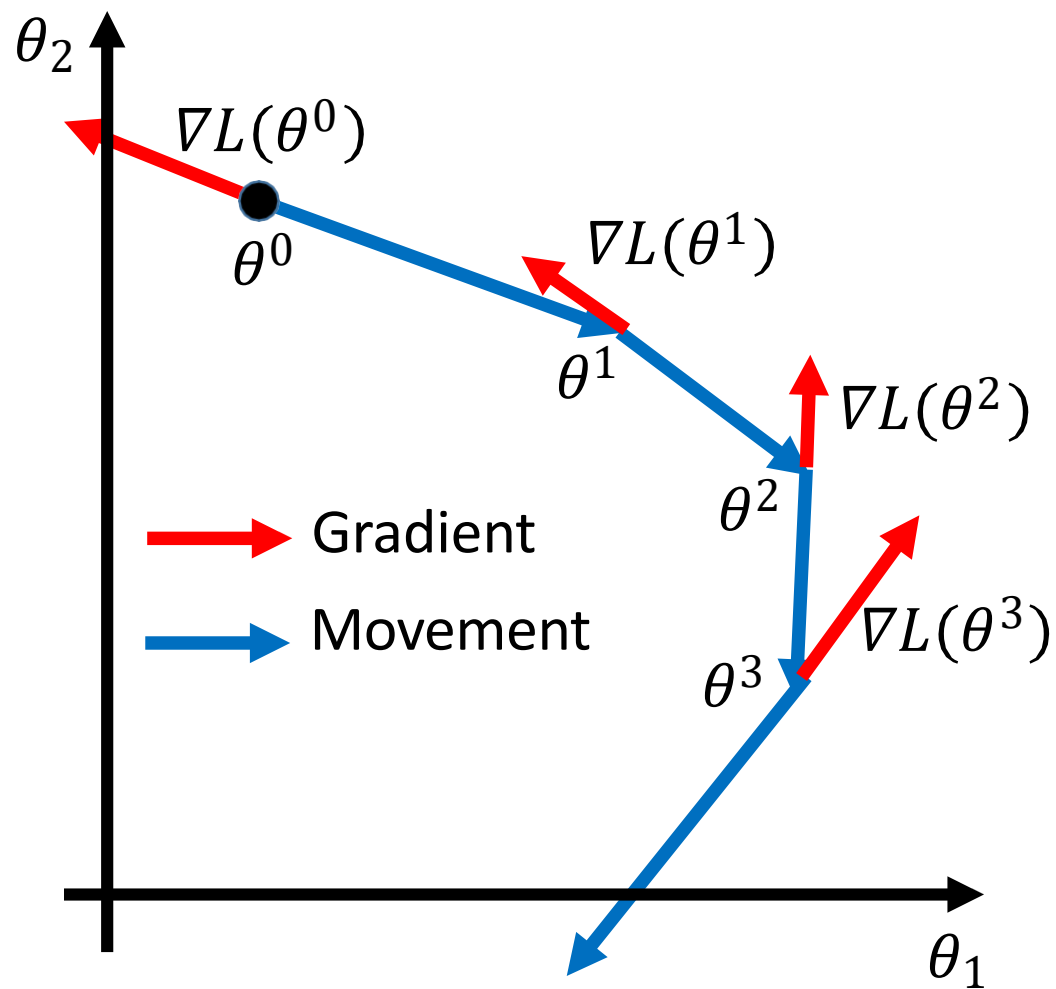
迭代

$$\nabla L(\theta) = \begin{bmatrix} \partial L(\theta_1)/\partial \theta_1 \\ \partial L(\theta_2)/\partial \theta_2 \end{bmatrix}$$

$$\begin{bmatrix} \theta_1^1 \\ \theta_2^1 \end{bmatrix} = \begin{bmatrix} \theta_1^0 \\ \theta_2^0 \end{bmatrix} - \eta \begin{bmatrix} \partial L(\theta_1^0)/\partial \theta_1 \\ \partial L(\theta_2^0)/\partial \theta_2 \end{bmatrix}$$

$$\begin{bmatrix} \theta_1^2 \\ \theta_2^2 \end{bmatrix} = \begin{bmatrix} \theta_1^1 \\ \theta_2^1 \end{bmatrix} - \eta \begin{bmatrix} \partial L(\theta_1^1)/\partial \theta_1 \\ \partial L(\theta_2^1)/\partial \theta_2 \end{bmatrix}$$

复习：梯度下降



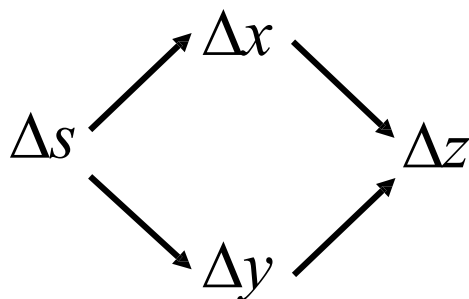
反向传播

$$y = g(x) \quad z = h(y)$$

$$\Delta x \rightarrow \Delta y \rightarrow \Delta z$$

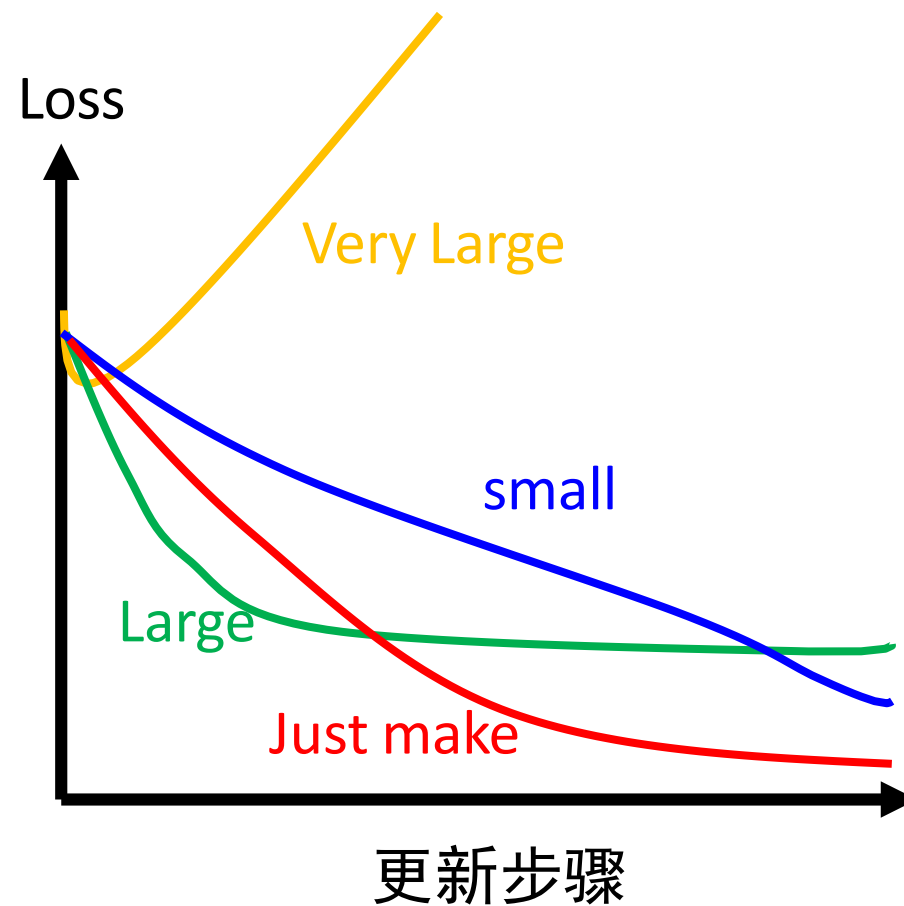
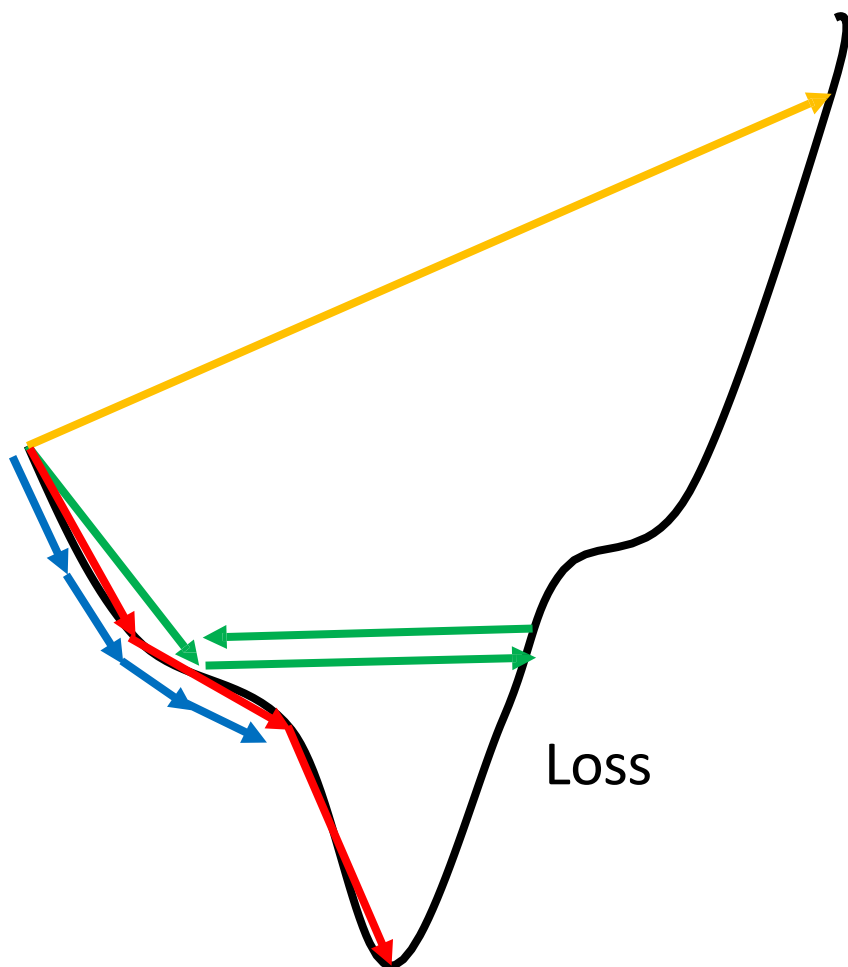
$$\frac{dz}{dx} = \frac{dz}{dy} \frac{dy}{dx}$$

$$x = g(s) \quad y = h(s) \quad z = k(x, y)$$



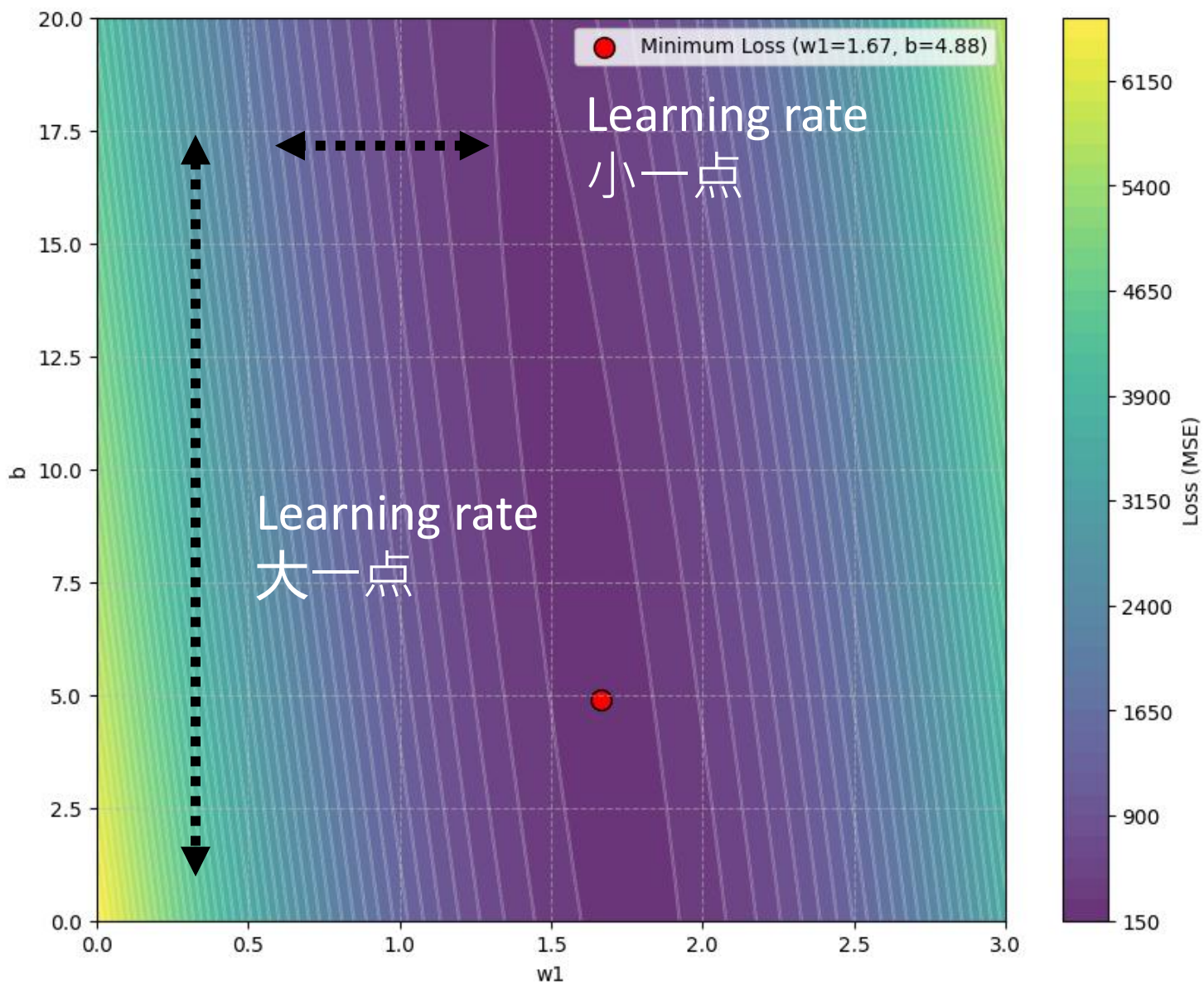
$$\frac{dz}{ds} = \frac{\partial z}{\partial x} \frac{dx}{ds} + \frac{\partial z}{\partial y} \frac{dy}{ds}$$

学习率



学习率

不同参数应该有不同的Learning Rate



将每个参数的学习率除以其先前导数的均方根

Vanilla梯度下降: $w^{t+1} \leftarrow w^t - \eta^t g^t$

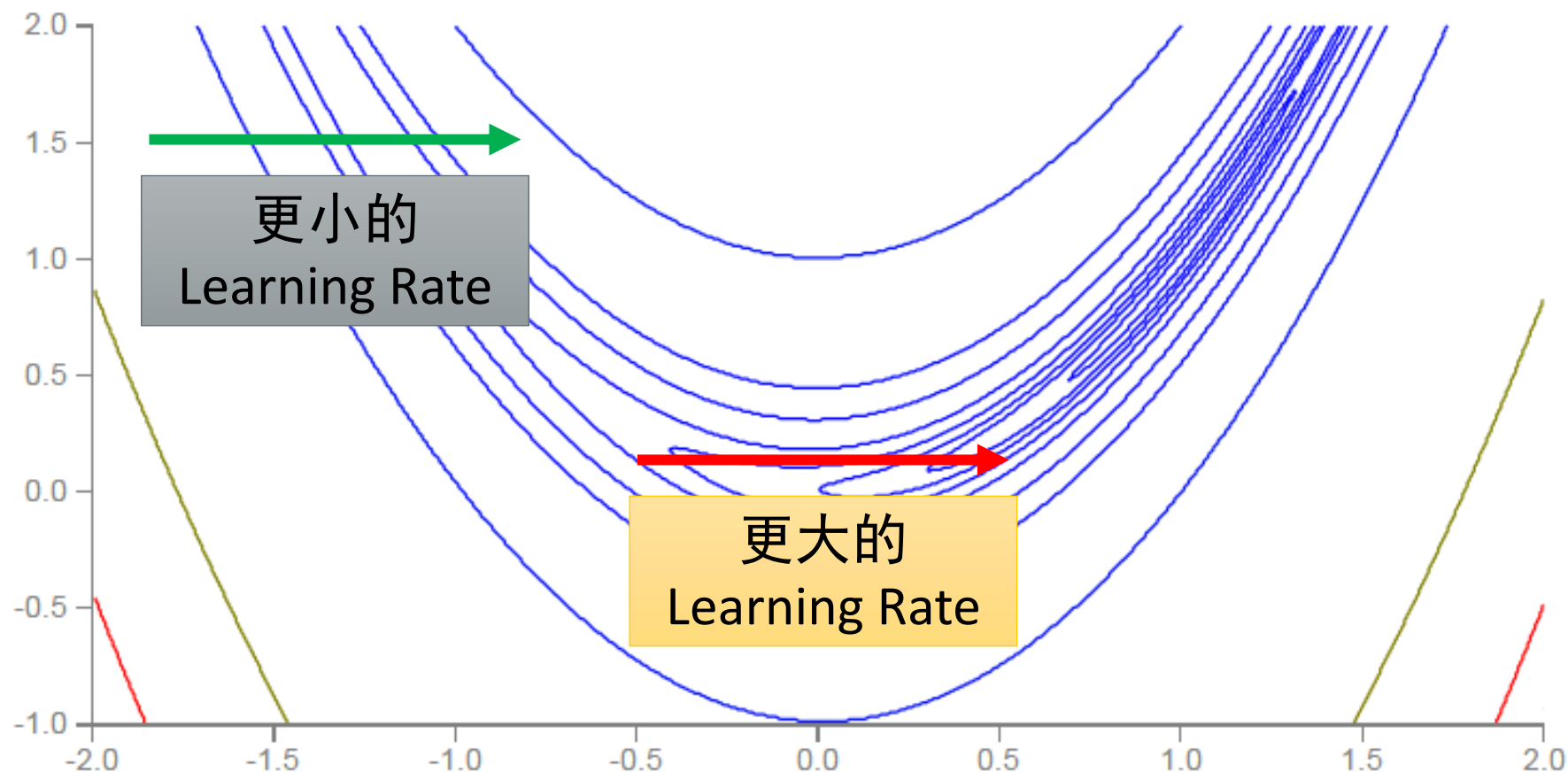
Adagrad: $w^{t+1} \leftarrow w^t - \frac{\eta^t}{\sigma^t} g^t$

参数空间更为平缓的方向，会取得更大的进步（因为平缓，所以历史梯度平方和较小，对应学习下降的幅度较小），并且能够使得陡峭的方向变得平缓，从而加快训练速度

Adagrad

$$\begin{aligned} g^0 &= \nabla L(\theta^0) & \sigma_i^0 &= \sqrt{(g_i^0)^2} = |g_i^0| & \theta_i^1 &\leftarrow \theta_i^0 - \frac{\eta}{\sigma_i^0} g_i^0 \\ g^1 &= \nabla L(\theta^1) & \sigma_i^1 &= \sqrt{[(g_i^0)^2 + (g_i^1)^2]} & \theta_i^2 &\leftarrow \theta_i^1 - \frac{\eta}{\sigma_i^1} g_i^1 \\ g^2 &= \nabla L(\theta^2) & \sigma_i^2 &= \sqrt{[(g_i^0)^2 + (g_i^1)^2 + (g_i^2)^2]} & \theta_i^3 &\leftarrow \theta_i^2 - \frac{\eta}{\sigma_i^2} g_i^2 \\ g^t &= \nabla L(\theta^t) & \sigma_i^t &= \sqrt{\sum_{i=0}^t (g_i^t)^2} & \theta_i^{t+1} &\leftarrow \theta_i^t - \frac{\eta}{\sigma_i^t} g_i^t \end{aligned}$$

同一个参数的Learning Rate不是一成不变的



RMSProp

$$0 < \alpha < 1$$

$$\sigma_i^0 = \sqrt{(g_i^0)^2}$$

$$\theta_i^1 \leftarrow \theta_i^0 - \frac{\eta}{\sigma_i^0} g_i^0$$

$$\sigma_i^1 = \sqrt{\alpha(\sigma_i^0)^2 + (1 - \alpha)(g_i^1)^2}$$

$$\theta_i^2 \leftarrow \theta_i^1 - \frac{\eta}{\sigma_i^1} g_i^1$$

$$\sigma_i^2 = \sqrt{\alpha(\sigma_i^1)^2 + (1 - \alpha)(g_i^2)^2}$$

$$\theta_i^3 \leftarrow \theta_i^2 - \frac{\eta}{\sigma_i^2} g_i^2$$

$$\sigma_i^t = \sqrt{\alpha(\sigma_i^{t-1})^2 + (1 - \alpha)(g_i^t)^2}$$

$$\theta_i^{t+1} \leftarrow \theta_i^t - \frac{\eta}{\sigma_i^t} g_i^t$$

Momentum

$$0 < \beta < 1$$

$$m_i^0 = g_i^0$$

$$\theta_i^1 \leftarrow \theta_i^0 - \eta m_i^0$$

$$m_i^1 = \beta m_i^0 + (1 - \beta) g_i^1$$

$$\theta_i^2 \leftarrow \theta_i^1 - \eta m_i^1$$

$$m_i^2 = \beta m_i^1 + (1 - \beta) g_i^2$$

$$\theta_i^3 \leftarrow \theta_i^2 - \eta m_i^2$$

$$m_i^t = \beta m_i^{t-1} + (1 - \beta) g_i^t$$

$$\theta_i^{t+1} \leftarrow \theta_i^t - \eta m_i^t$$

Adam: RMSProp + Momentum

Compute gradient $g^t = \nabla L(\theta^t)$

Momentum

For each dimension i : $m_i^t = \beta m_i^{t-1} + (1 - \beta) g_i^t$

$$\theta_i^{t+1} \leftarrow \theta_i^t - \eta m_i^t$$

RMSProp

For each dimension i : $\sigma_i^t = \sqrt{\alpha (\sigma_i^{t-1})^2 + (1 - \alpha) (g_i^t)^2}$

$$\theta_i^{t+1} \leftarrow \theta_i^t - \frac{\eta}{\sigma_i^t} g_i^t$$

Adam

$$\theta_i^{t+1} \leftarrow \theta_i^t - \frac{\eta}{\sigma_i^t} m_i^t$$

$$L = \sum_{x \sim \text{Train}} l(f(x), \hat{y}) \quad \cdots \cdots \cdots \rightarrow \quad L = \sum_{x \sim \text{Test}} l(f(x), \hat{y})$$

收集更多数据



数据增强

$$L = \sum_{x \sim \text{Train}} l(f(x), \hat{y}) \quad \cdots \cdots \cdots \rightarrow \quad L = \sum_{x \sim \text{Test}} l(f(x), \hat{y})$$

收集更多数据



右



翻转

右

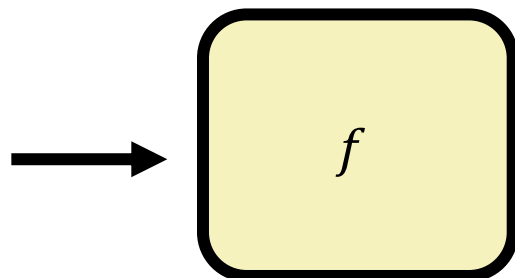


模糊

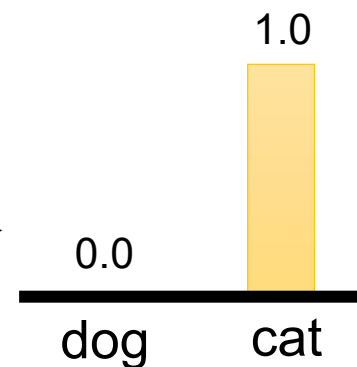
右

数据增强

- mixup

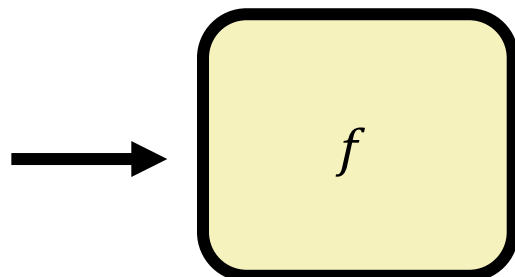


???

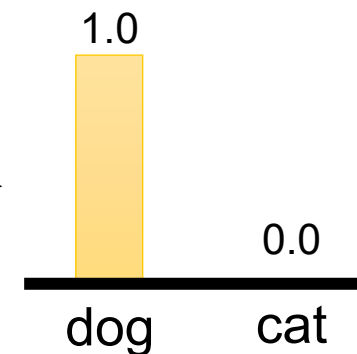


x
0.5

+

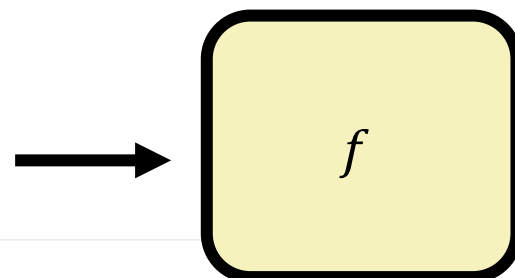


???

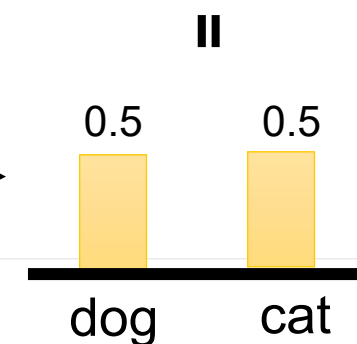


x
0.5

||



???



||

半监督学习



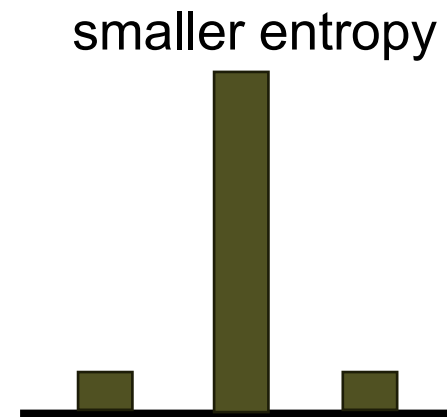
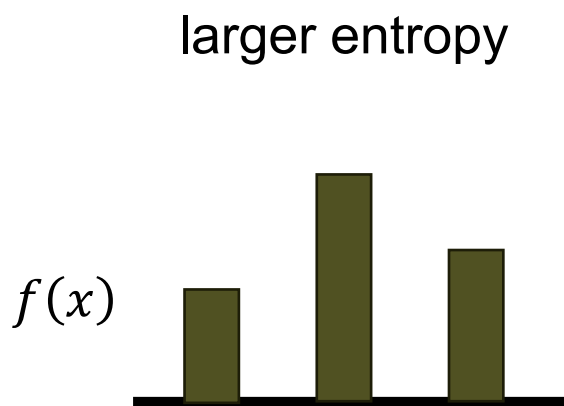
海量数据

$$L = \sum_{x \sim \text{Train}} l(f(x), \hat{y}) + \lambda \sum_{x \sim \text{Unlabel}} \text{?????}$$

半监督学习

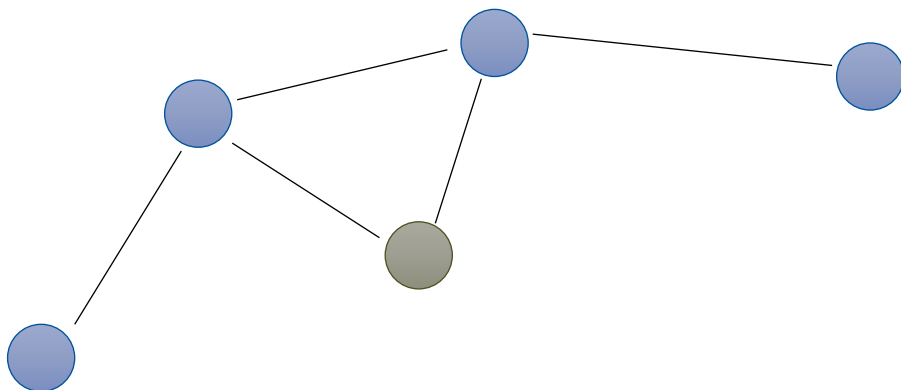
$$L = \sum_{x \sim \text{Train}} l(f(x), \hat{y}) + \lambda \sum_{x \sim \text{Unlabel}} l'(f(x))$$

↓
entropy

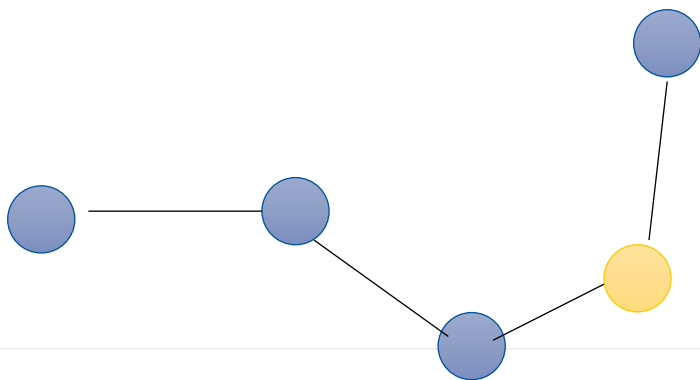


半监督学习

$$L = \sum_{x \sim \text{Train}} l(f(x), \hat{y}) + \lambda \sum_{x, x' \sim \text{Unlabel}} g(x, x') l'(f(x), f(x'))$$



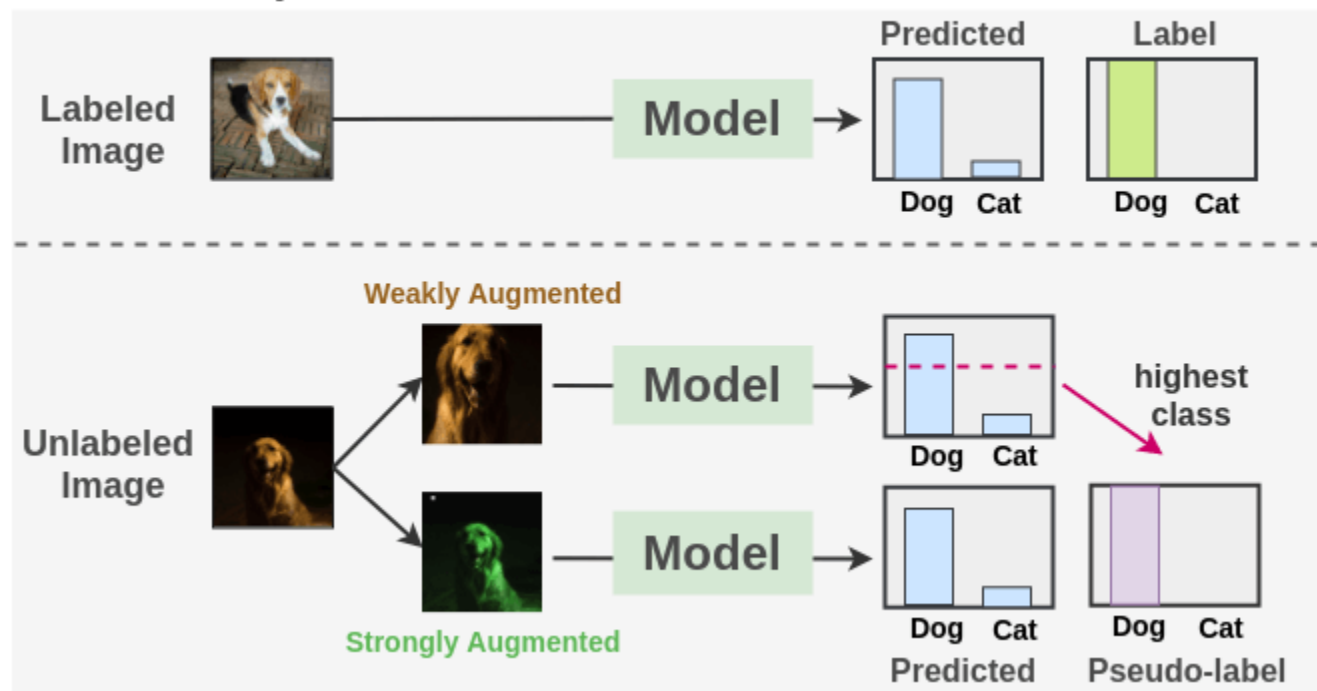
$$g(x, x') = \begin{cases} 1 & \text{connect} \\ 0 & \text{otherwise} \end{cases}$$



$l'(f(x), f(x'))$:

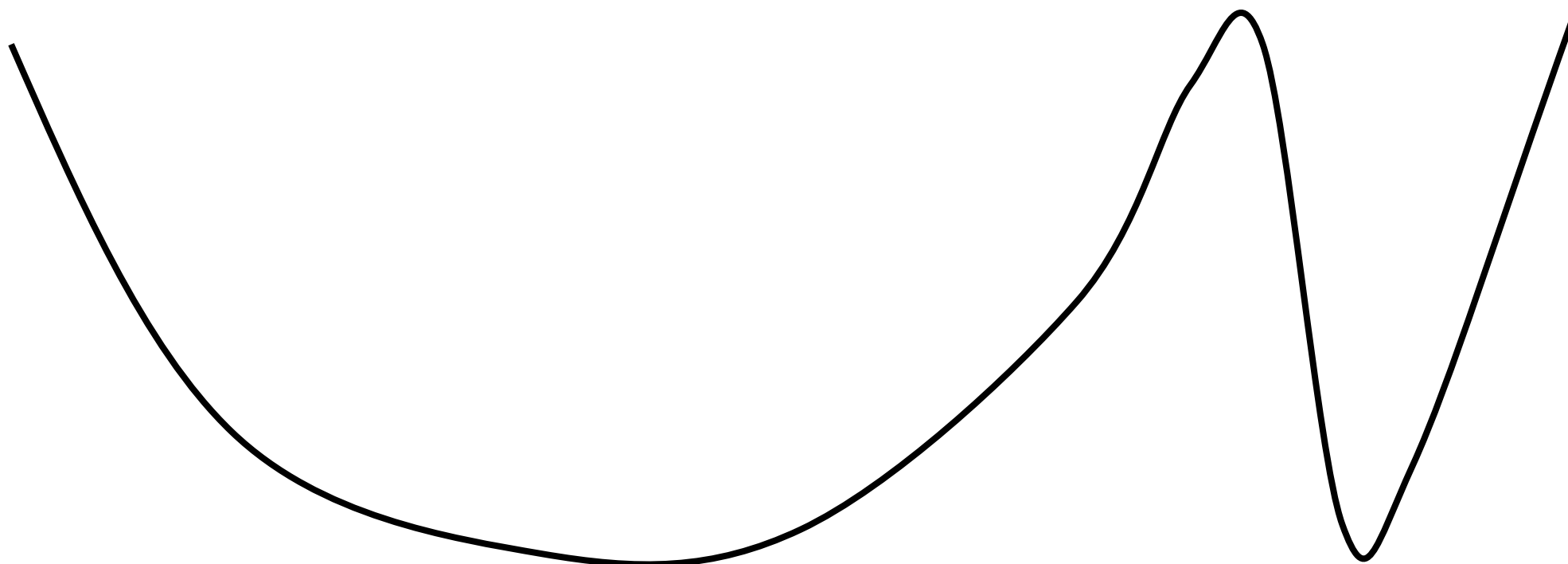
$f(x), f(x')$ 的距离

FixMatch Pipeline



初始化

不同的起始位置会决定我们走到哪里



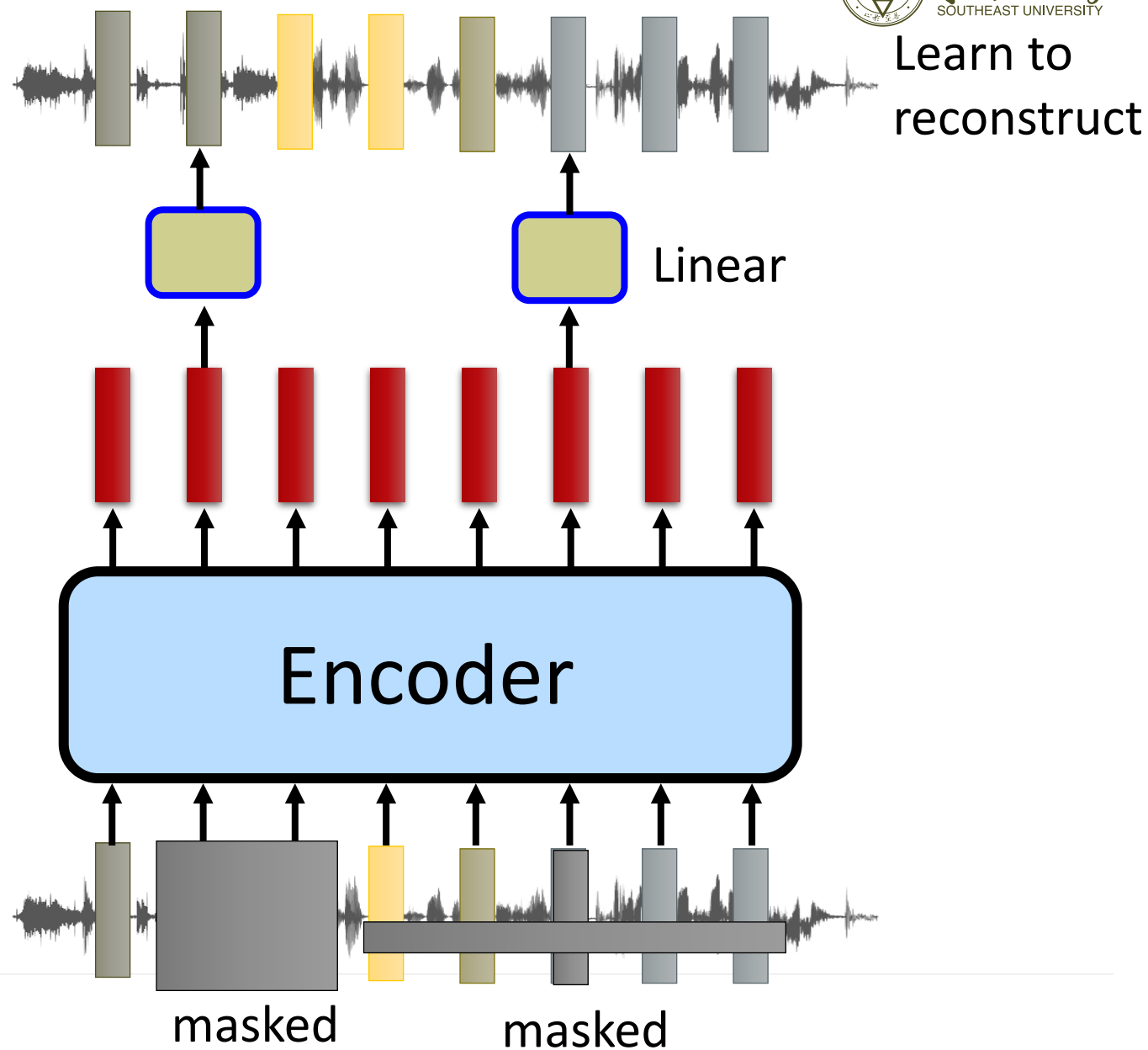
预训练

1. 生成式方法



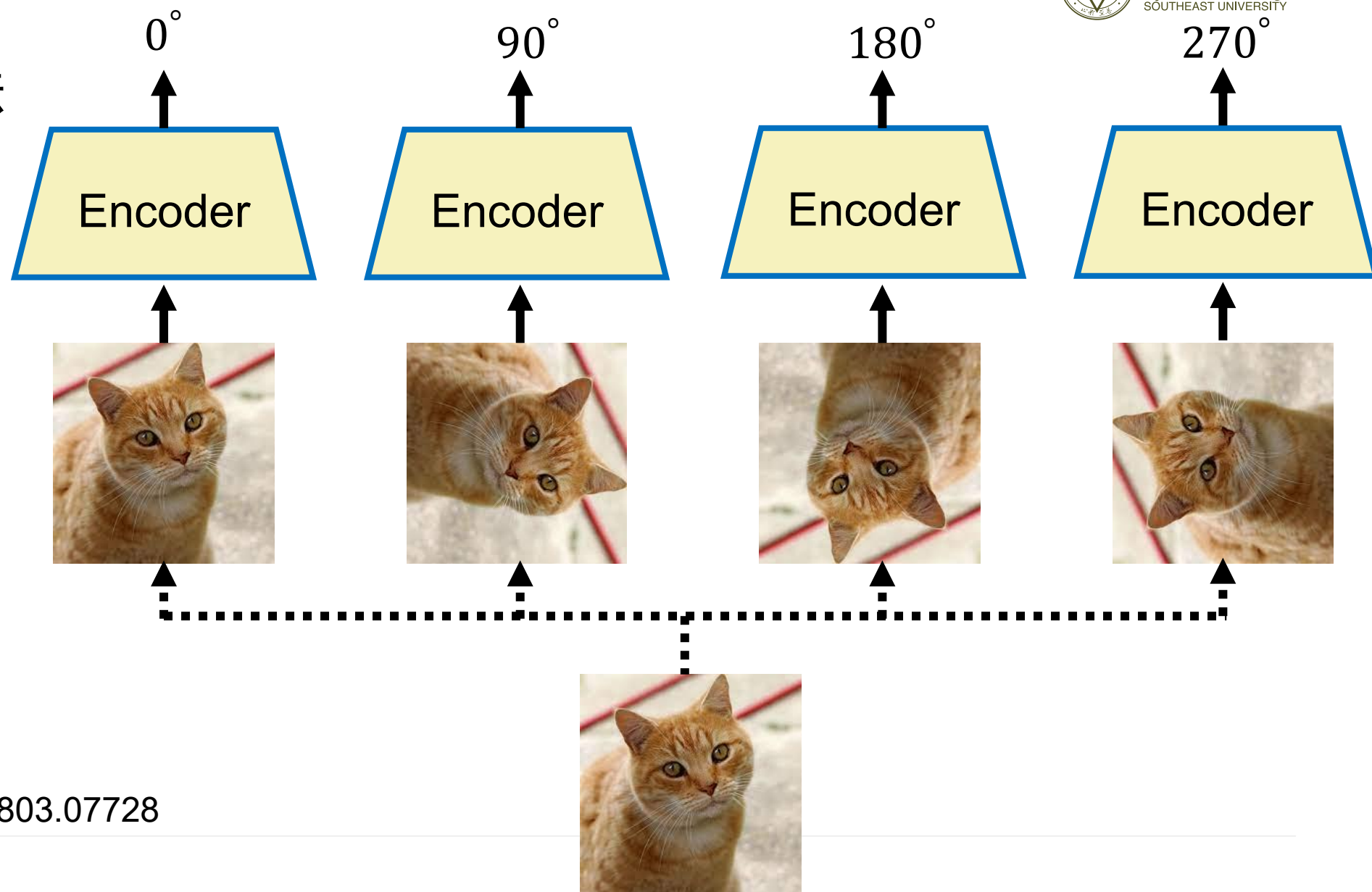
<https://arxiv.org/abs/2007.06028>

止於至善



预训练

2. 预测方法

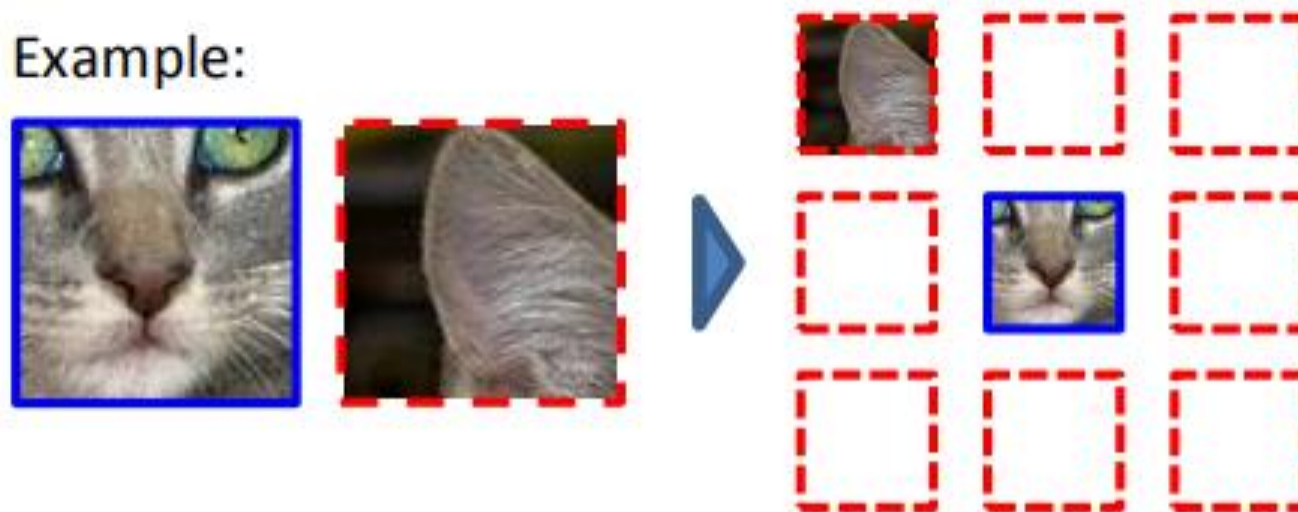


<https://arxiv.org/abs/1803.07728>

预训练

2. 预测方法

Example:



Question 1:



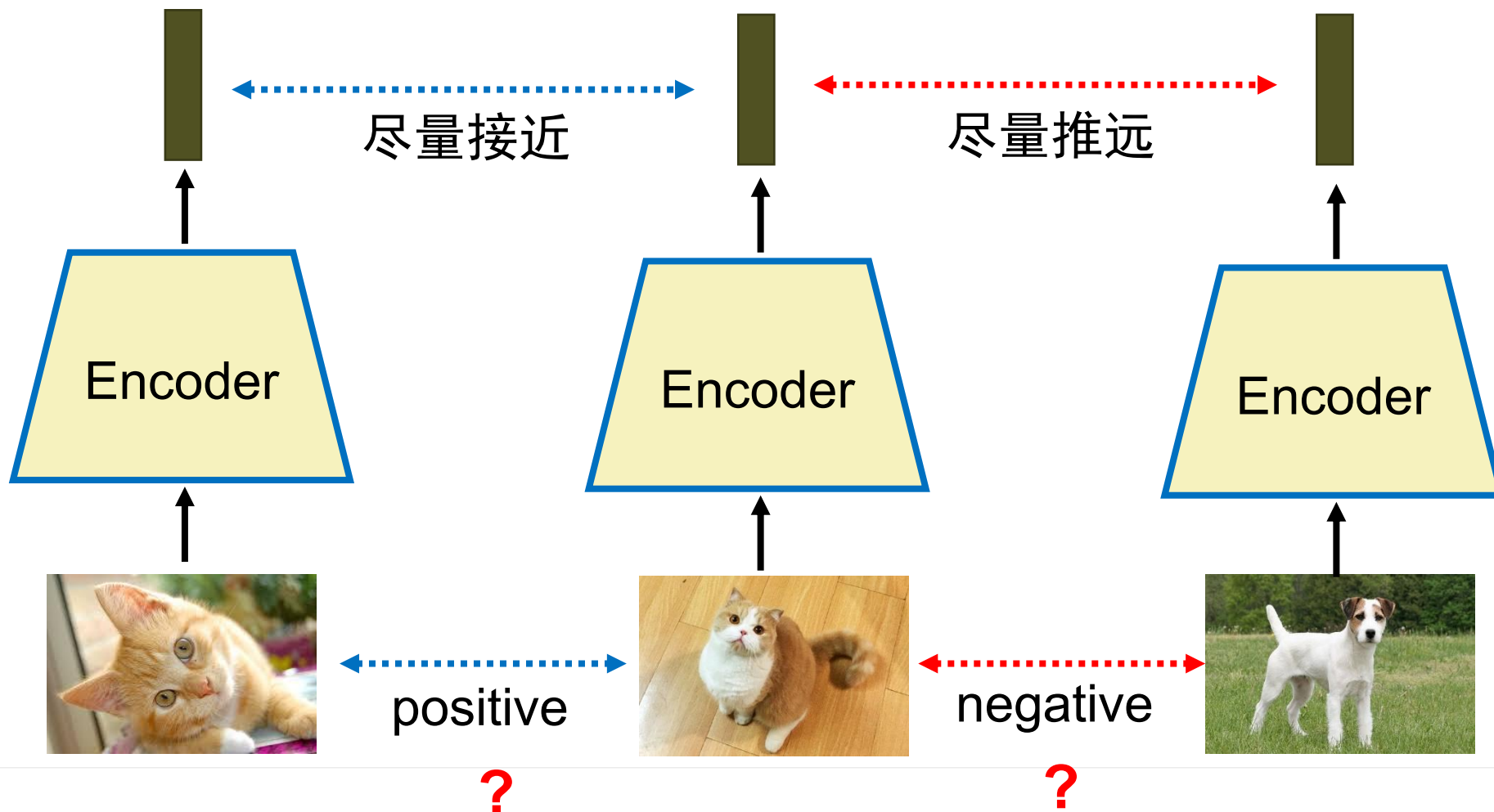
Question 2:



<https://arxiv.org/abs/1505.05192>

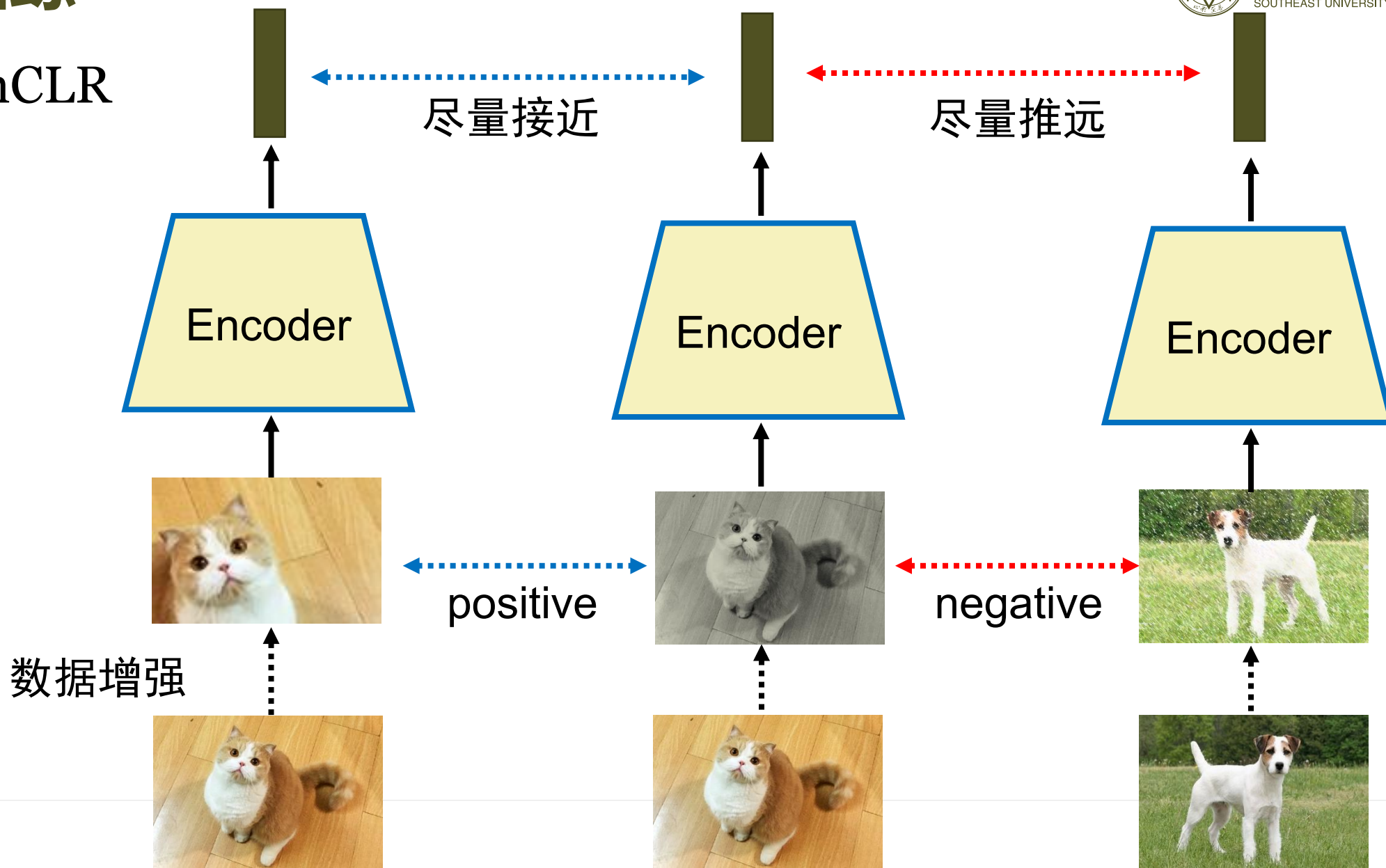
3. 对比学习：通过对比正负样本对来提取有意义表征

预训练



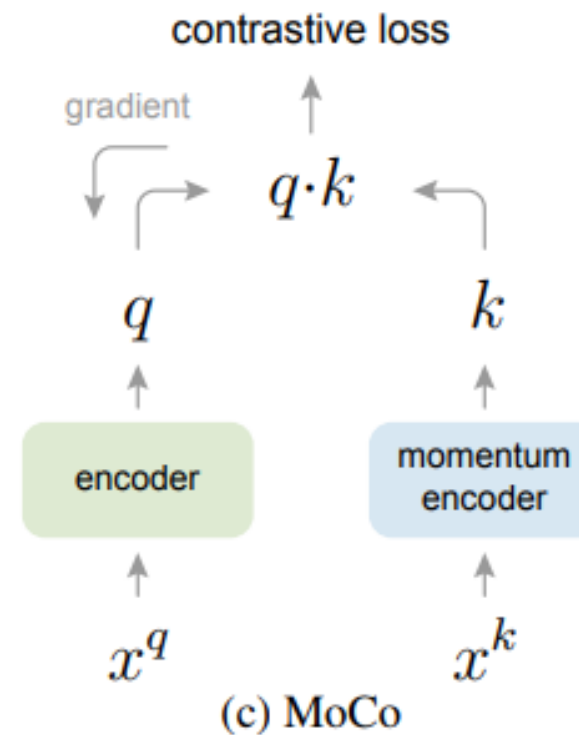
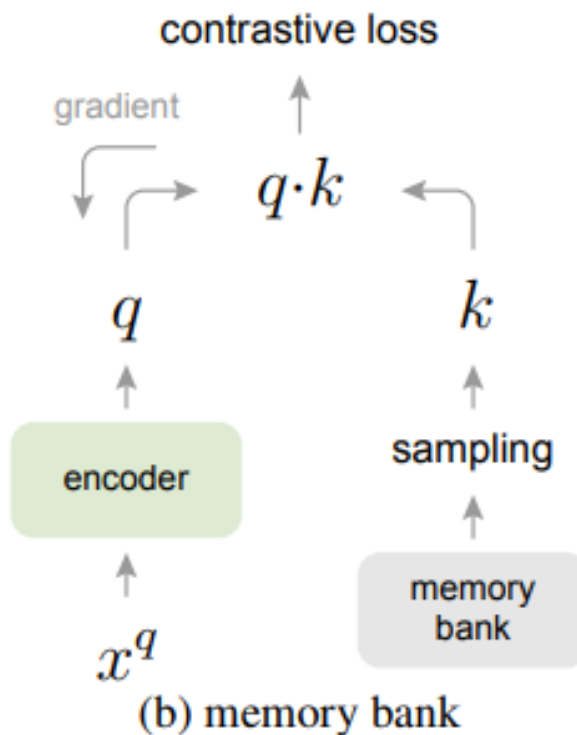
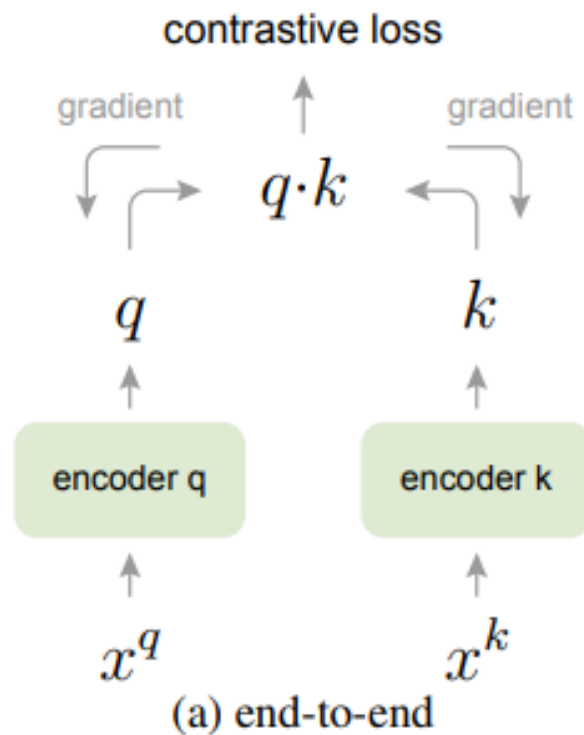
预训练

- SimCLR



预训练

- MoCo/MoCo v2



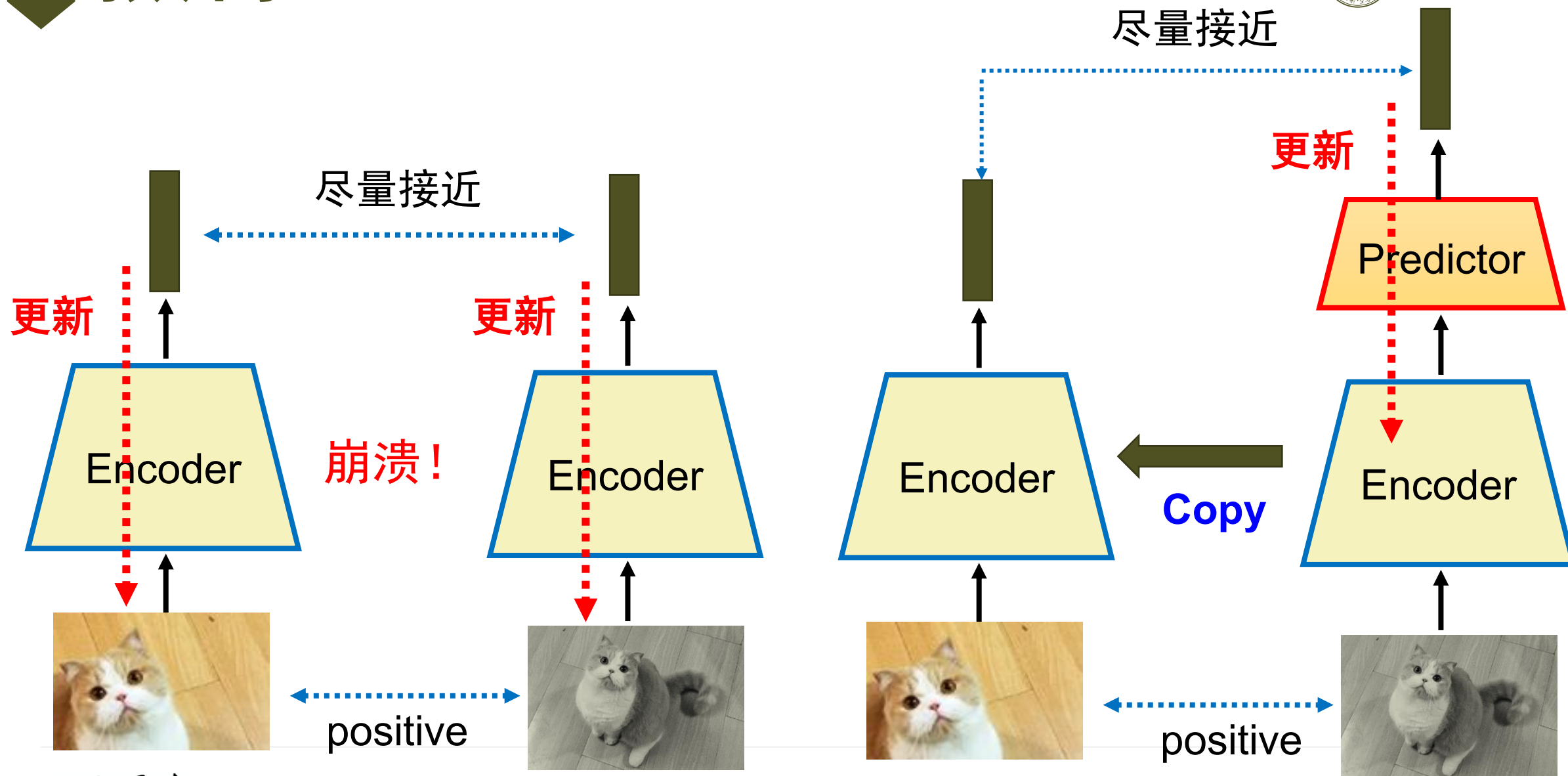
<https://arxiv.org/abs/2003.04297>

选取负面案例并非易事，这些负面案例必须足够具有代表性，但又不能过于极端

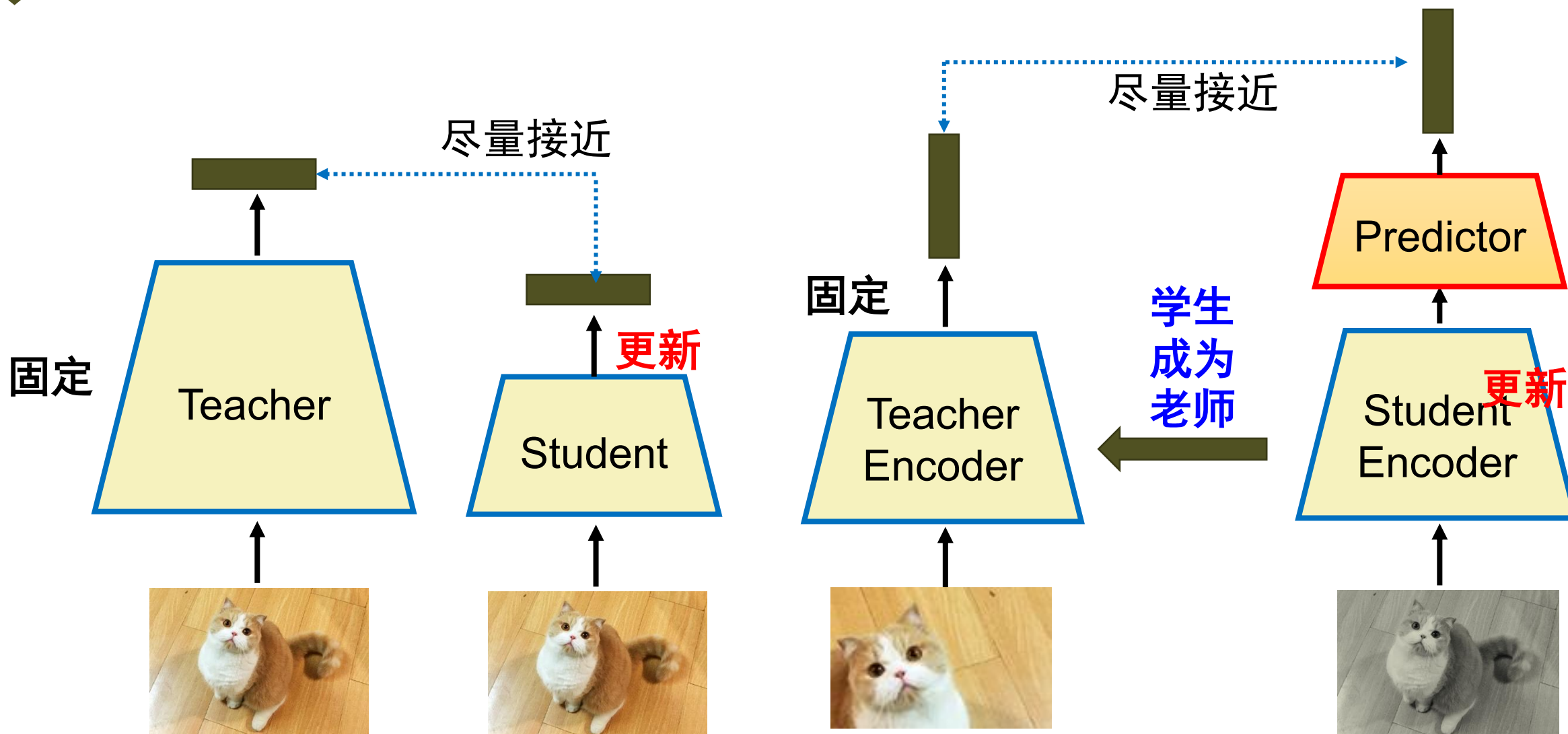


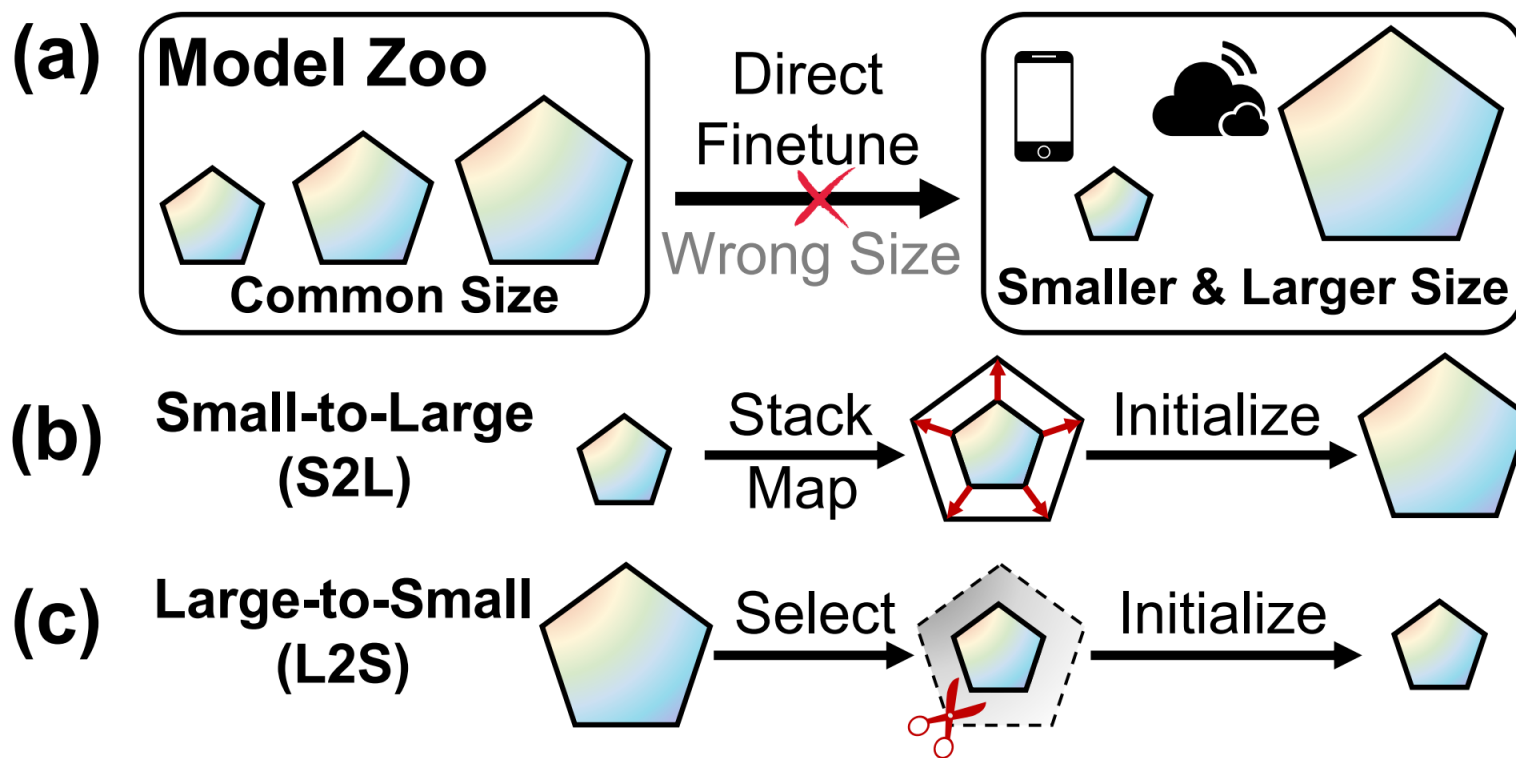
4. 自举法：在没有负例时学习

预训练

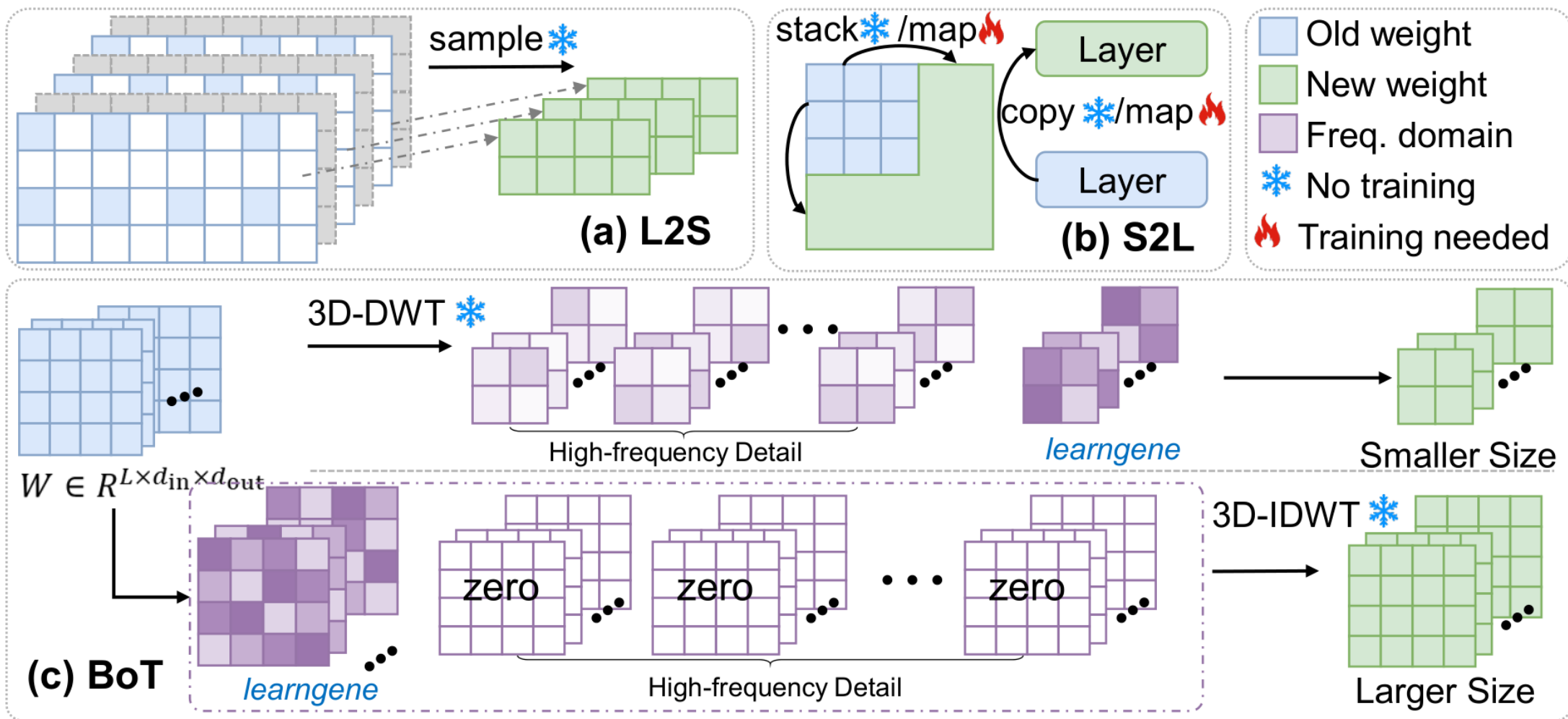


预训练

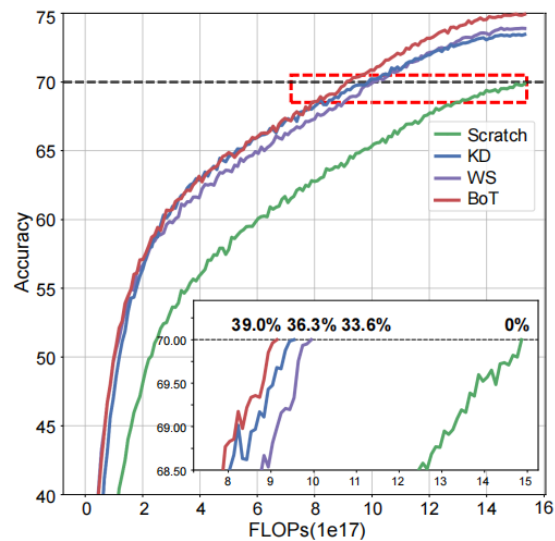




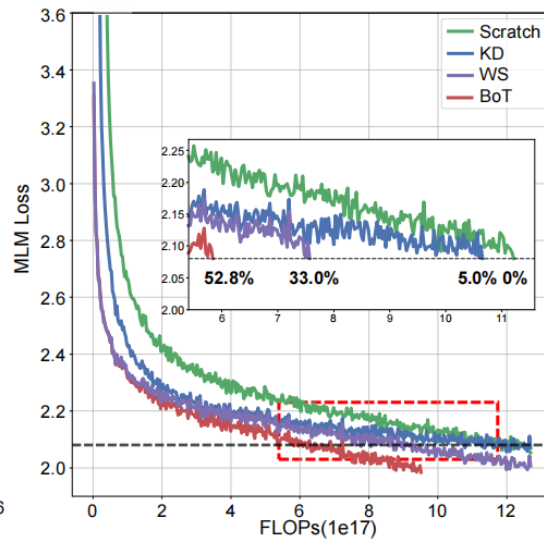
初始化



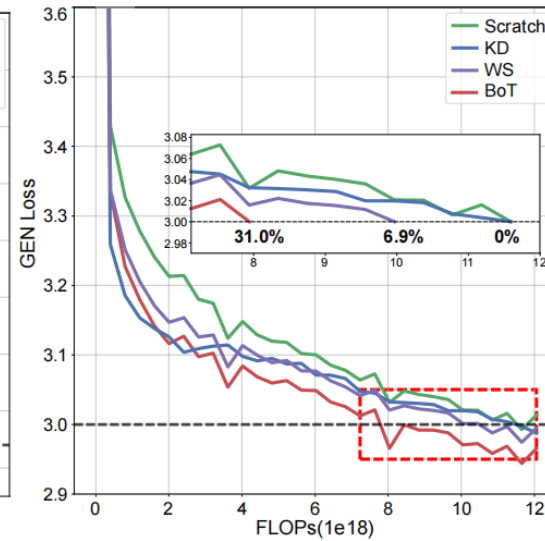
初始化



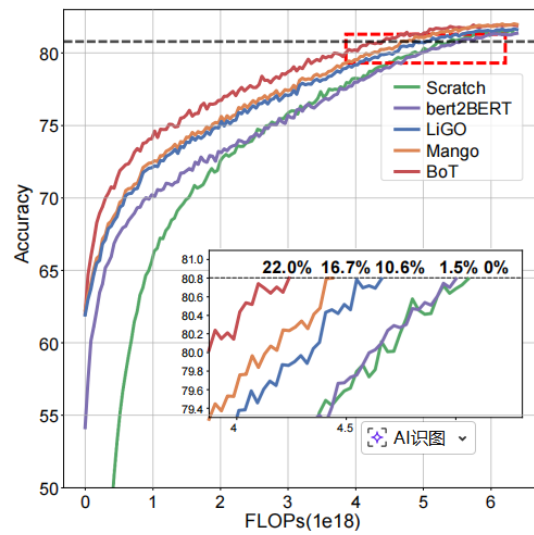
(a) DeiT-B $\rightarrow$ DeiT-S



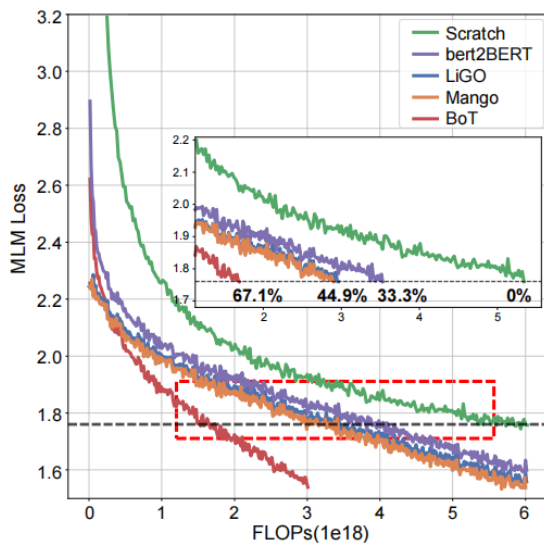
(b) BERT-B $\rightarrow$ BERT-S



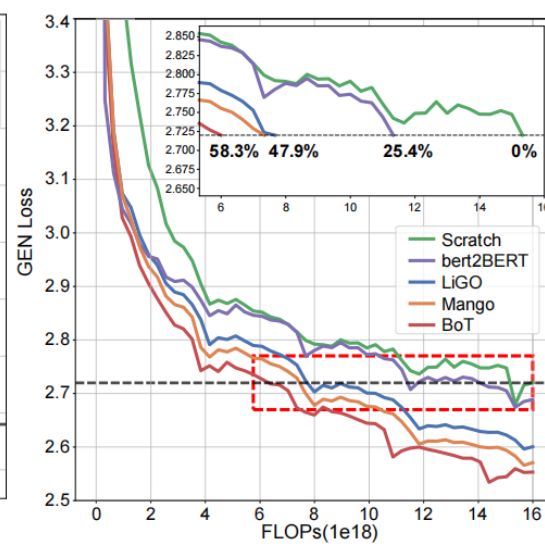
(c) GPT-B $\rightarrow$ GPT-S



(a) DeiT-S $\rightarrow$ DeiT-B



(b) BERT-S $\rightarrow$ BERT-B



(c) GPT-S $\rightarrow$ GPT-B

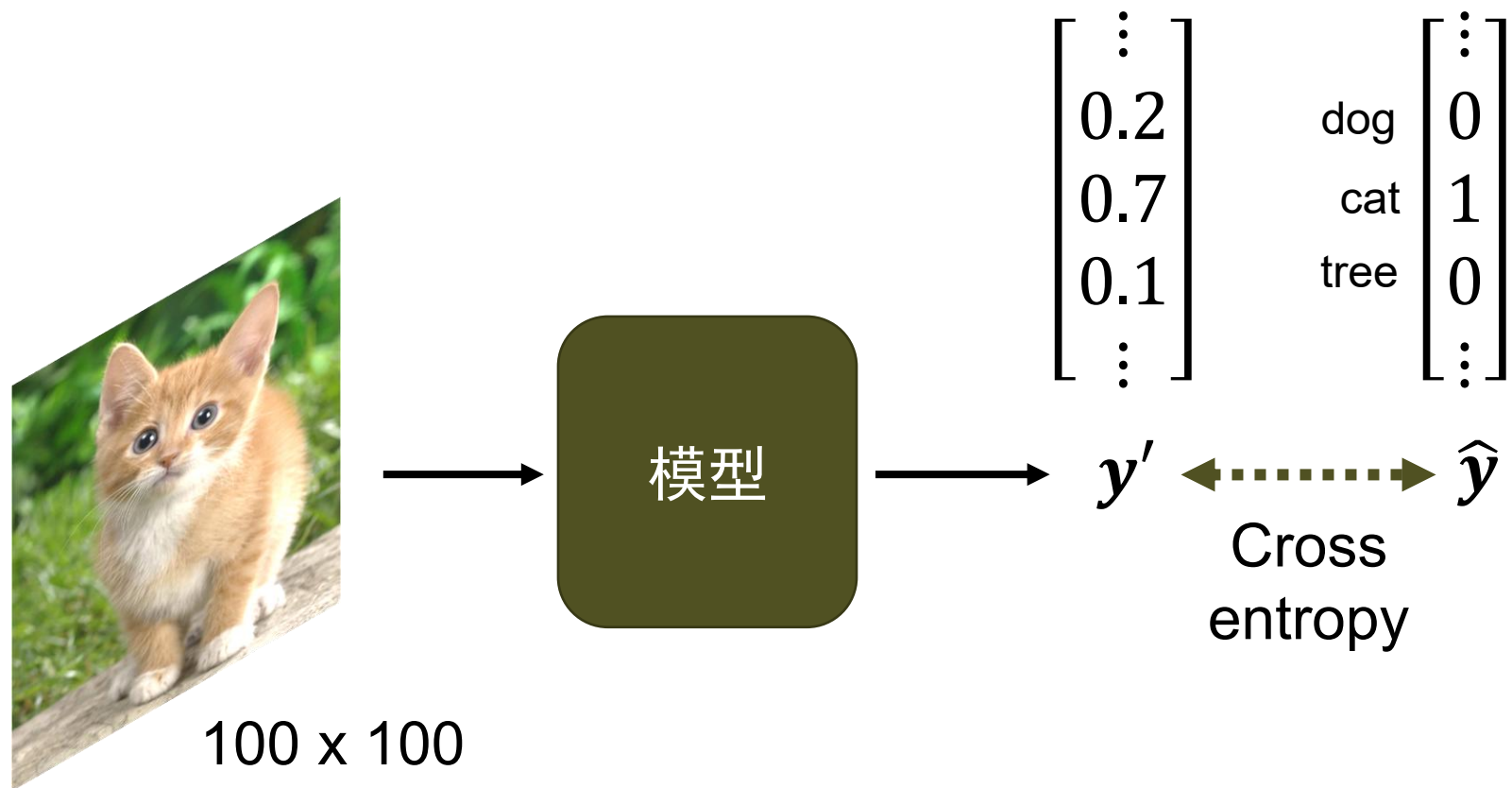
找一个更适合的函数?



目录

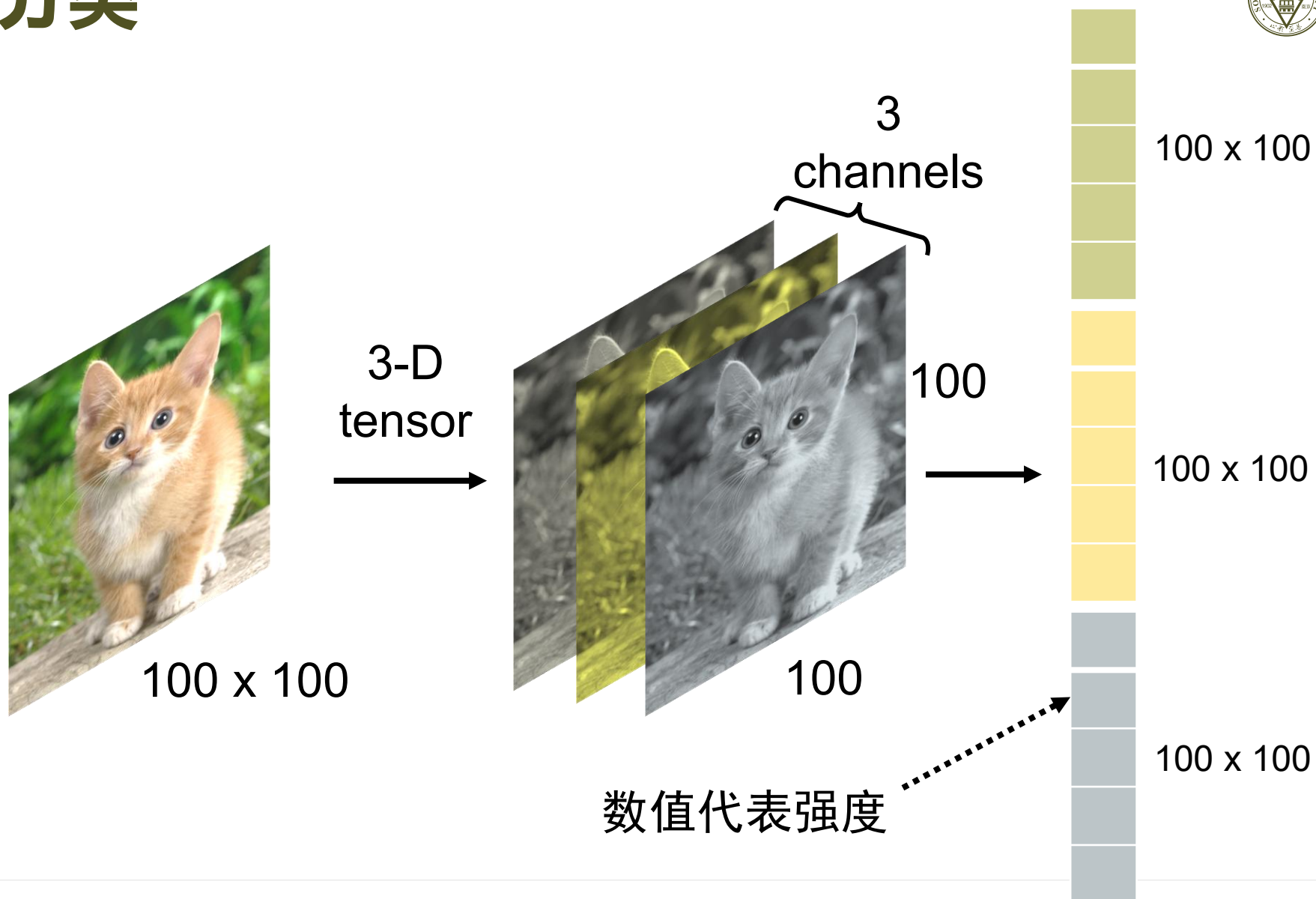
- 神经网络基本概念
- 卷积神经网络
- 循环神经网络
- Transformer

图像分类

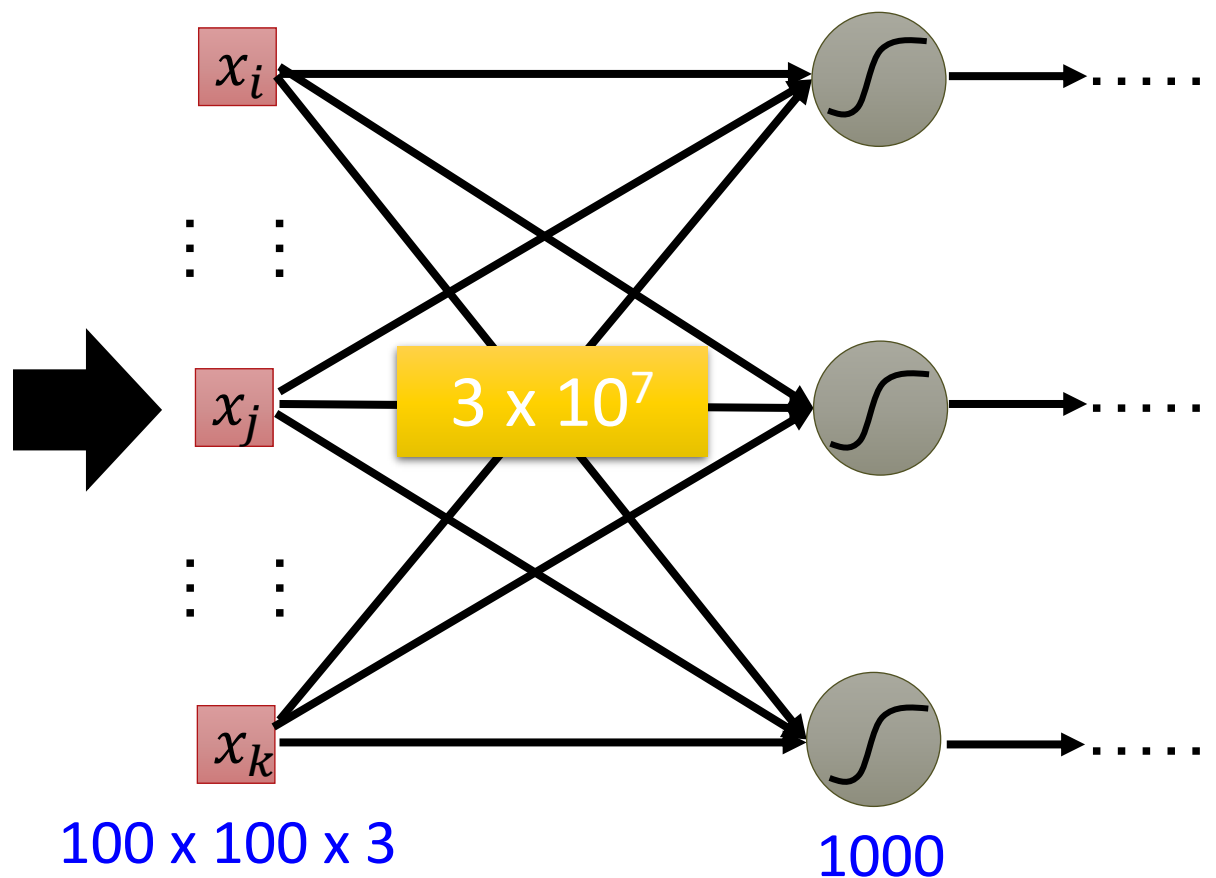


(所有待分类的图像大小都相同)

图像分类

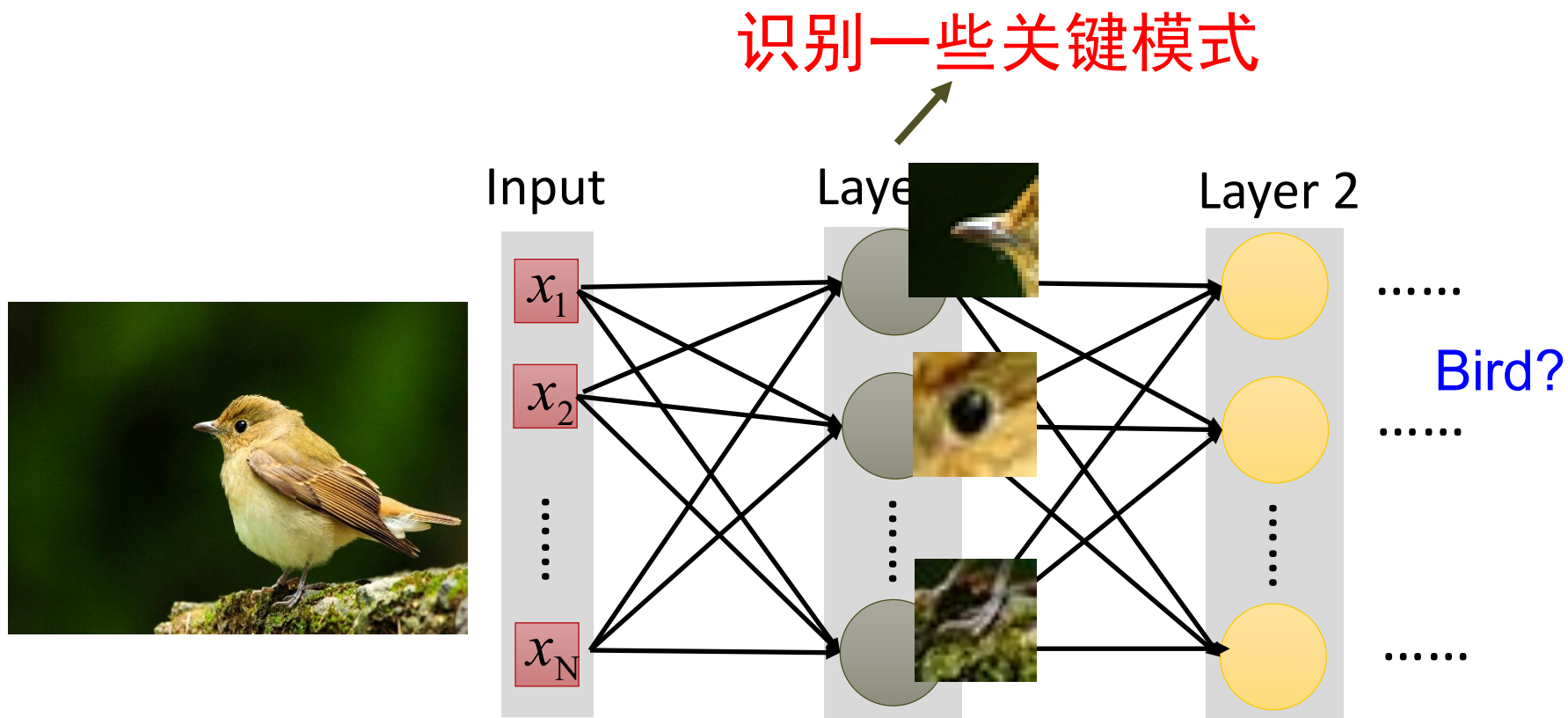


图像分类



我们在图像处理中真的需要全连接吗？

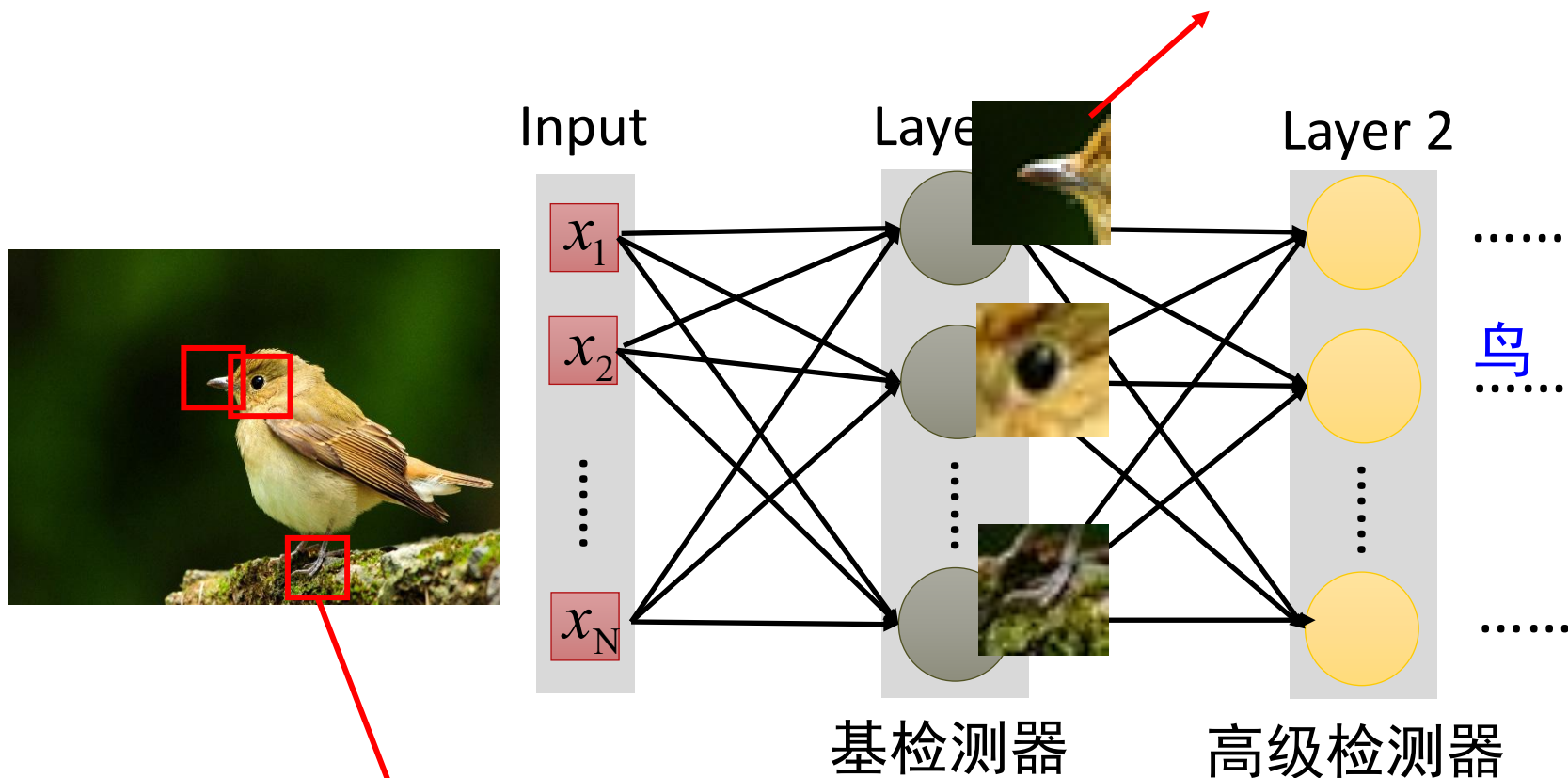
观察 1



人类也是以类似的方式来识别鸟的

观察 1

每个神经元并不需要看到整个图像



某些特定模式的尺寸要比整个图像小得多

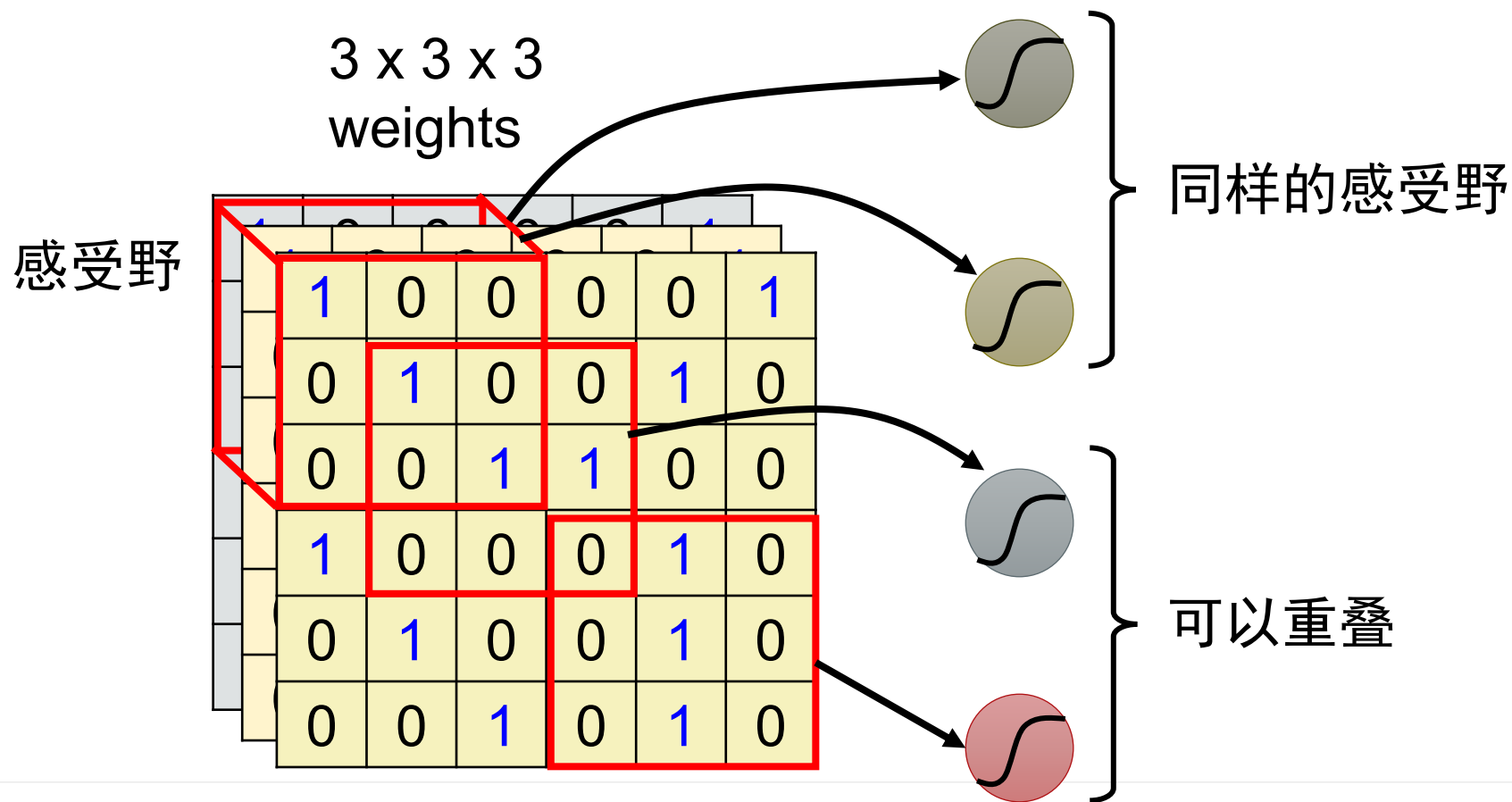


简化 1



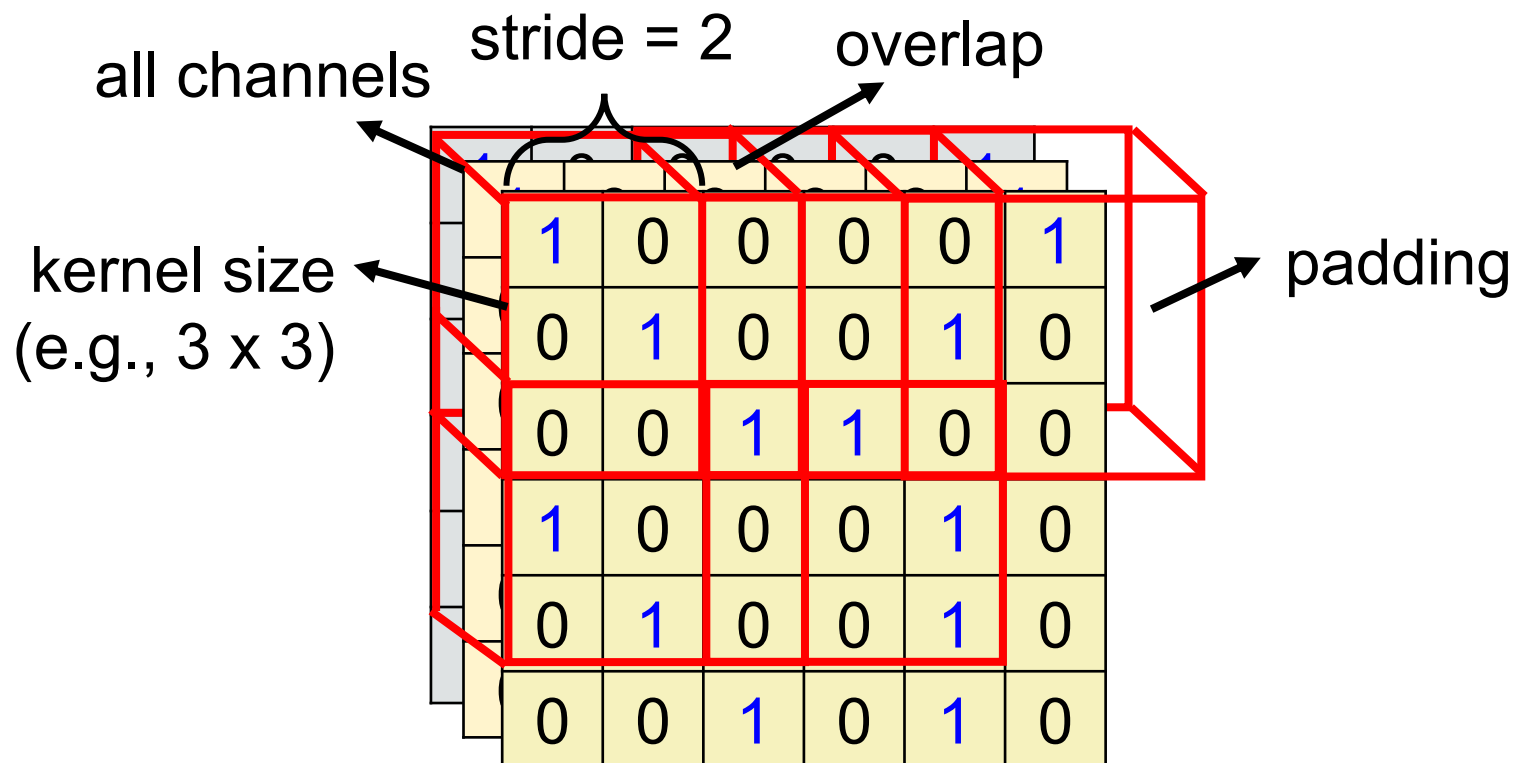
東南大學
SOUTHEAST UNIVERSITY

- 不同的神经元是否可以具有不同的感受野大小呢?
- 只覆盖部分通道吗?
- 有非方形的感受野吗?



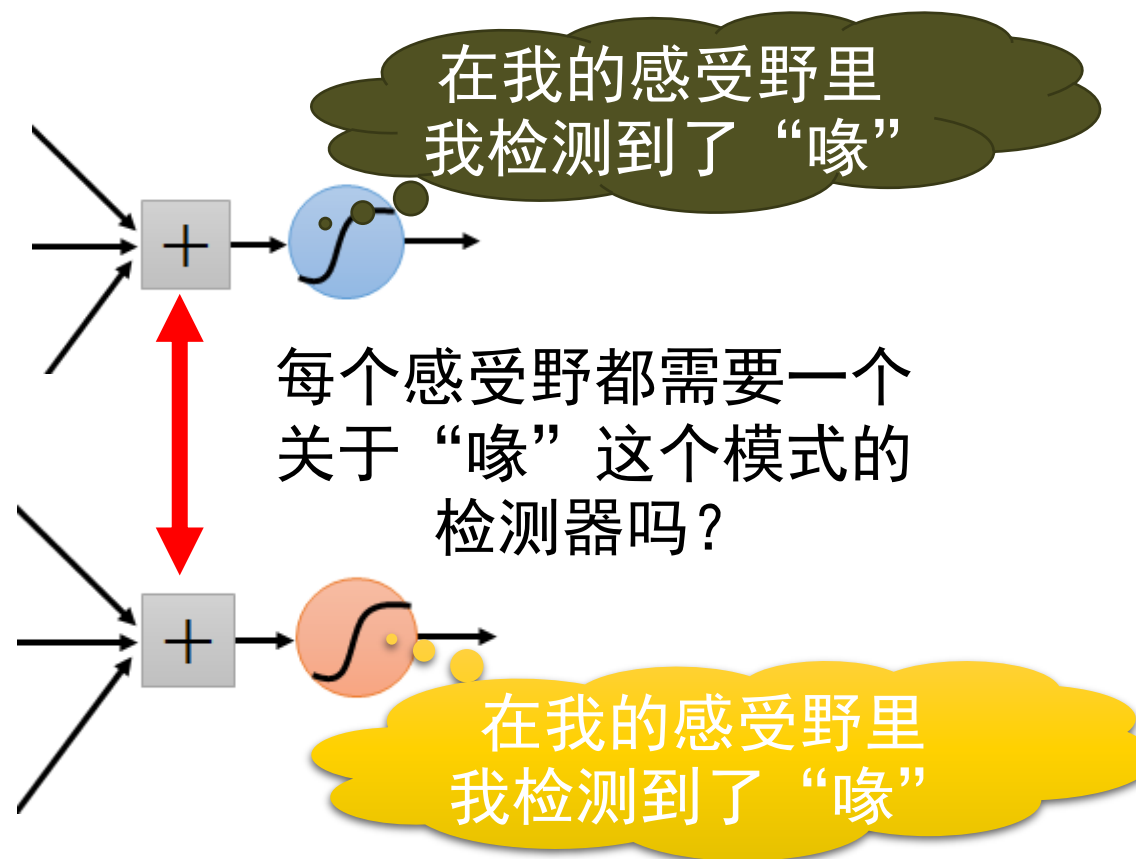
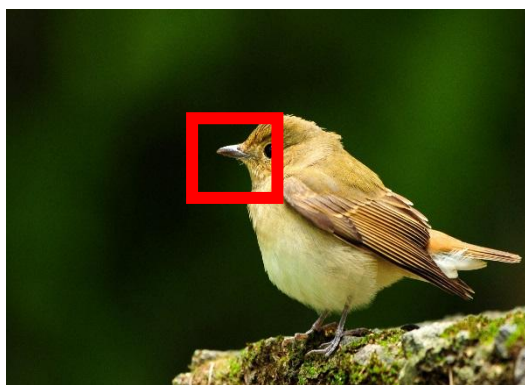
简化 1 —— 典型设定

- 每个感受区域都包含一组神经元
- 感受野覆盖整个图像

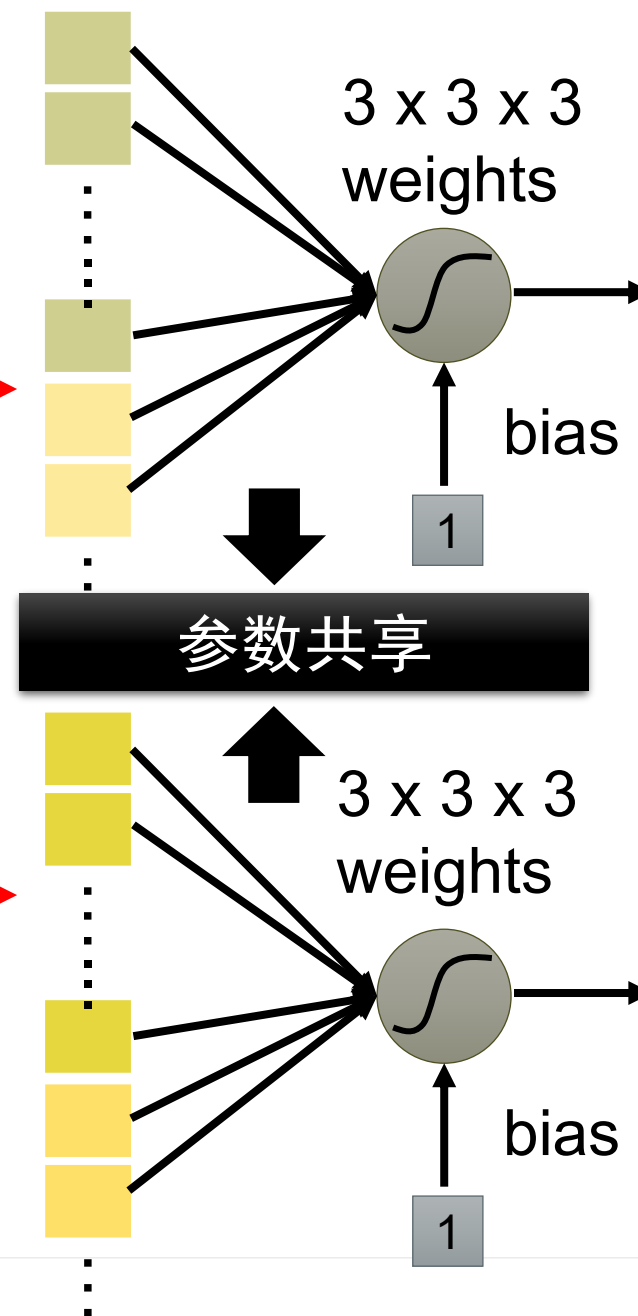
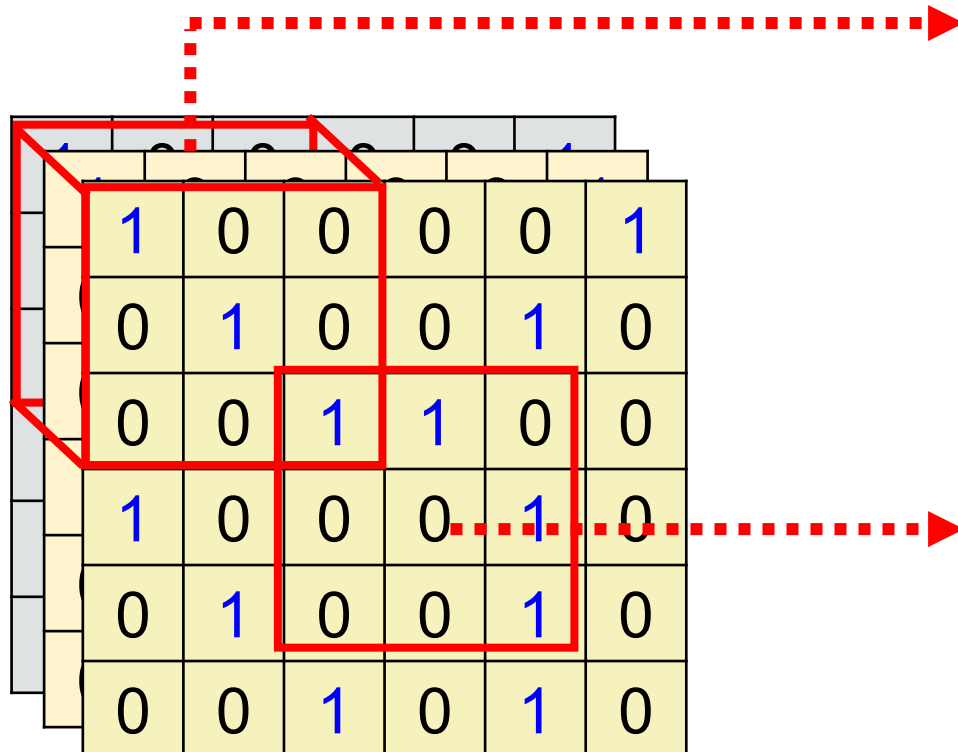


观察 2

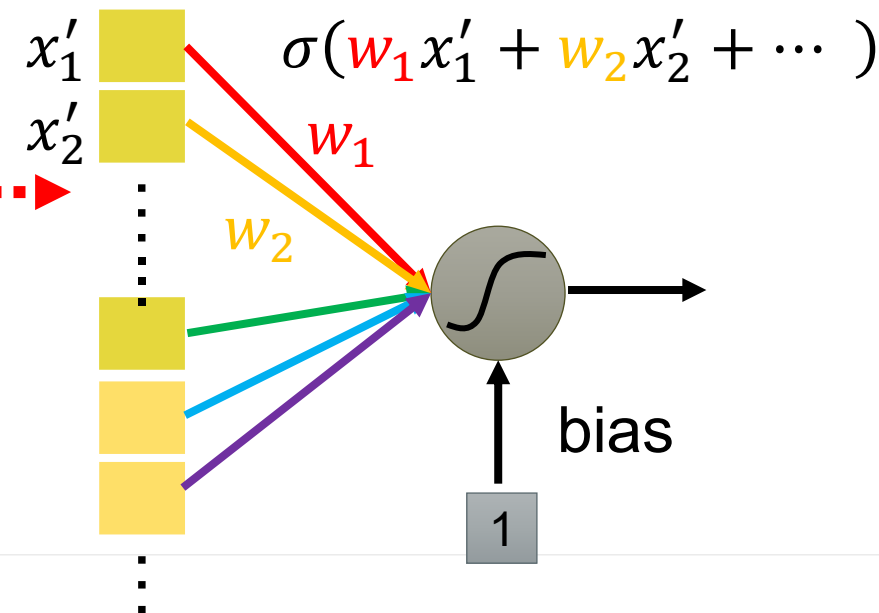
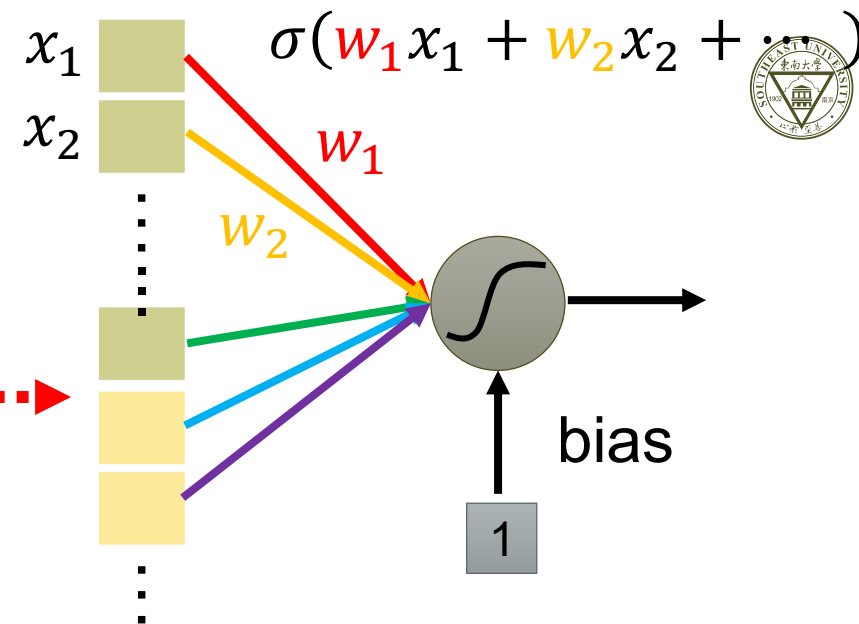
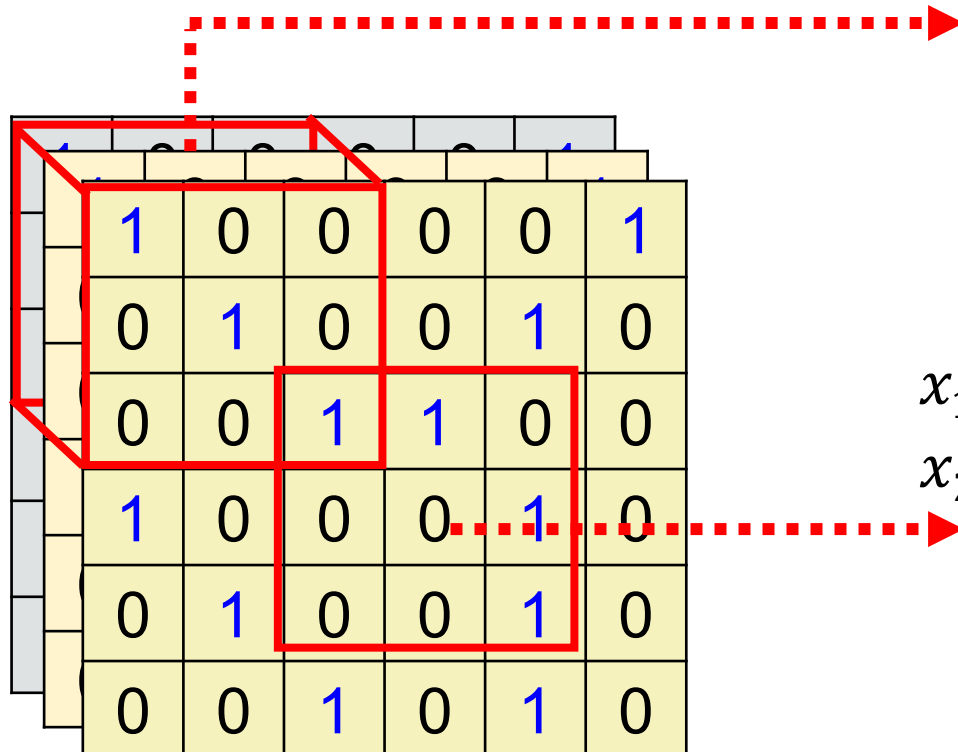
- 同样的模式在不同的区域也存在



简化 2

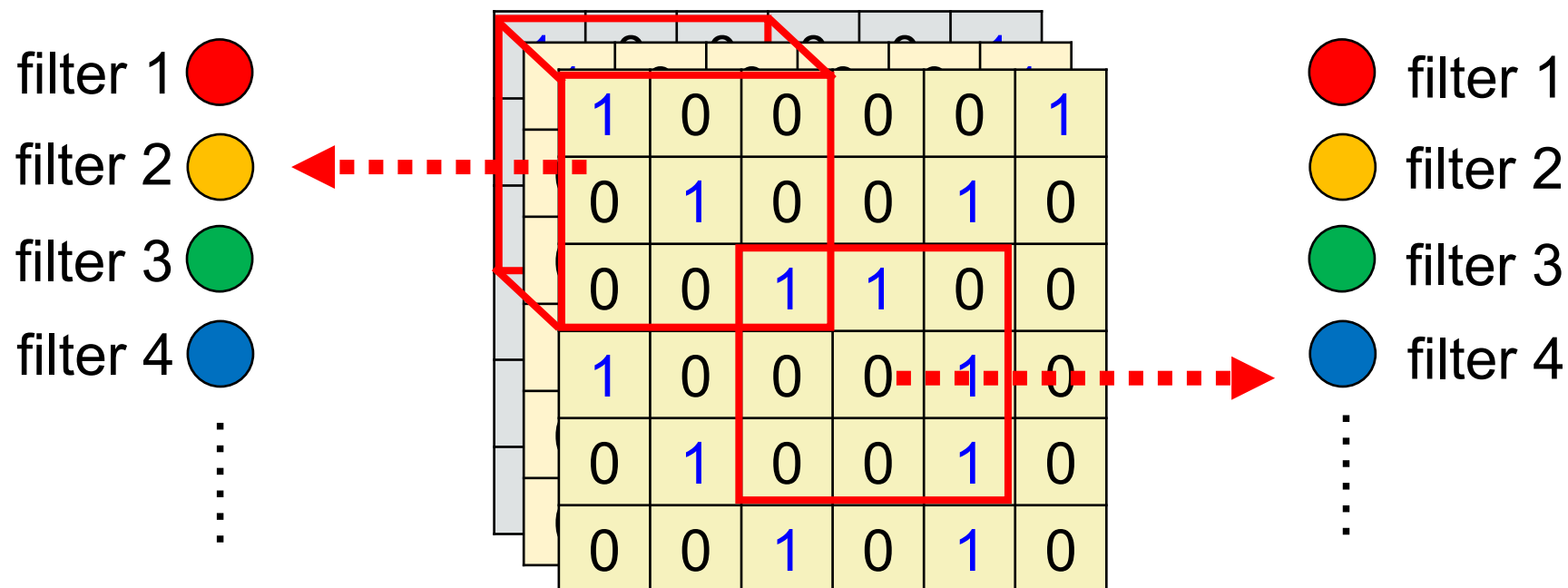


简化 2

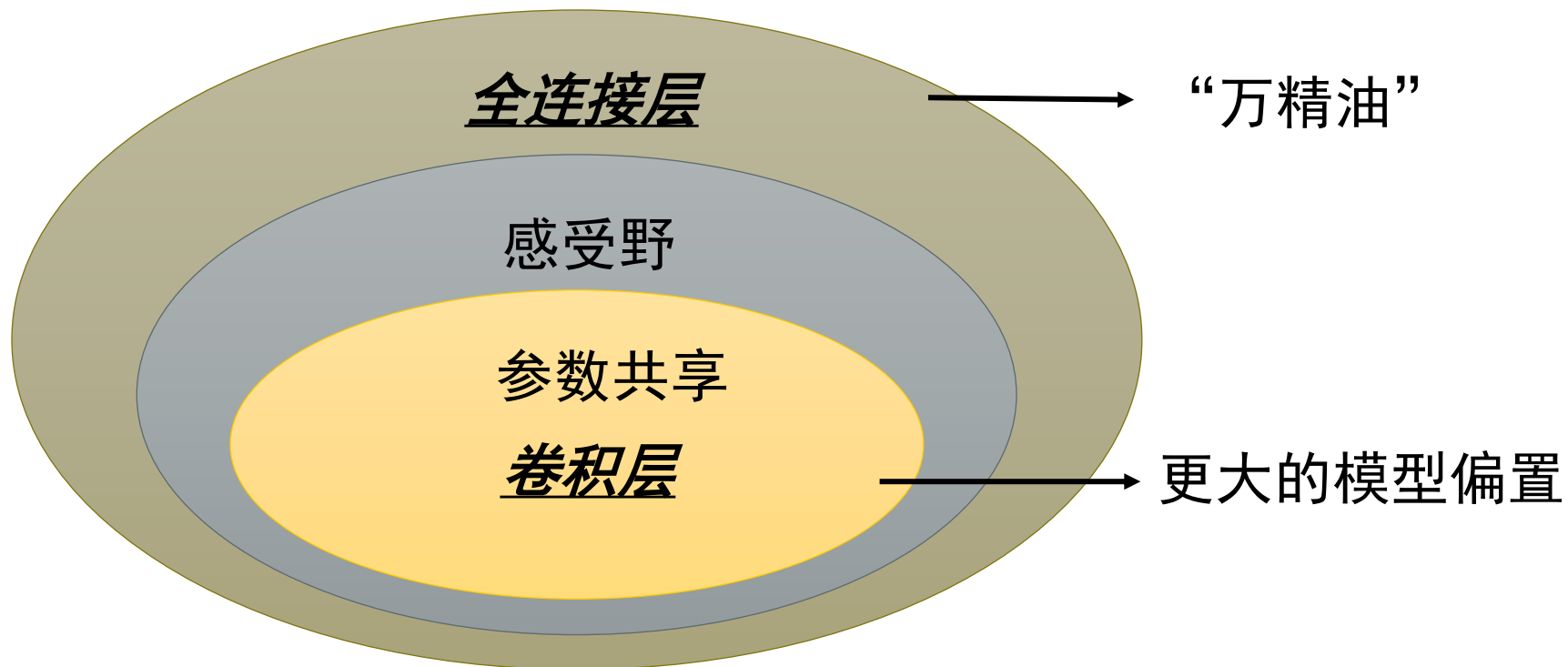


简化 2 —— 典型设定

- 关注不同感受野的神经元共享参数
- 关注同一个感受野的神经元不共享参数



卷积层的好处



- 有些模式的尺寸要比整个图像小得多
- 同样的模式会出现在不同的区域

从Filter的角度看

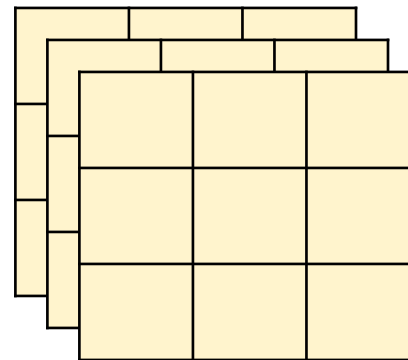


卷积

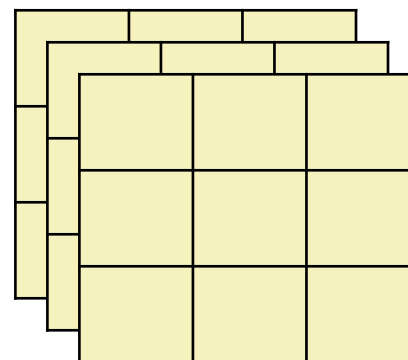
⋮

channel = 3(彩图)

channel = 1(黑白图)



Filter 1
 $3 \times 3 \times \text{channel}$
tensor



Filter 2
 $3 \times 3 \times \text{channel}$
tensor

⋮

每个Filter都会检测到一个较小的模式 ($3 \times 3 \times \text{channel}$)

从Filter的角度看

考虑 channel = 1

1	0	0	0	0	1
0	1	0	0	1	0
0	0	1	1	0	0
1	0	0	0	1	0
0	1	0	0	1	0
0	0	1	0	1	0

6 x 6 image

1	-1	-1
-1	1	-1
-1	-1	1

Filter 1

-1	1	-1
-1	1	-1
-1	1	-1

Filter 2

⋮

从Filter的角度看

1	-1	-1
-1	1	-1
-1	-1	1

Filter 1

stride=1

1	0	0	0	0	1
0	1	0	0	1	0
0	0	1	1	0	0
1	0	0	0	1	0
0	1	0	0	1	0
0	0	1	0	1	0

6 x 6 image

3	-1	-3	-1
-3	1	0	-3
-3	-3	0	1
3	-2	-2	-1

从Filter的角度看

-1	1	-1
-1	1	-1
-1	1	-1

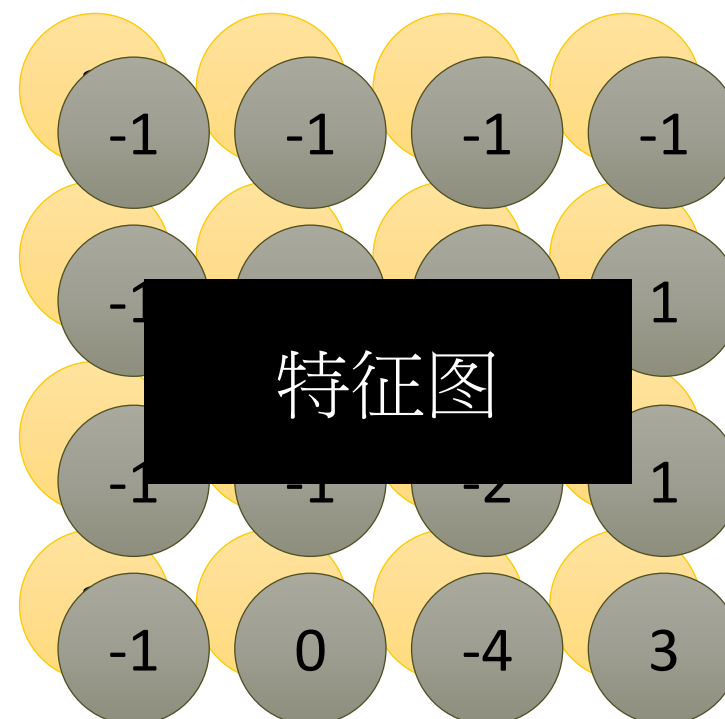
Filter 2

stride=1

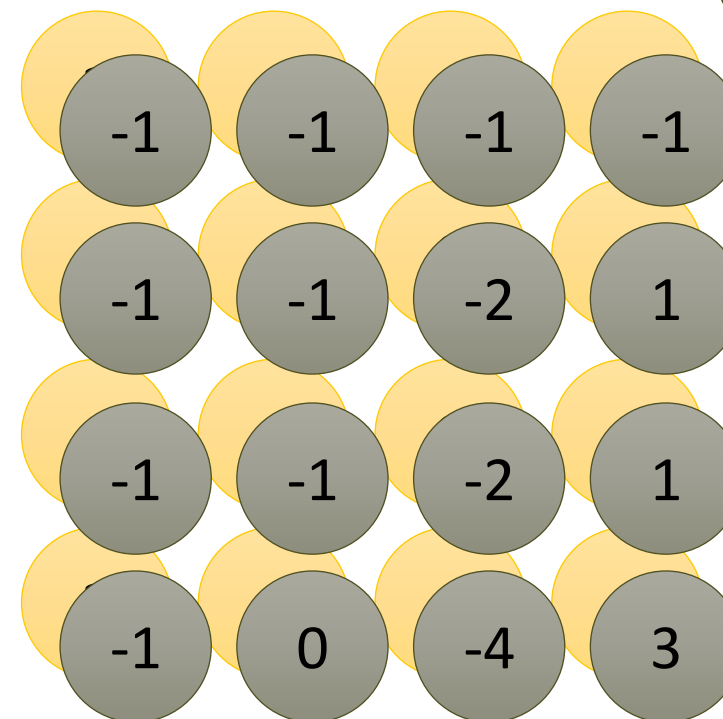
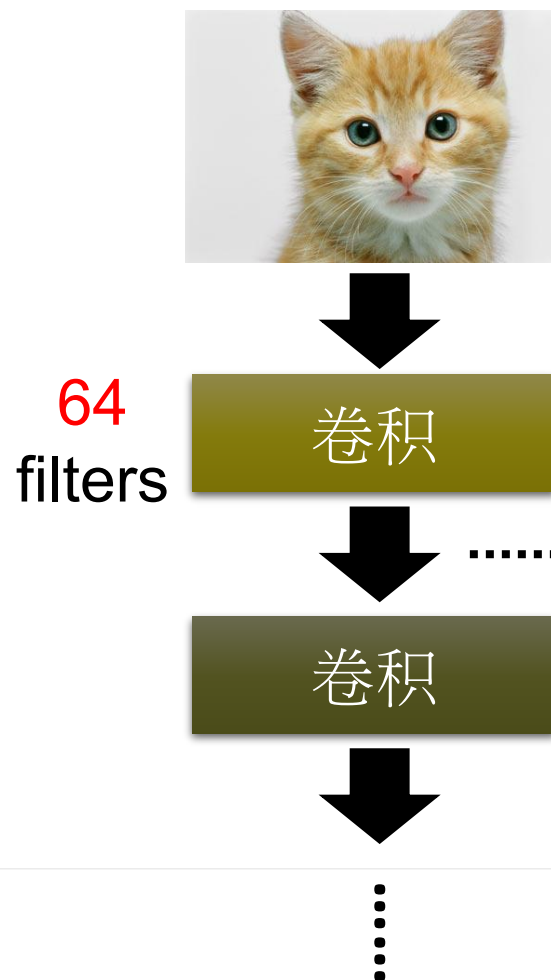
1	0	0	0	0	1
0	1	0	0	1	0
0	0	1	1	0	0
1	0	0	0	1	0
0	1	0	0	1	0
0	0	1	0	1	0

6 x 6 image

对每个Filter都执行同样的
操作流程



从Filter的角度看



有 64 channels 的新 “图片”

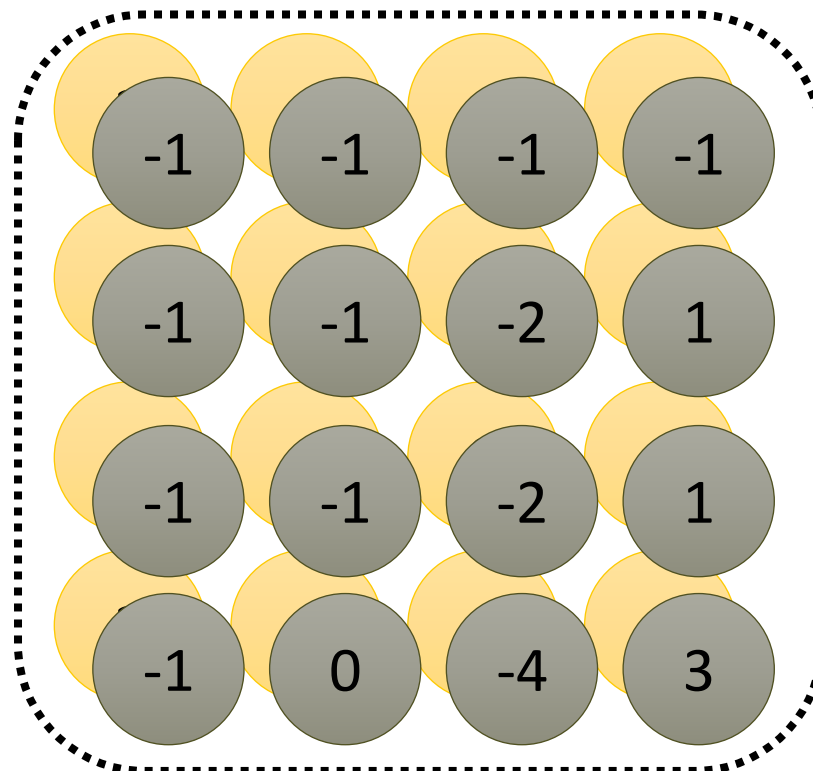
从Filter的角度看



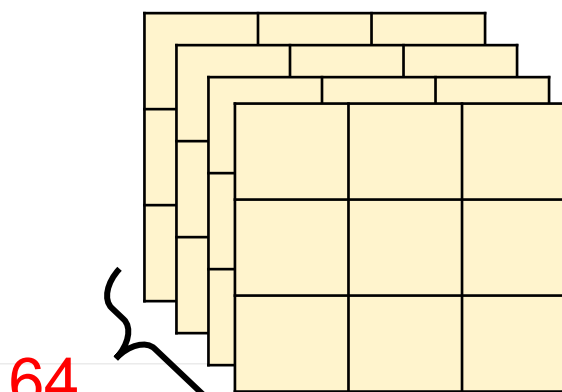
卷积

卷积

...



有 64 channels 的新“图片”



从Filter的角度看



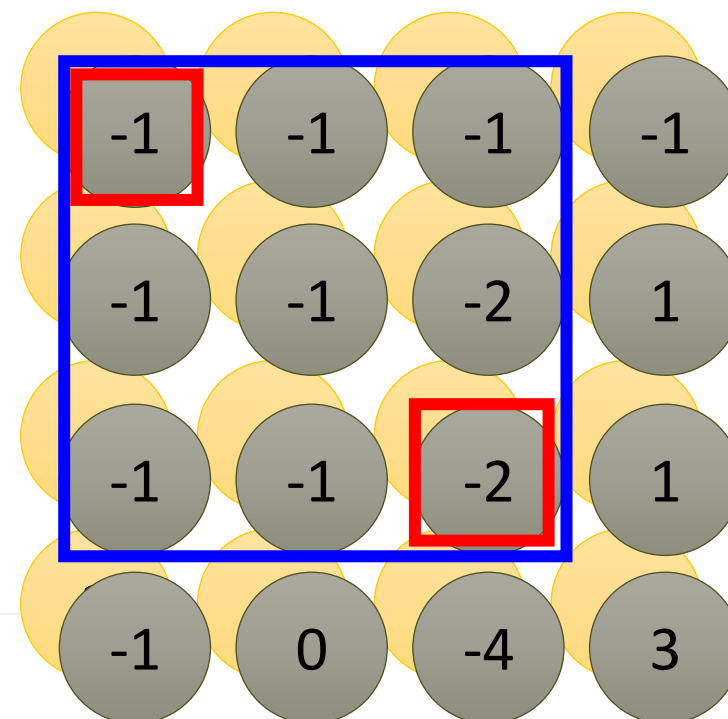
64
filters

卷积

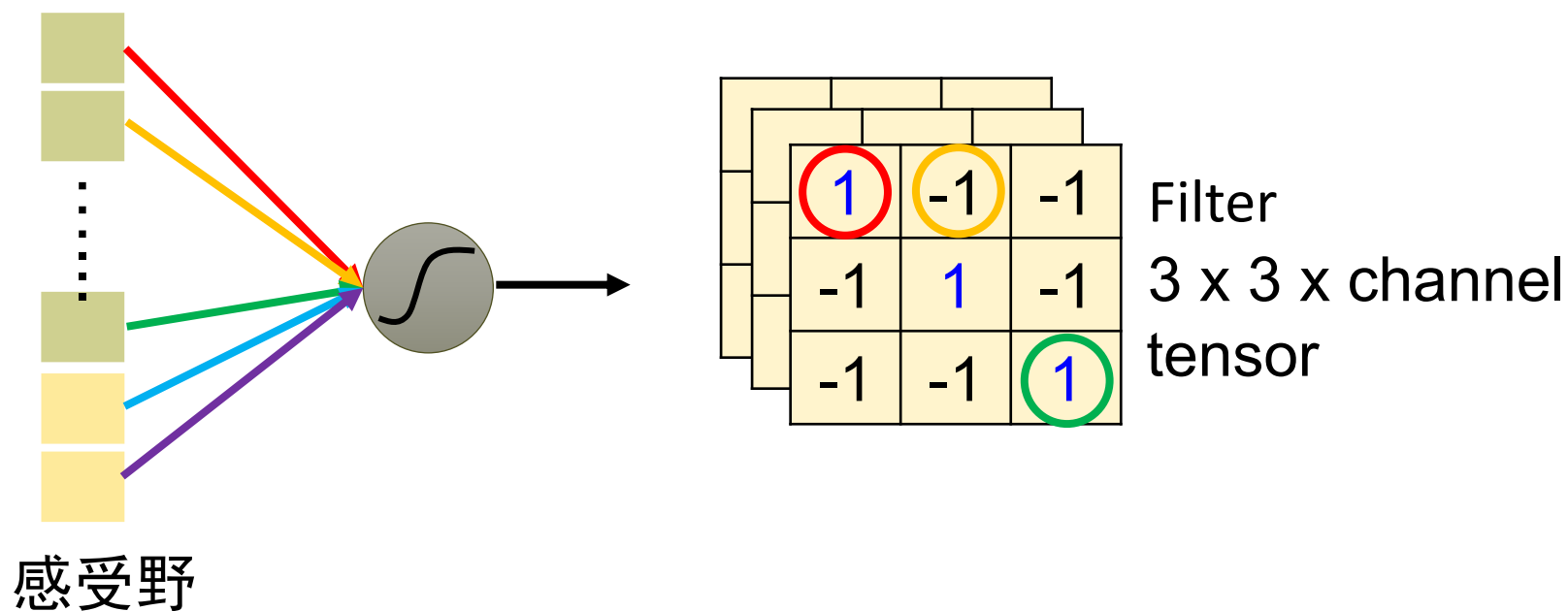
卷积

⋮

1	0	0	0	0	1
0	1	0	0	1	0
0	0	1	1	0	0
1	0	0	0	1	0
0	1	0	0	1	0
0	0	1	0	1	0



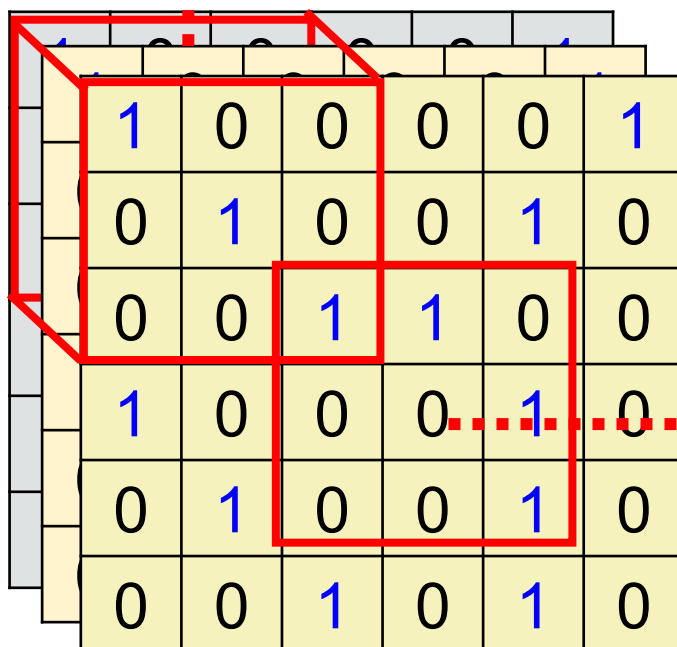
比较两个角度



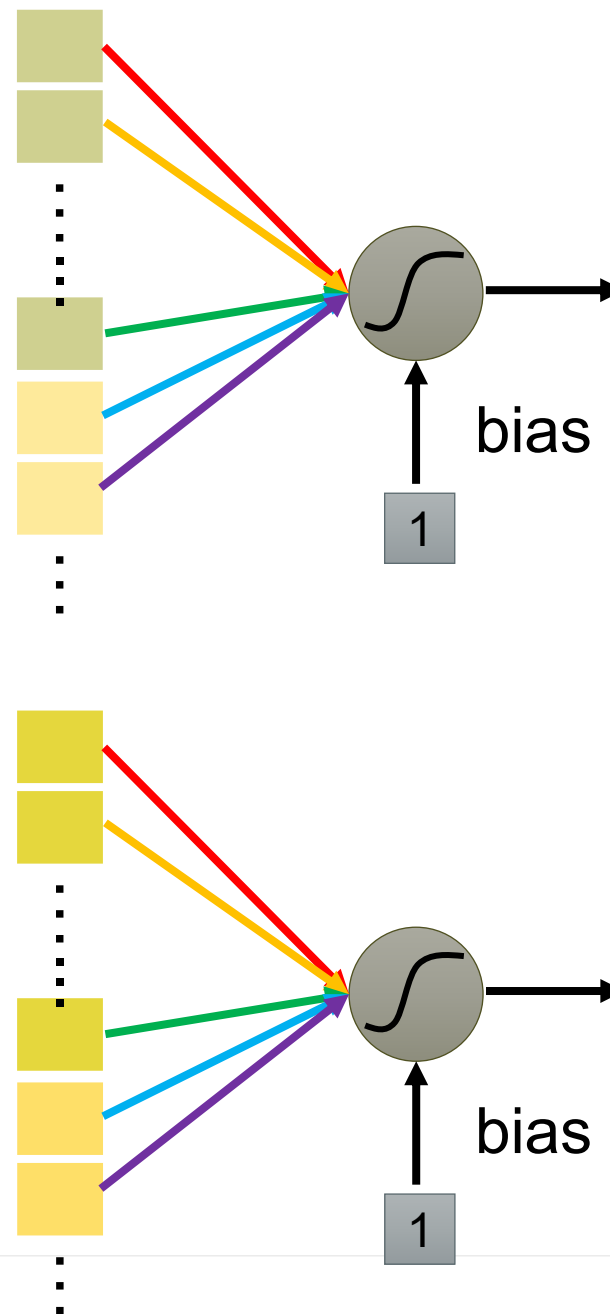
- 第一个版本里说有一些神经元会共享参数，这些参数就是第二个版本里的Filter
- 左边有 $3 \times 3 \times 3$ 个对应共享的参数，右边就是 $3 \times 3 \times 3$ 大小的Filter

比较两个角度

- 第一个版本说不同感受野的神经元共享参数



- 也就是第二个版本里用filter扫过整张图像，这就是卷积操作



观察 3

- 对像素进行下采样不会改变物体的形态

鸟



下采样

鸟



池化 —— Max Pooling

1	-1	-1
-1	1	-1
-1	-1	1

Filter 1

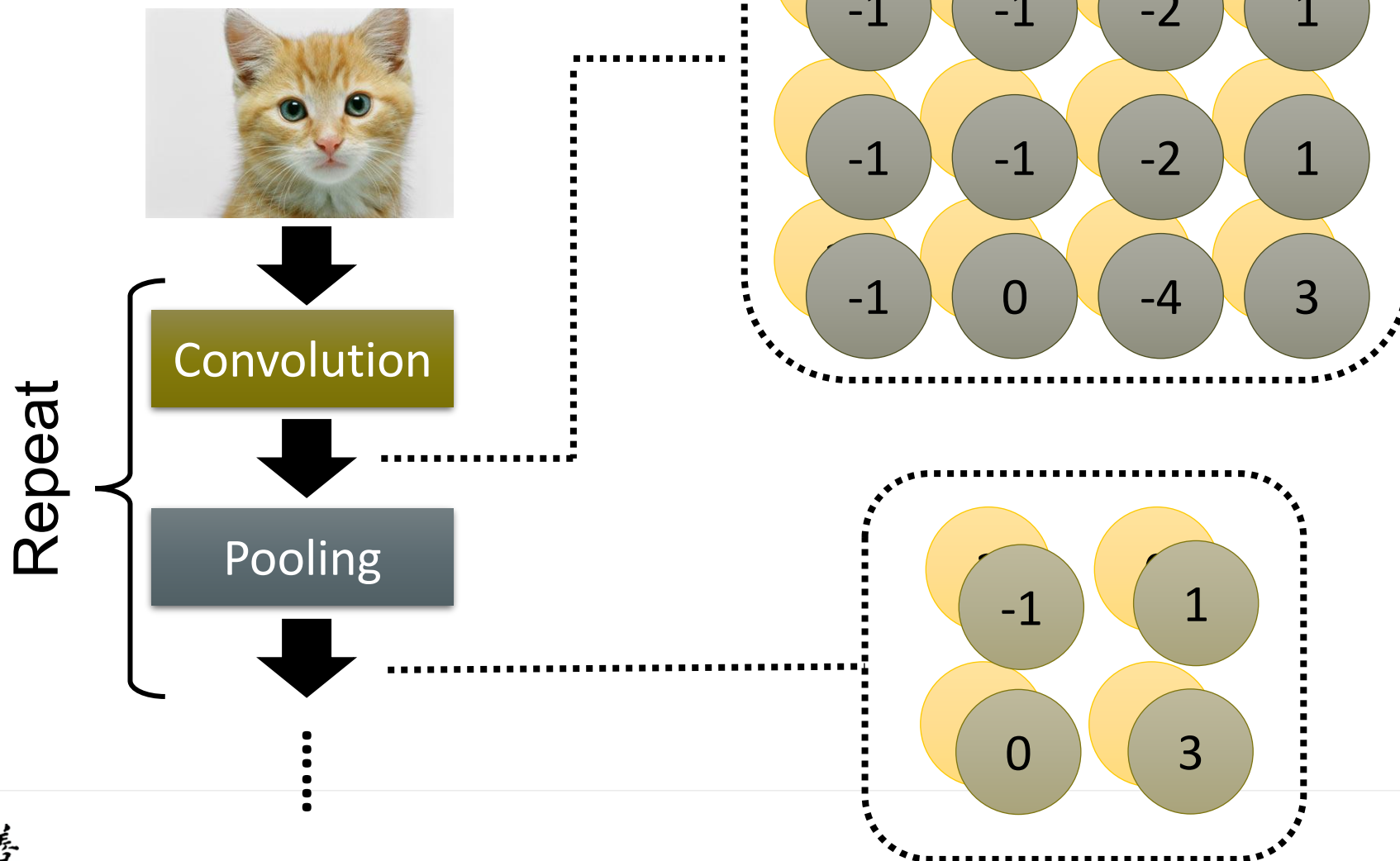
-1	1	-1
-1	1	-1
-1	1	-1

Filter 2

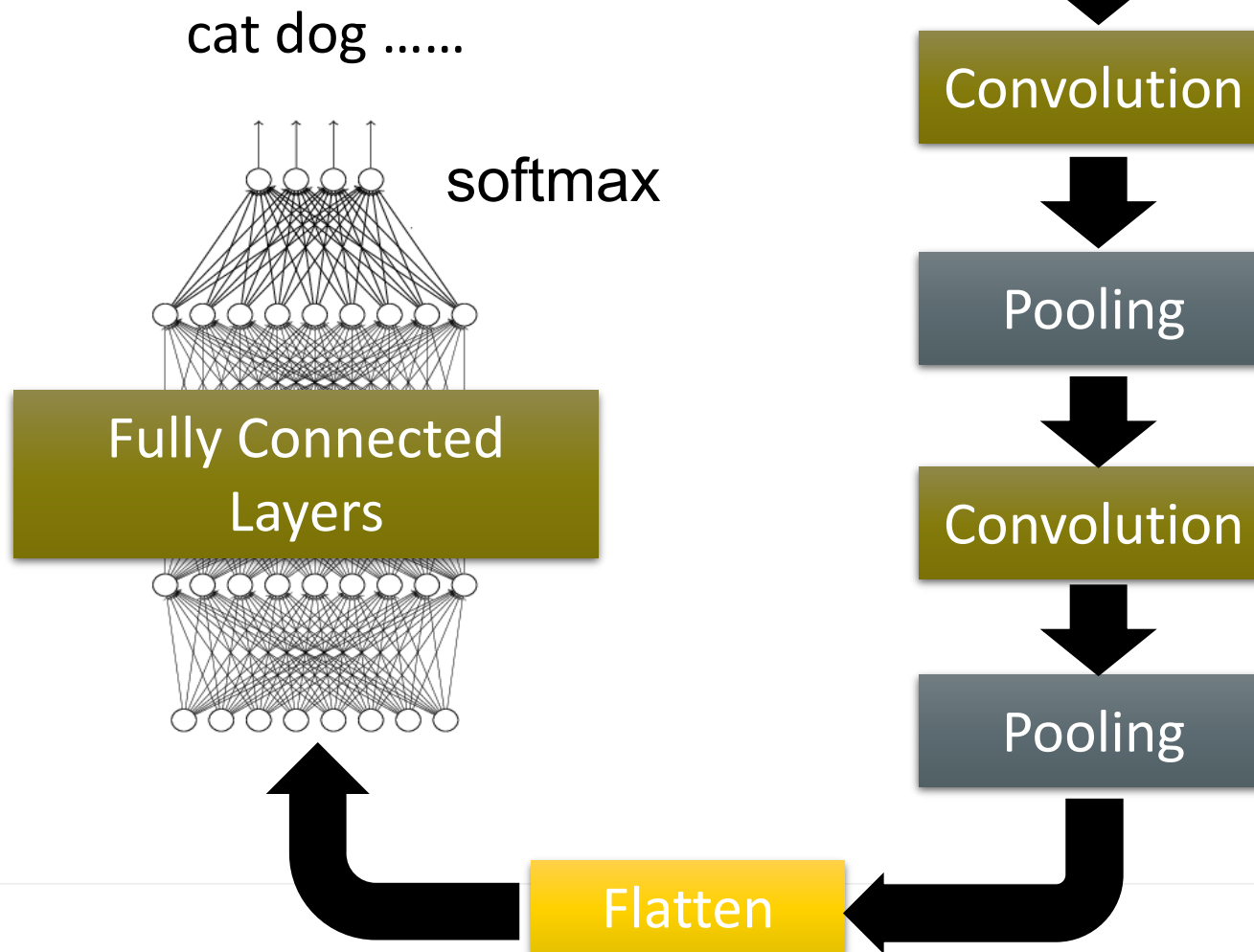
3	-1	-3	-1
-3	1	0	-3
-3	-3	0	1
3	-2	-2	-1

-1	-1	-1	-1
-1	-1	-2	1
-1	-1	-2	1
-1	0	-4	3

卷积 + 池化



CNN

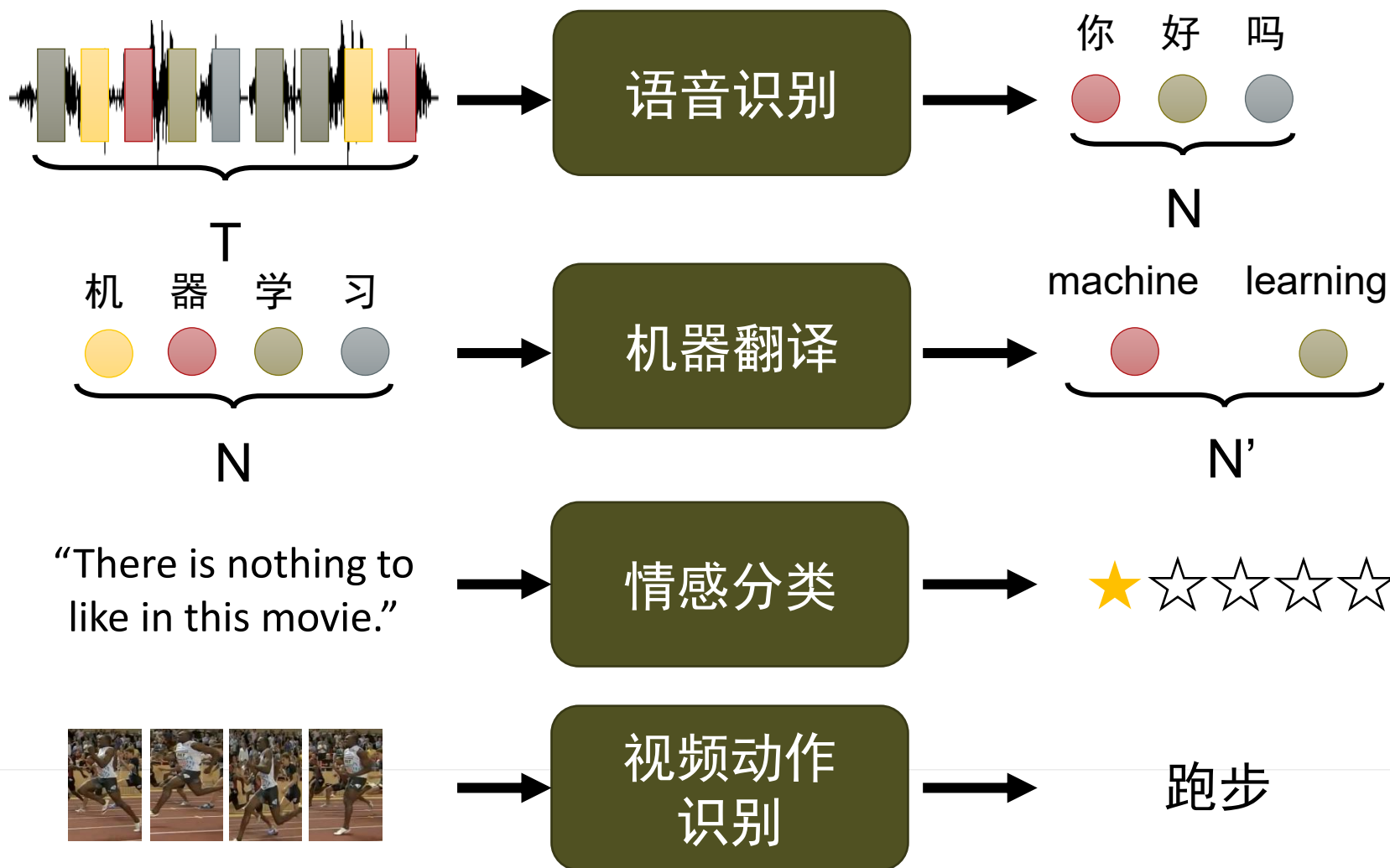


目录

- 神经网络基本概念
- 卷积神经网络
- 循环神经网络
- Transformer

Sequence-to-Sequence (Seq2seq)

- 输入一个序列，输出一个序列
- 输出的长度由模型决定



Sequence-to-Sequence (Seq2seq)

x: I would like to arrive Nanjing on April 2nd.
 $x^{<1>} \ x^{<2>} \ x^{<3>} \quad \dots \quad x^{<t>} \ x^{<9>}$

y: other other other ... dest other time time
 $y^{<1>} \ y^{<2>} \ y^{<3>} \quad \dots \quad y^{<t>} \ y^{<9>}$

$x^{(i)<t>}$: i-th 训练样本中的 t-th 单词

$y^{(i)<t>}$: i-th 训练样本的 t-th 输出

$T_x^{(i)}$: i-th 训练样本的长度

Sequence-to-Sequence (Seq2seq)

如何将每个单词表示为一个向量？

1-of-N Encoding lexicon = {apple, bag, cat, dog, elephant}

- 该向量的大小为词汇表的大小
- 每个维度对应词汇表中的一个单词
- 该单词所在的维度为 1, 其他维度为 0

apple = [1 0 0 0 0]

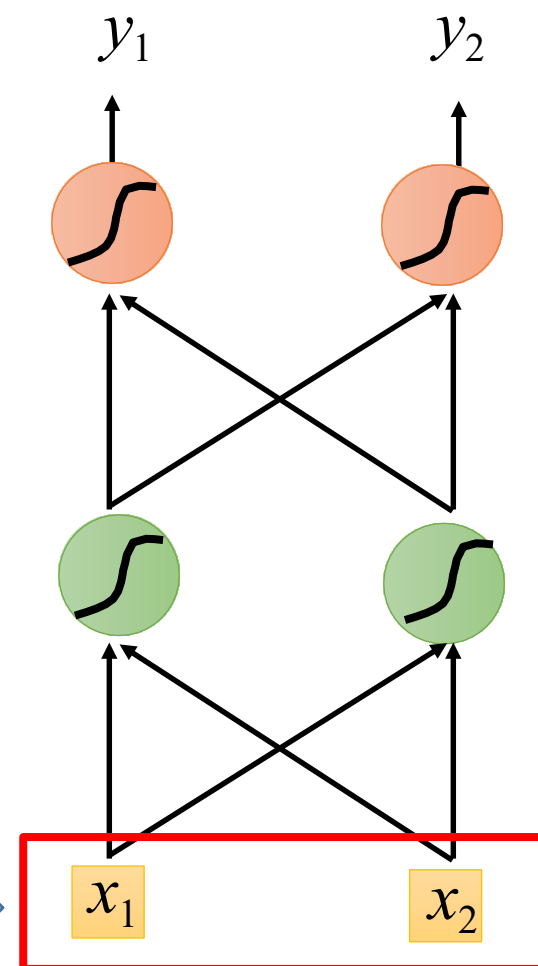
bag = [0 1 0 0 0]

cat = [0 0 1 0 0]

dog = [0 0 0 1 0]

elephant = [0 0 0 0 1]

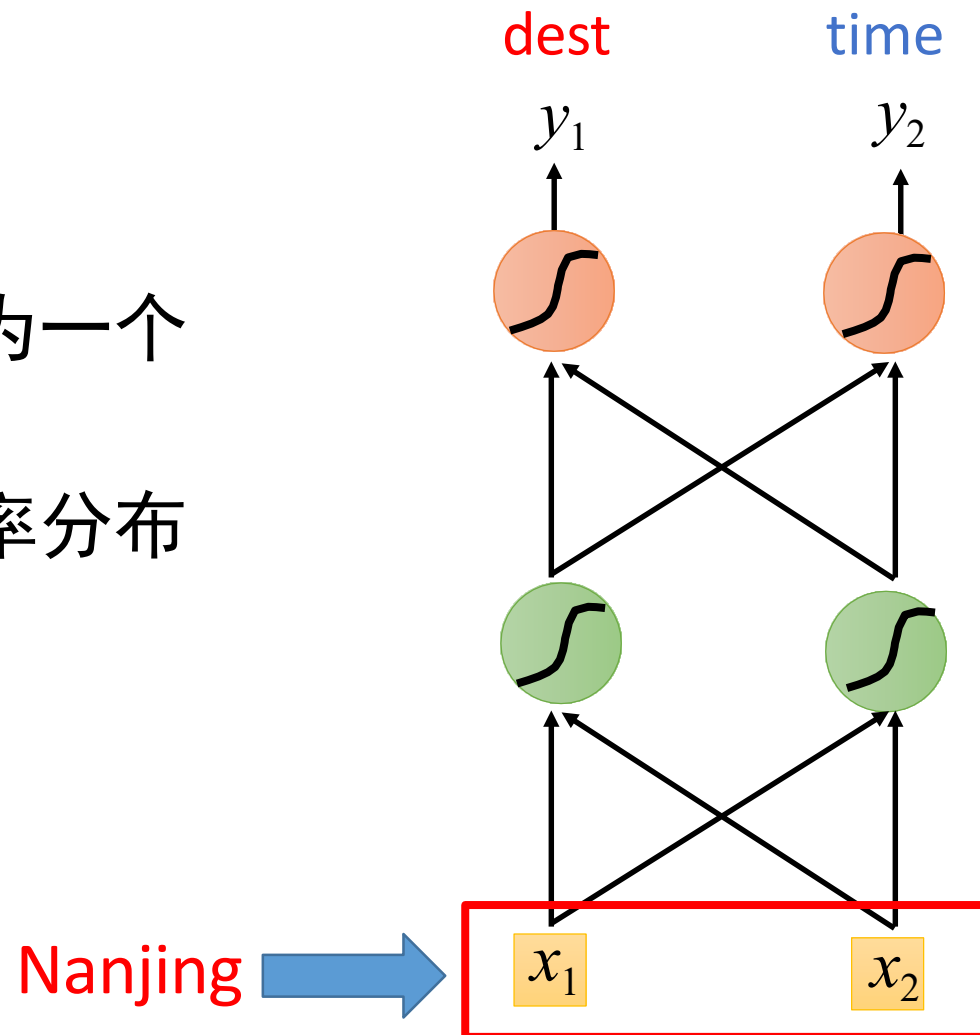
Nanjing →



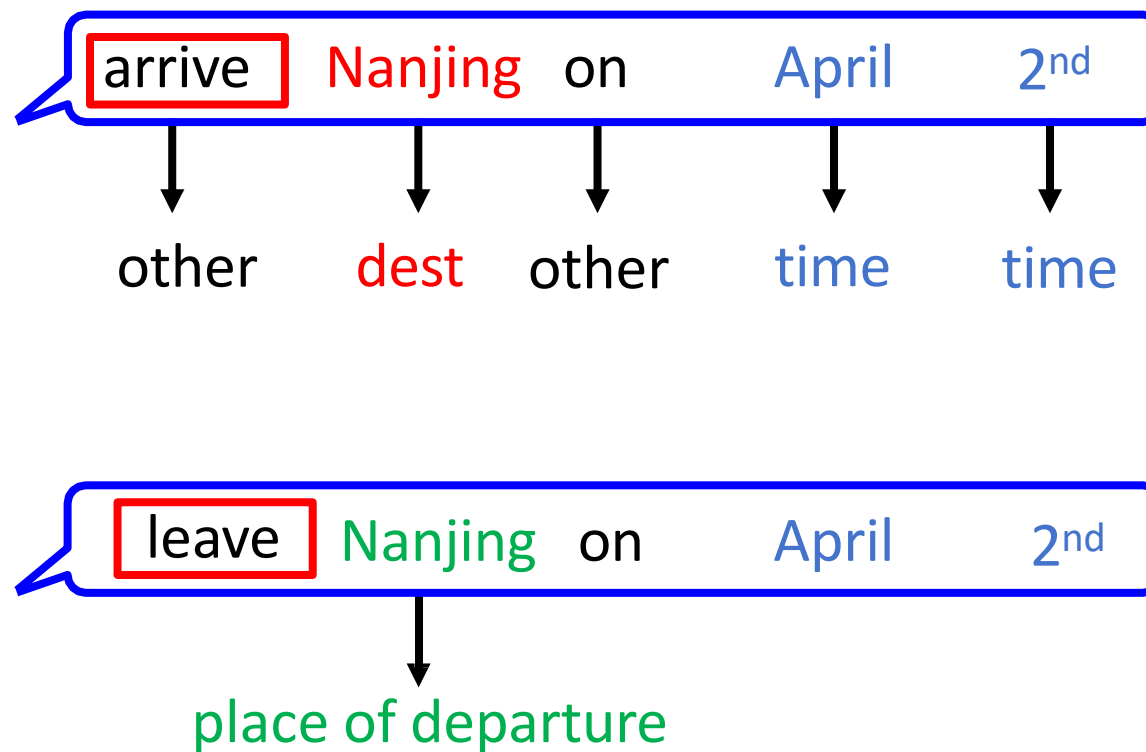
Sequence-to-Sequence (Seq2seq)

如何通过前馈网络解决槽位填充问题？

- 输入：一个单词（每个单词都表示为一个向量）
- 输出：输入单词属于这些槽位的概率分布

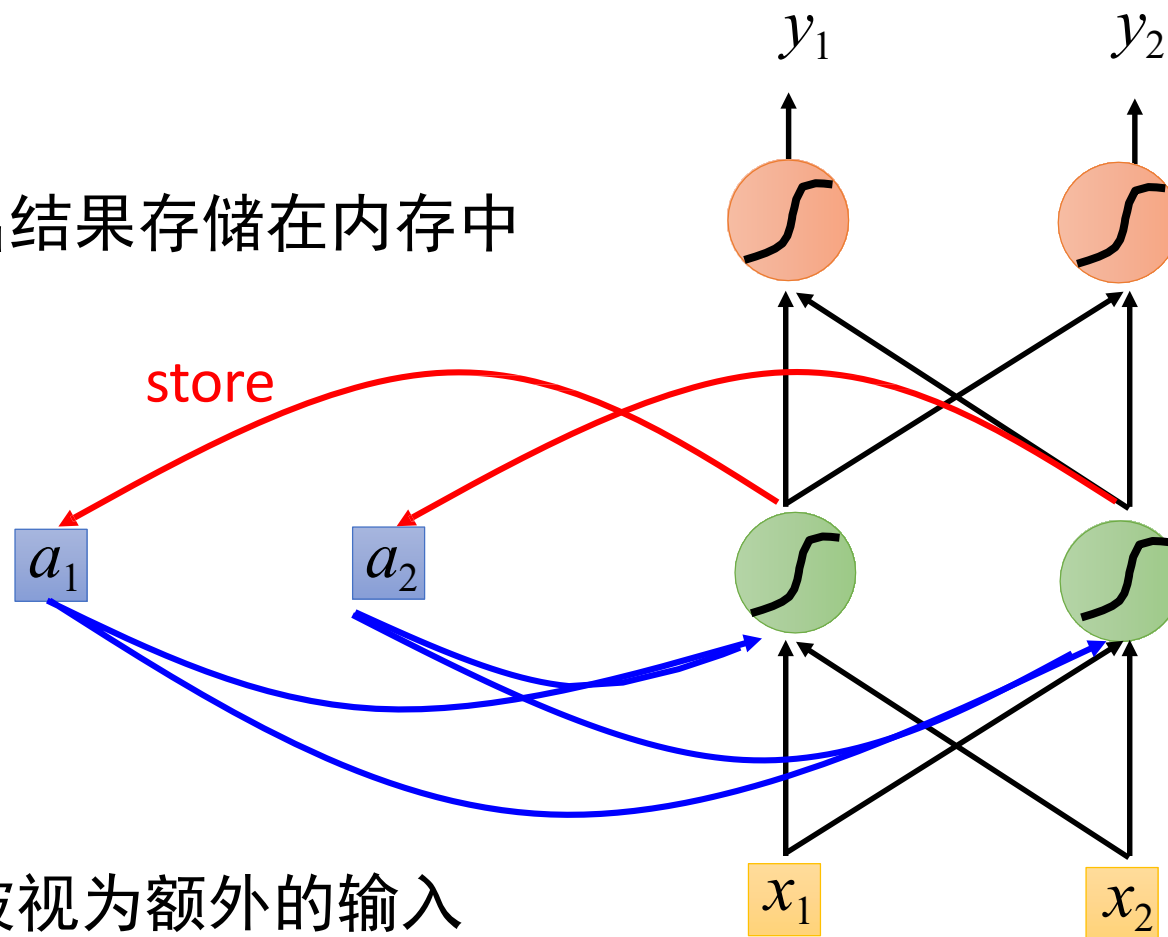


Sequence-to-Sequence (Seq2seq)



神经网络需要有记忆！

隐藏层的输出结果存储在内存中

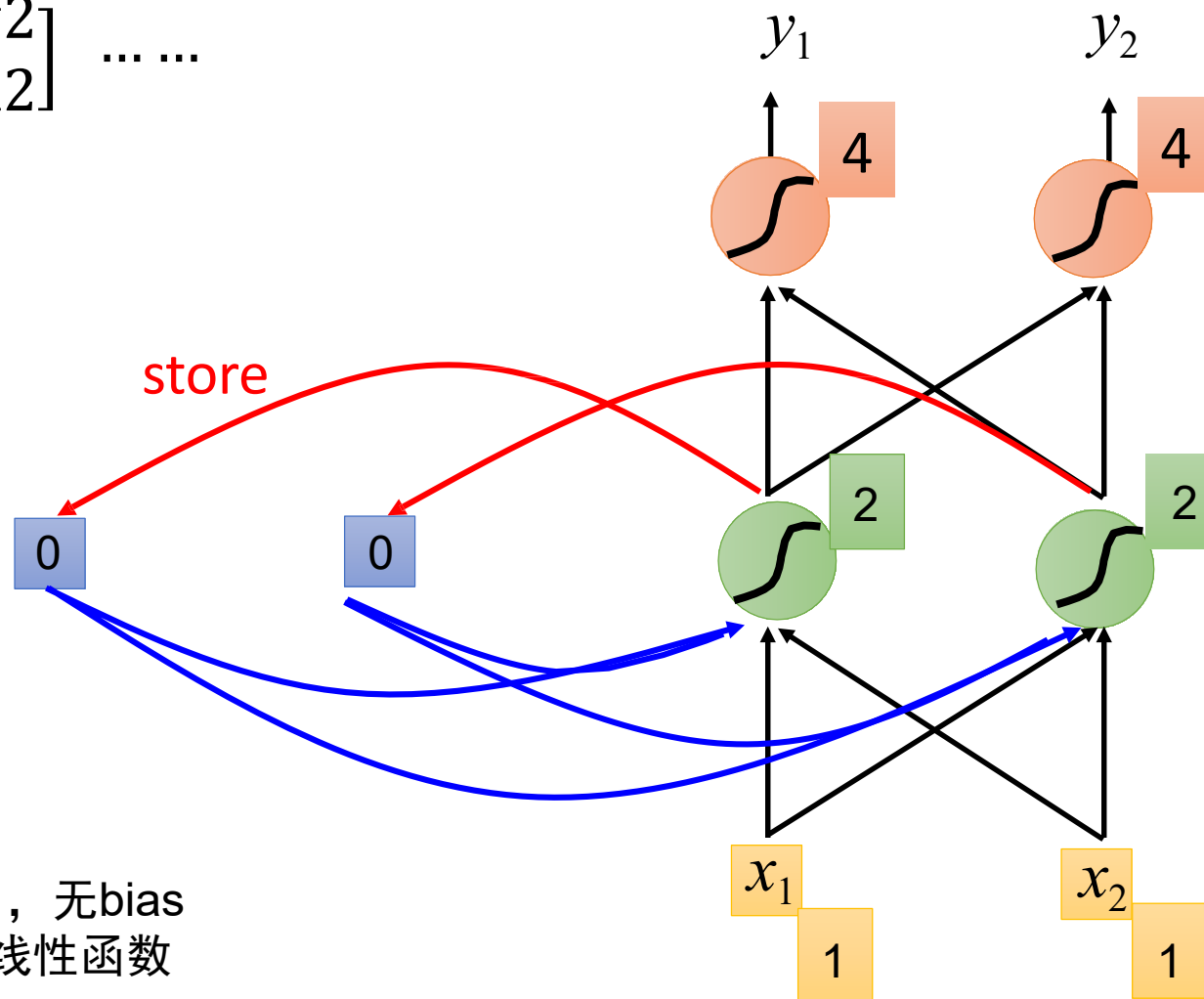


记忆可以被视为额外的输入

RNN

输入: $\begin{bmatrix} 1 \\ 1 \end{bmatrix} \begin{bmatrix} 1 \\ 1 \end{bmatrix} \begin{bmatrix} 2 \\ 2 \end{bmatrix} \dots$

输出: $\begin{bmatrix} 4 \\ 4 \end{bmatrix}$

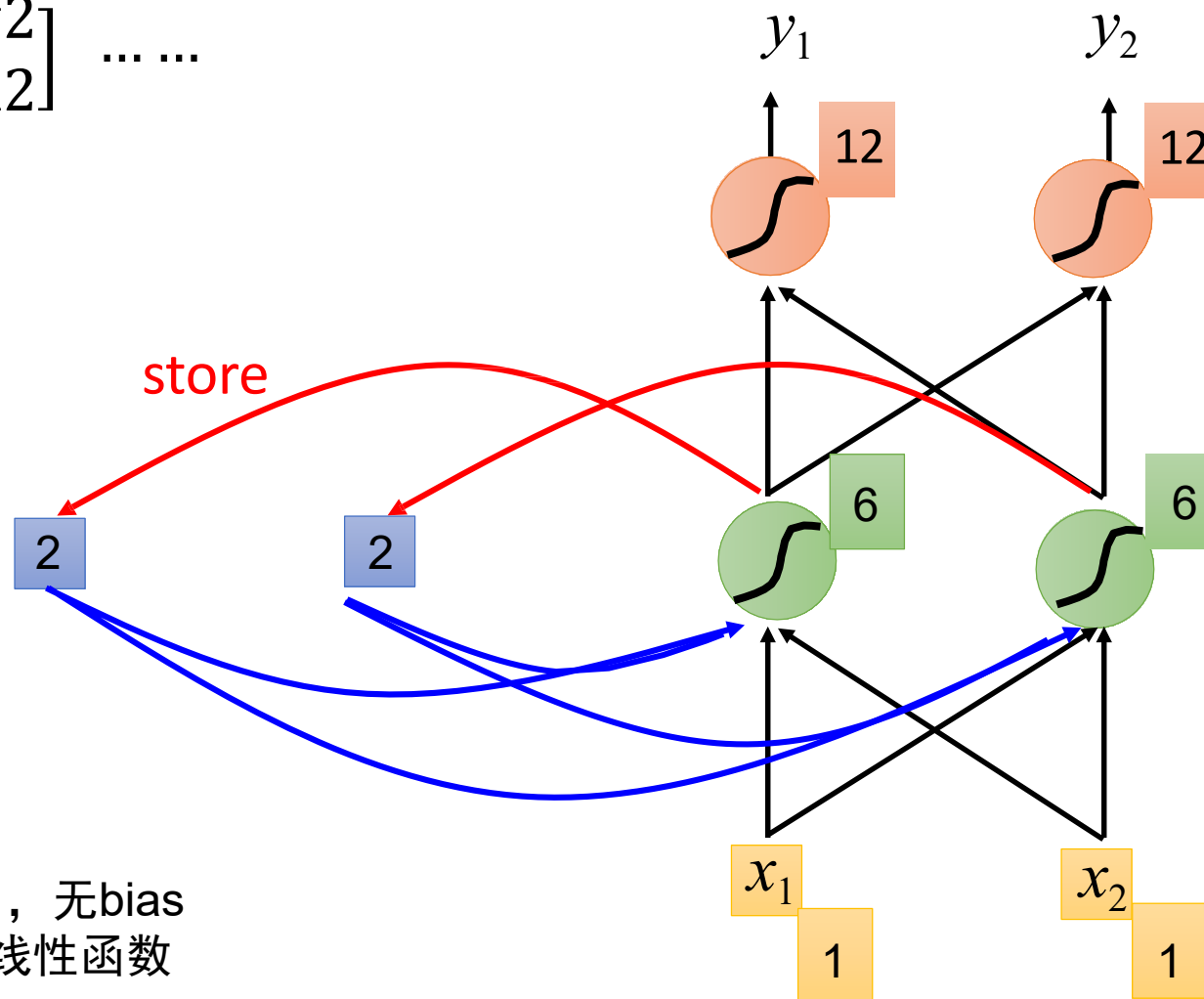


假设所有的weight为1, 无bias
所有的激活函数均为线性函数

RNN

输入: $\begin{bmatrix} 1 \\ 1 \end{bmatrix} \begin{bmatrix} 1 \\ 1 \end{bmatrix} \begin{bmatrix} 2 \\ 2 \end{bmatrix} \dots$

输出: $\begin{bmatrix} 4 \\ 4 \end{bmatrix} \begin{bmatrix} 12 \\ 12 \end{bmatrix}$

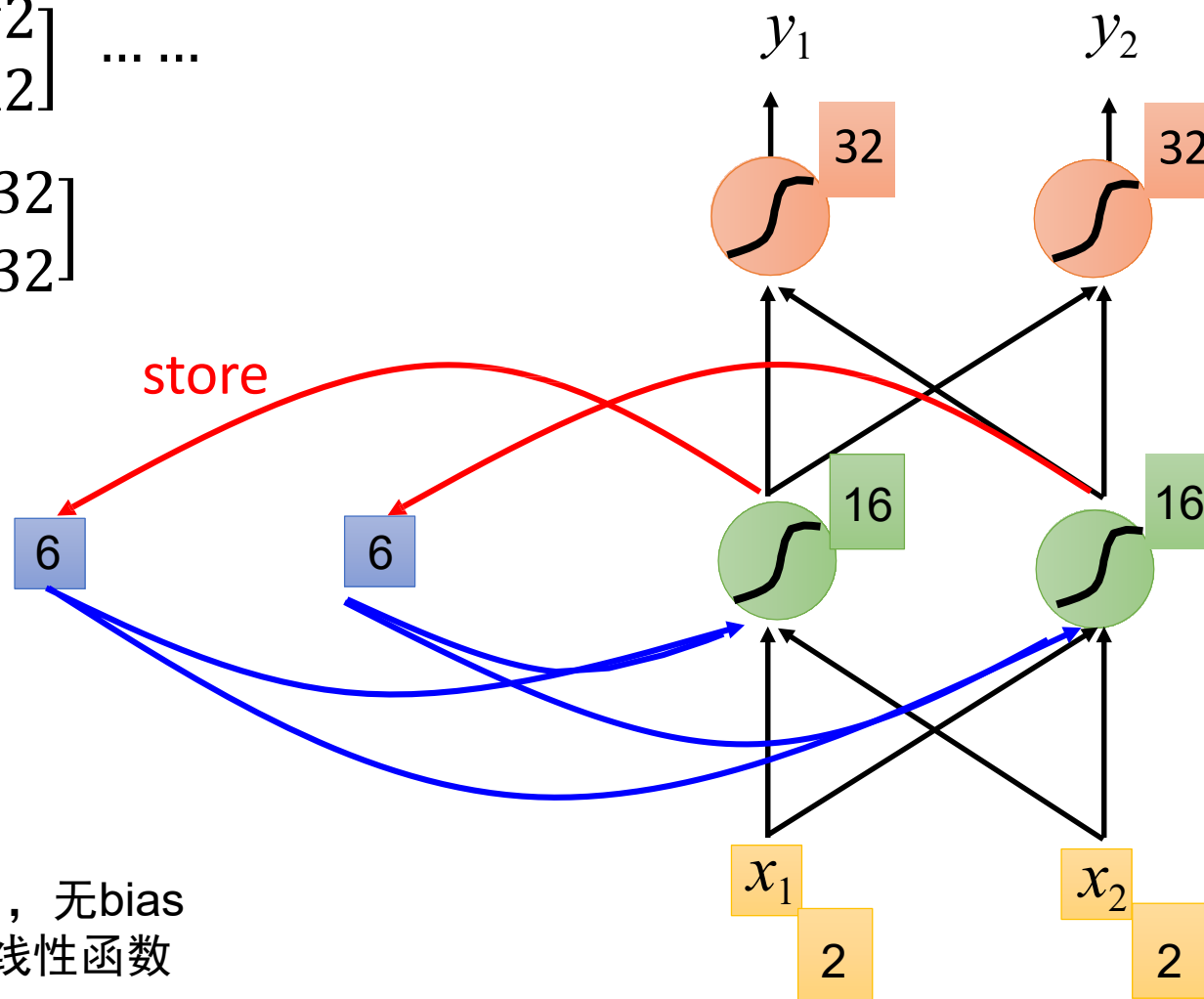


假设所有的weight为1, 无bias
所有的激活函数均为线性函数

改变sequence顺序会改变输出

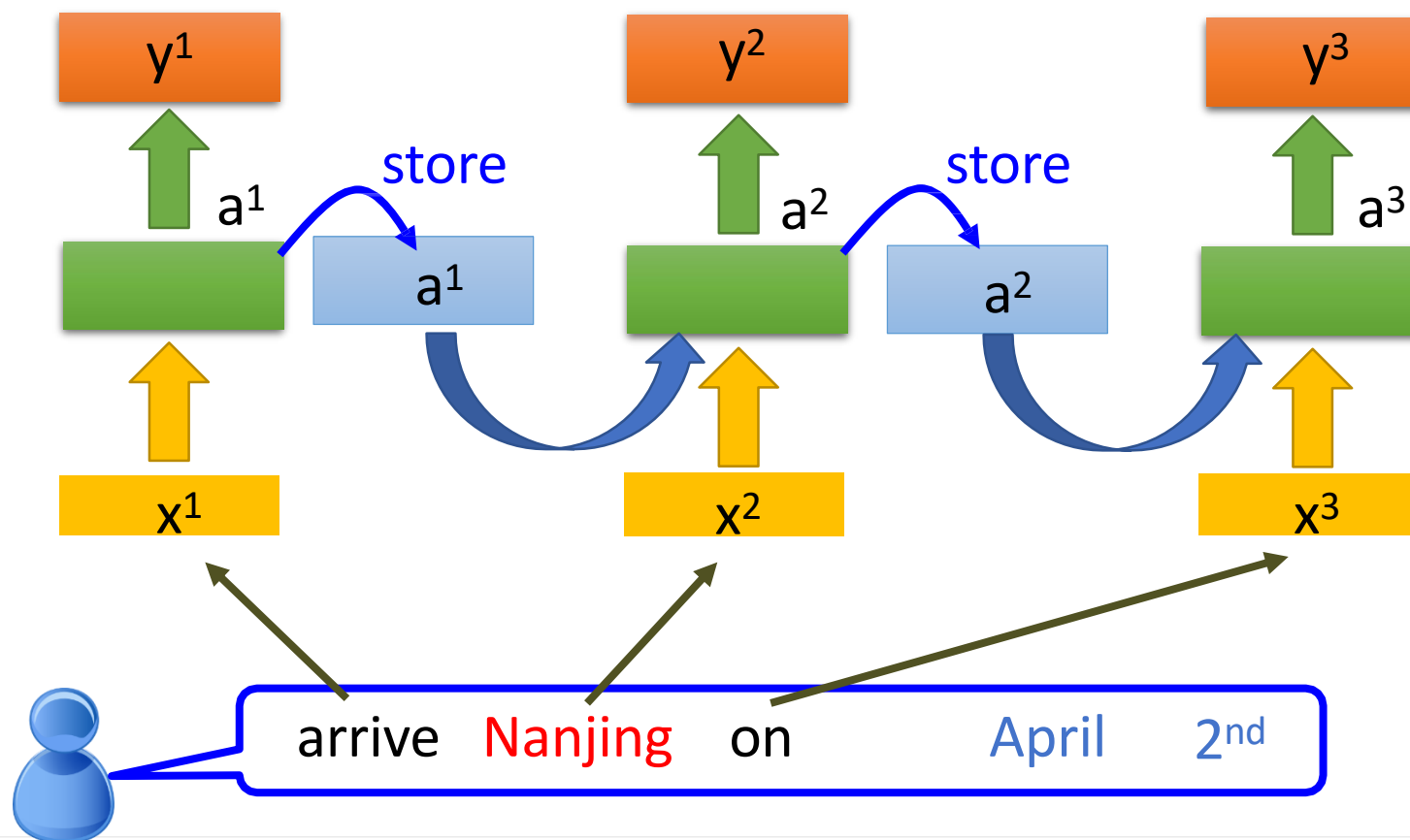
输入: $\begin{bmatrix} 1 \\ 1 \end{bmatrix} \begin{bmatrix} 1 \\ 1 \end{bmatrix} \begin{bmatrix} 2 \\ 2 \end{bmatrix} \dots$

输出: $\begin{bmatrix} 4 \\ 4 \end{bmatrix} \begin{bmatrix} 12 \\ 12 \end{bmatrix} \begin{bmatrix} 32 \\ 32 \end{bmatrix}$

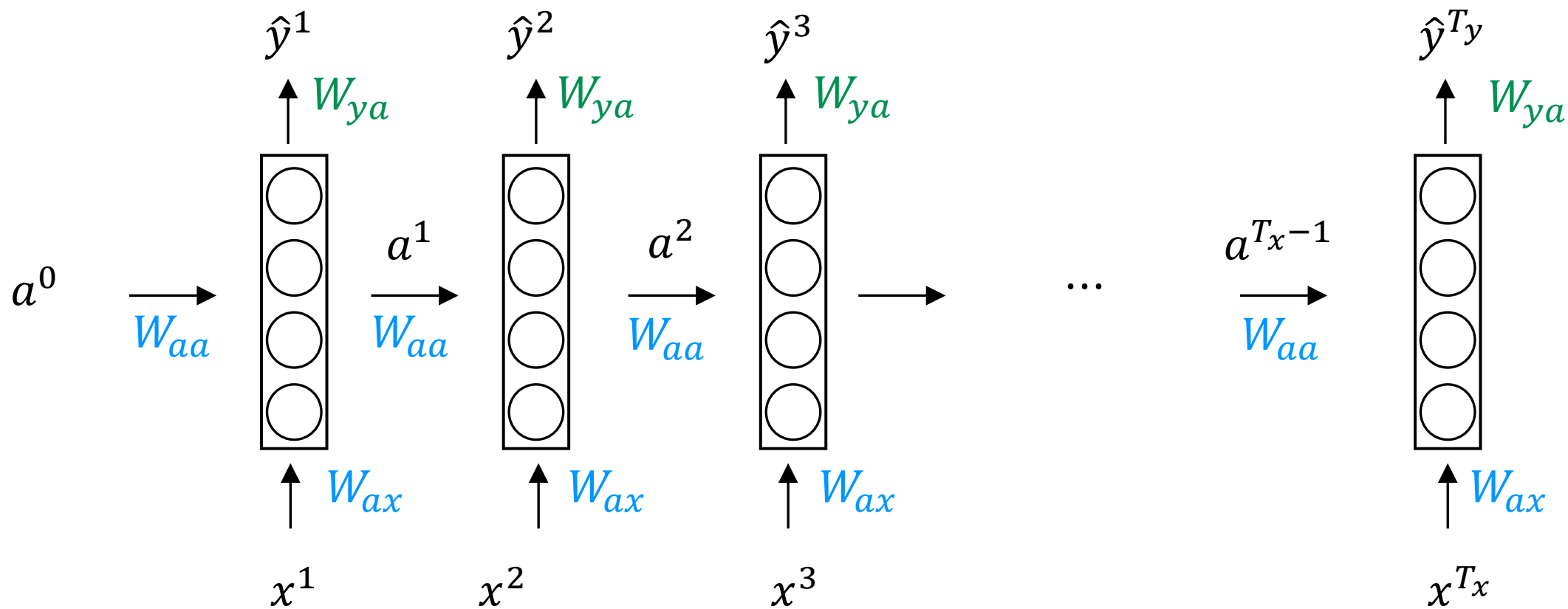


假设所有的weight为1, 无bias
所有的激活函数均为线性函数

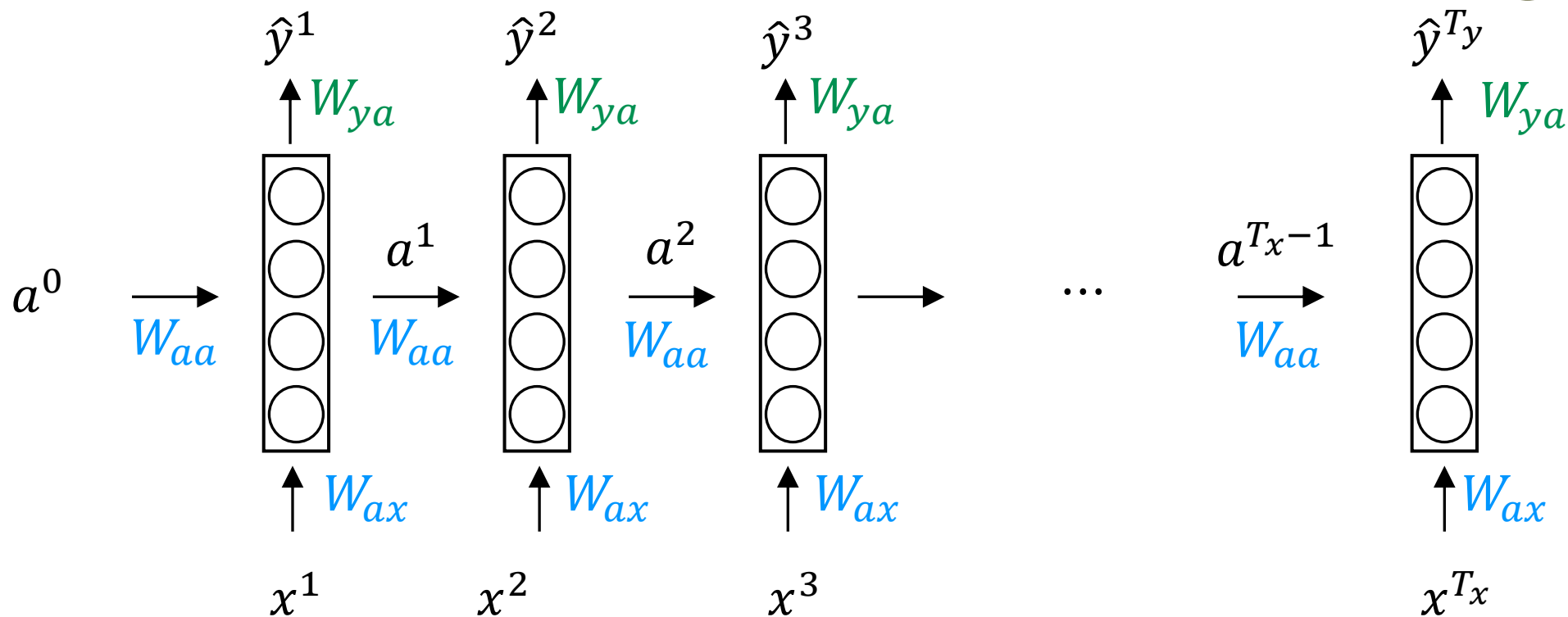
相同的神经网络循环使用！

arrive属于各槽位的
概率分布Nanjing属于各槽位的
概率分布on属于各槽位的
概率分布

RNN



RNN



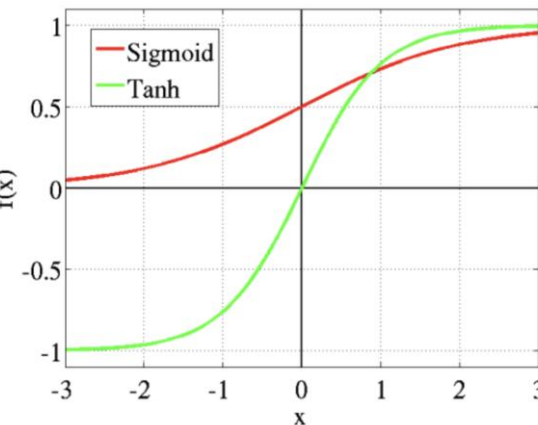
$$a^0 = \mathbf{0}$$

$$a^1 = g(W_{aa}a^0 + W_{ax}x^1 + b_a)$$

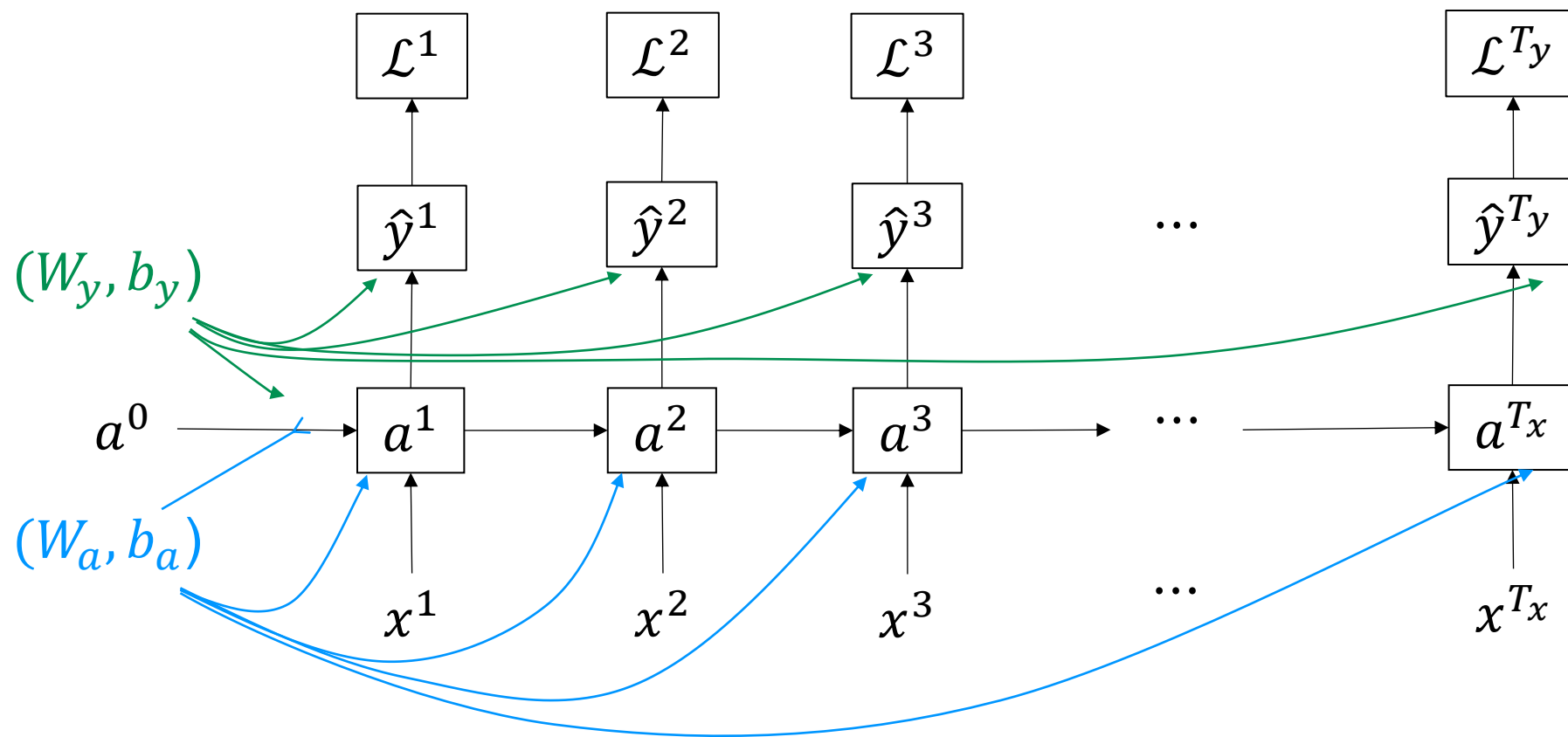
$$\hat{y}^1 = g(W_{ya}a^1 + b_y)$$

$$a^t = g(W_{aa}a^{t-1} + W_{ax}x^t + b_a)$$

$$\hat{y}^t = g(W_{ya}a^t + b_y)$$

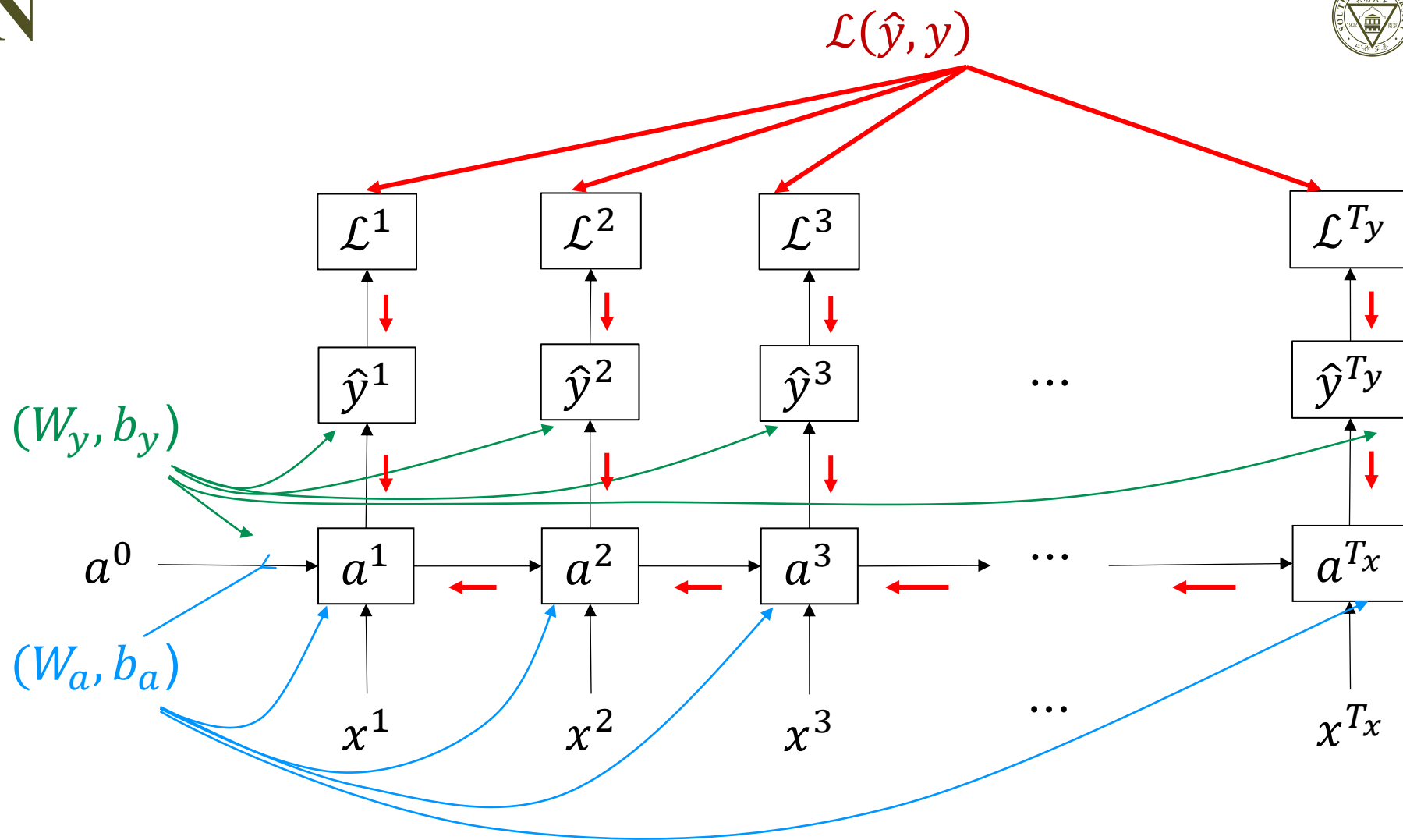


RNN

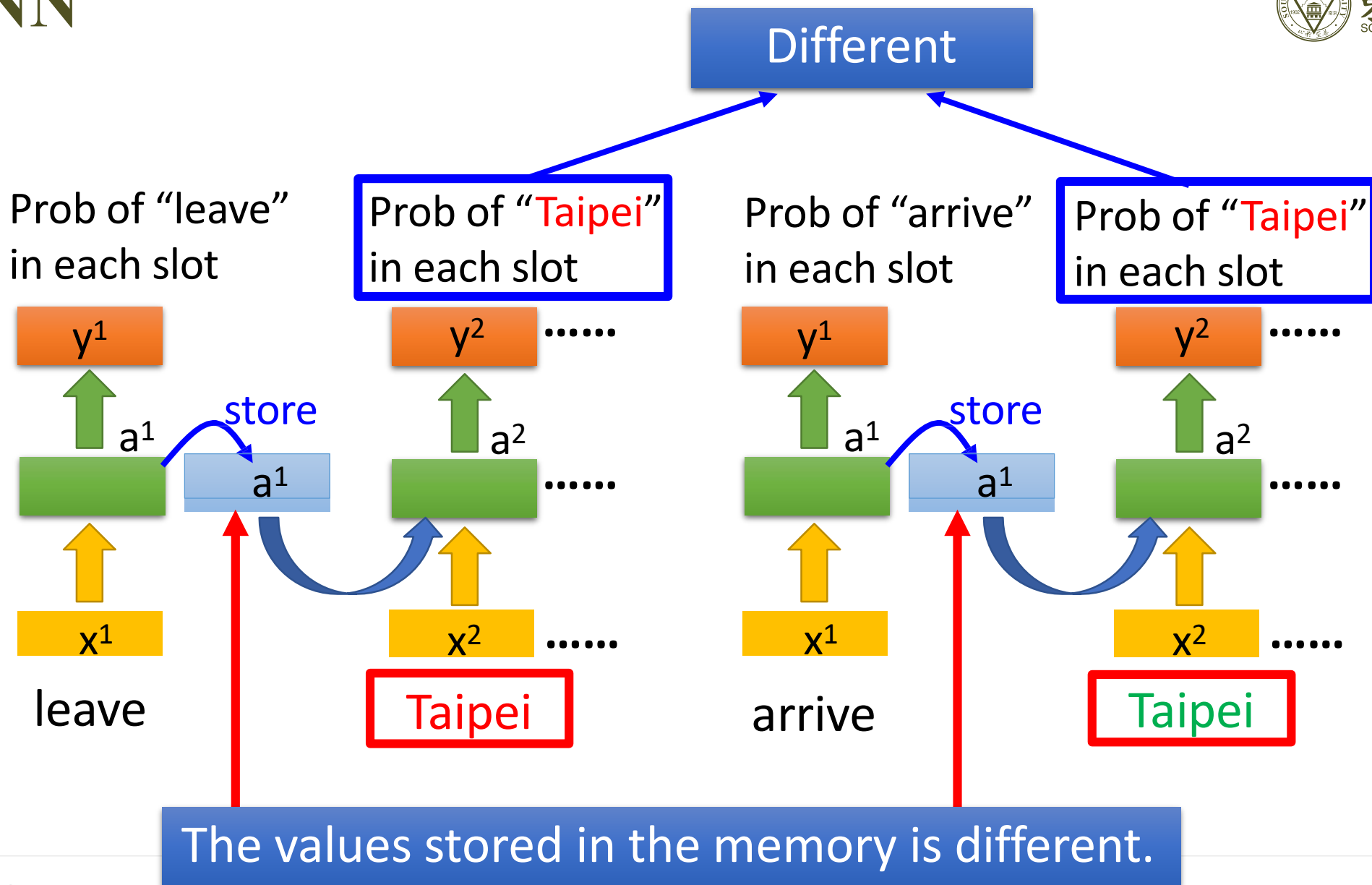


$$\mathcal{L}(\hat{y}, y) = \sum_{t=1}^{T_x} \mathcal{L}^t(\hat{y}^t, y^t)$$

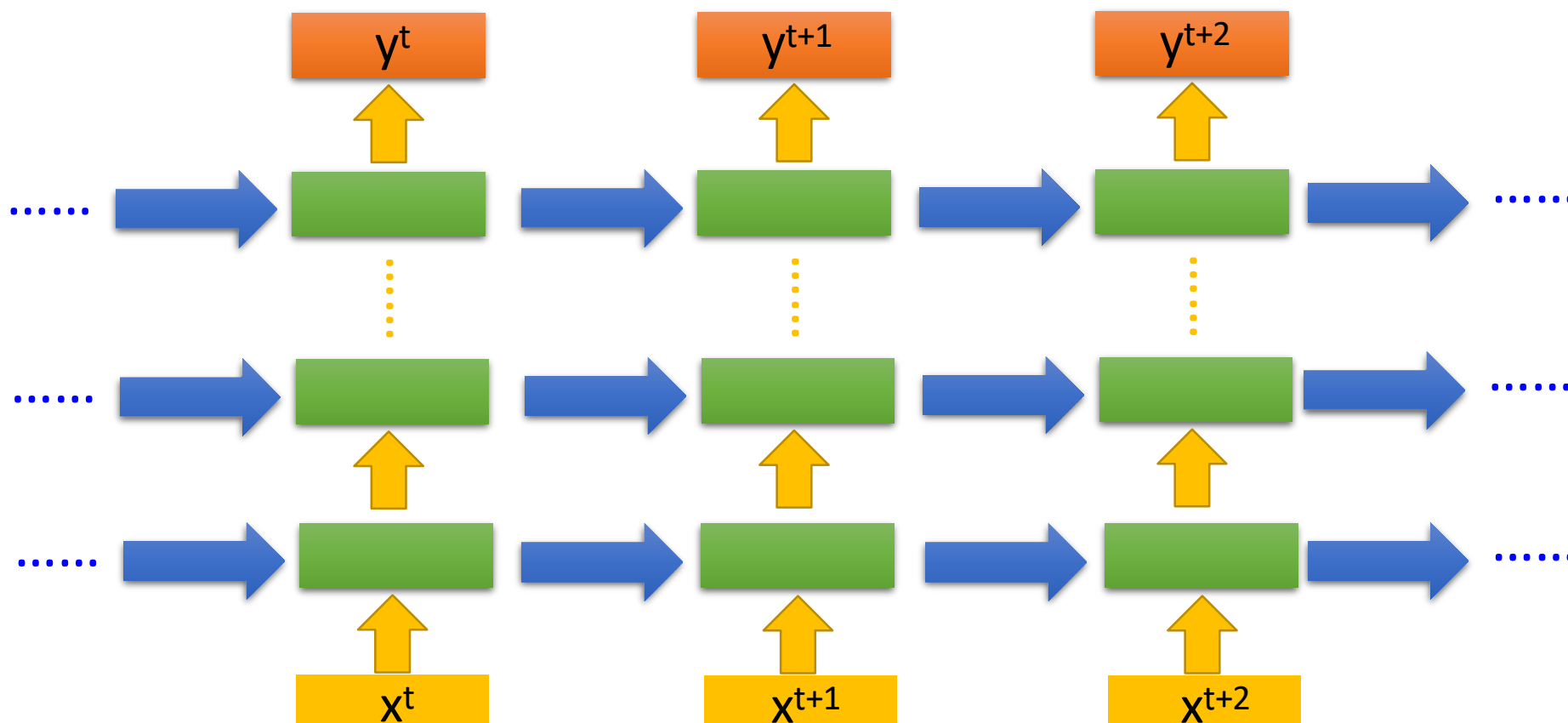
RNN



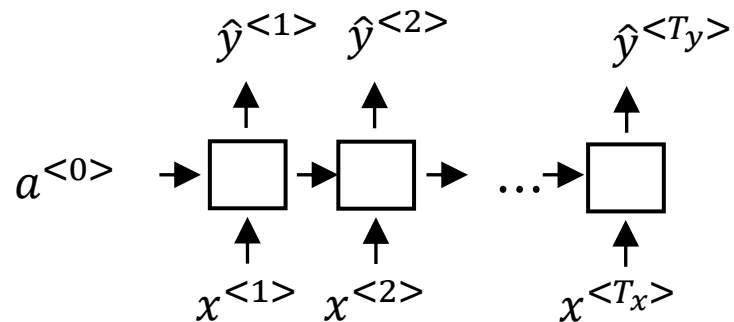
$$\mathcal{L}(\hat{y}, y) = \sum_{t=1}^{T_x} \mathcal{L}^t(\hat{y}^t, y^t)$$



RNN

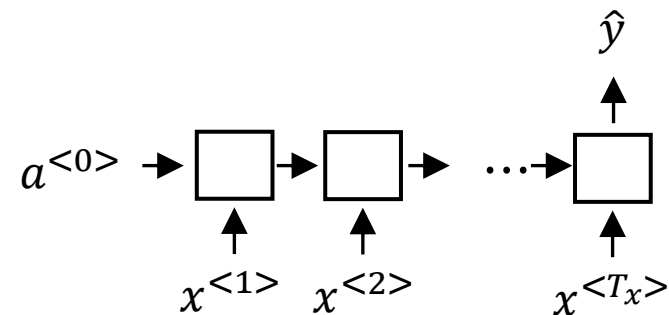


命名实体识别



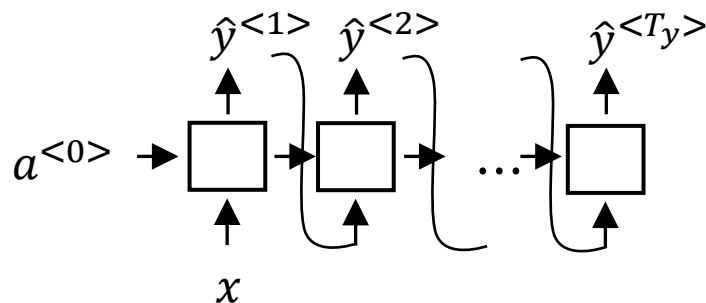
Many to many

情感分类



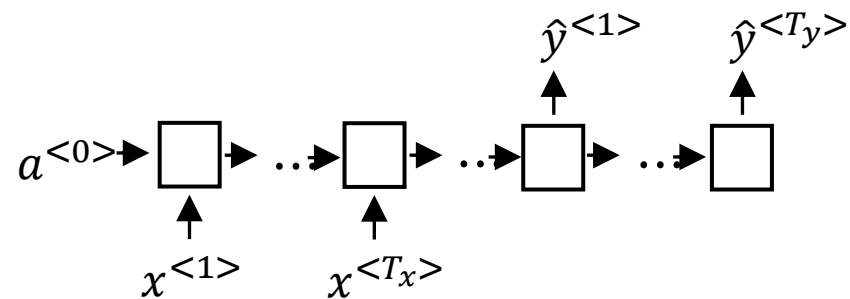
Many to one

音频生成



One to many

机器翻译



Many to many

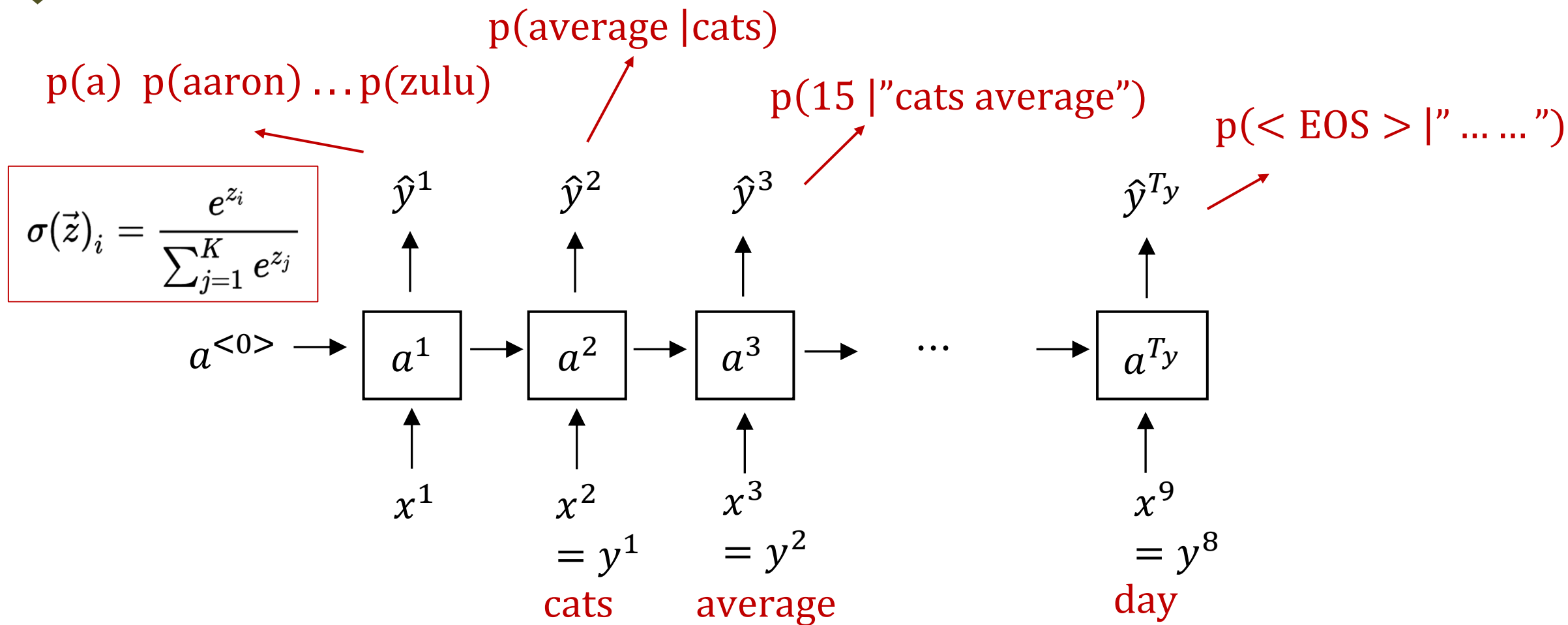
基于 RNN 语言模型

Cats average 15 hours of sleep a day.

- 当输入是 “Begin” 时，输出是 Cats
- 当输入是 Cats 时，输出是 average
- 当输入是 average 时，输出是 15
- 也就是说，第 t 步的输入 x^t ，其实就是第 $t-1$ 步的真实单词 y^{t-1}
- 我们在训练网络预测下一个词

基于 RNN 语言模型

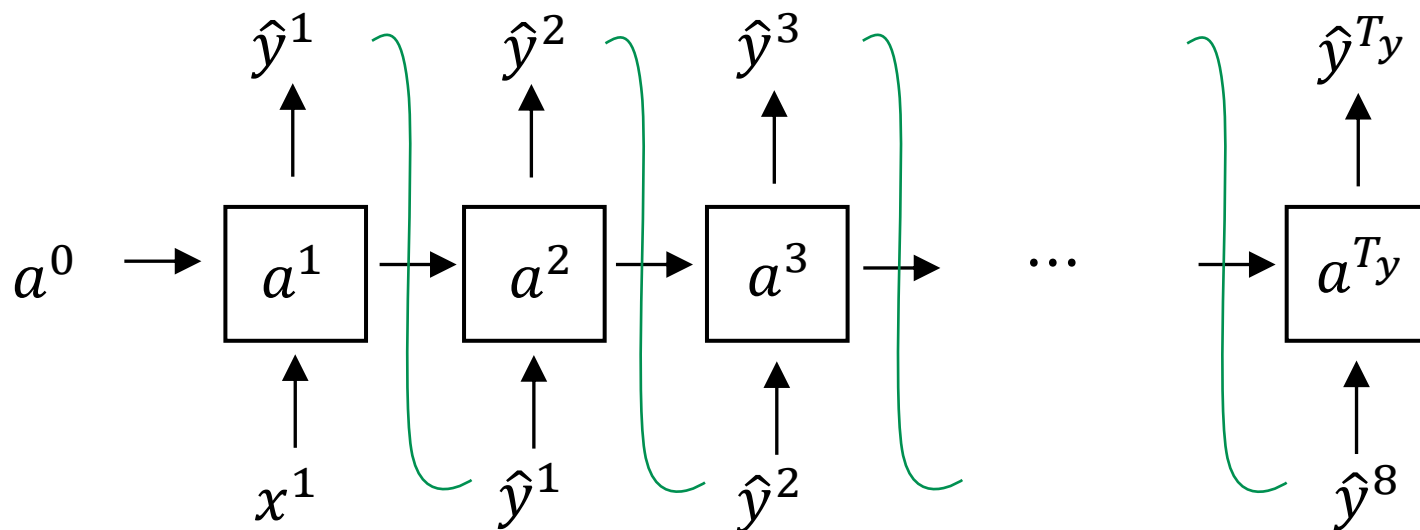
训练: Teacher Forcing



止於至善

$$\mathcal{L}(\hat{y}^t, y^t) = - \sum_i y_i^t \log \hat{y}_i^t$$

$$\mathcal{L} = \sum_t \mathcal{L}^t(\hat{y}^t, y^t)$$

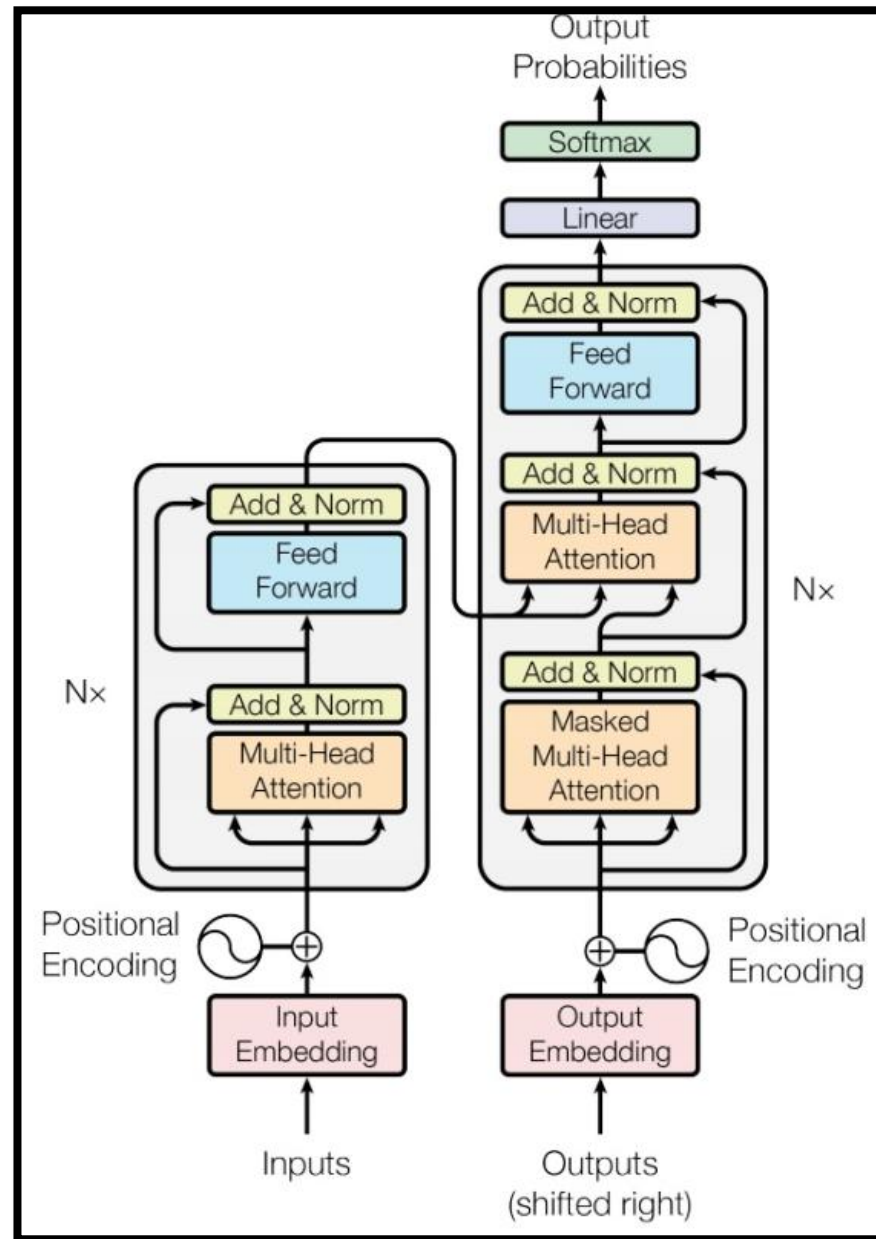
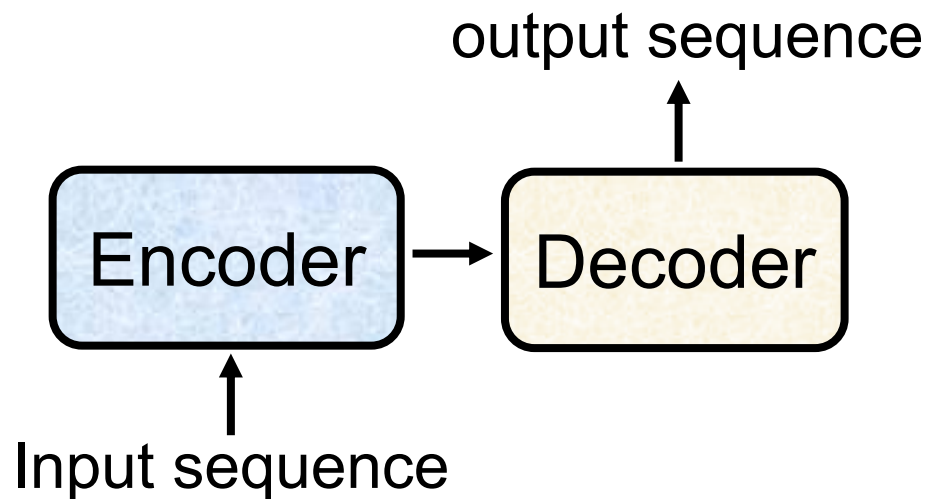


- 给网络一个输入，网络输出第一个词的概率分布（比如 The: 20%, Cats: 10%...）
- 根据这个概率，**随机抽取**一个词
- 假设我们抽到了 The，就把 The 喂给下一个时刻，让它继续猜

目录

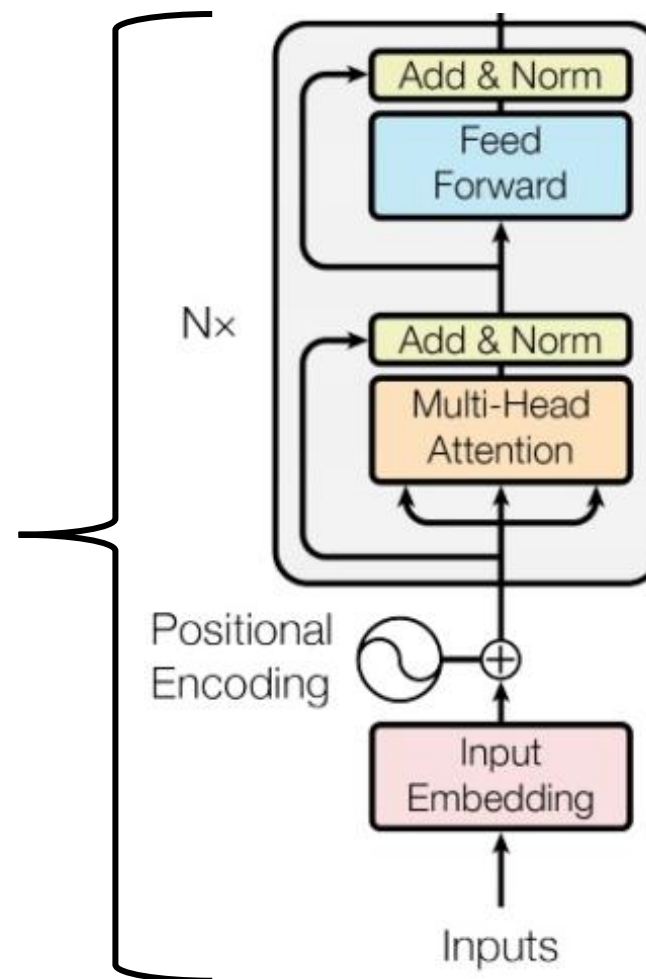
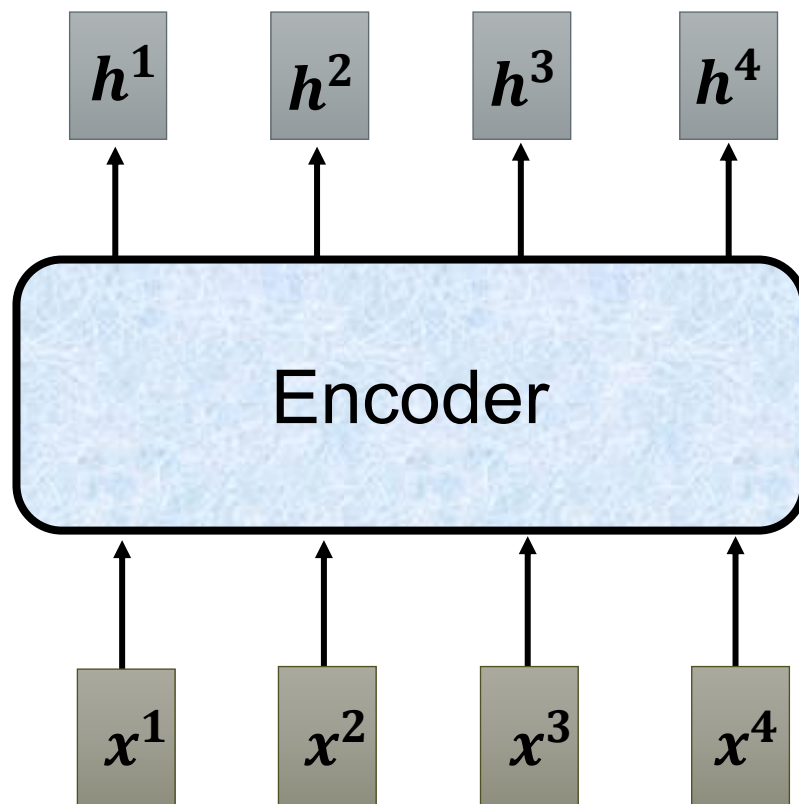
- 神经网络基本概念
- 卷积神经网络
- 循环神经网络
- Transformer

Seq2seq Model

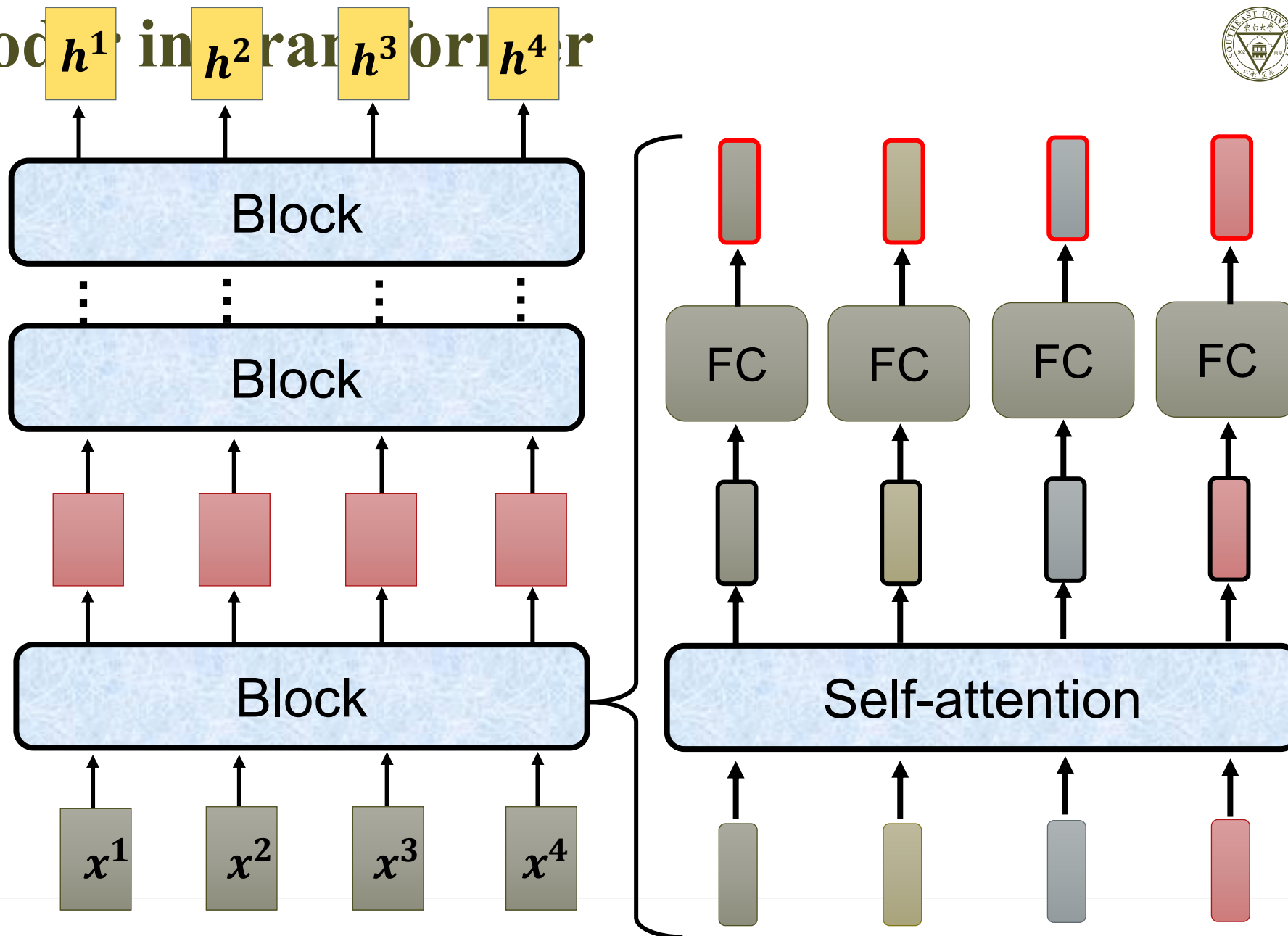


Encoder

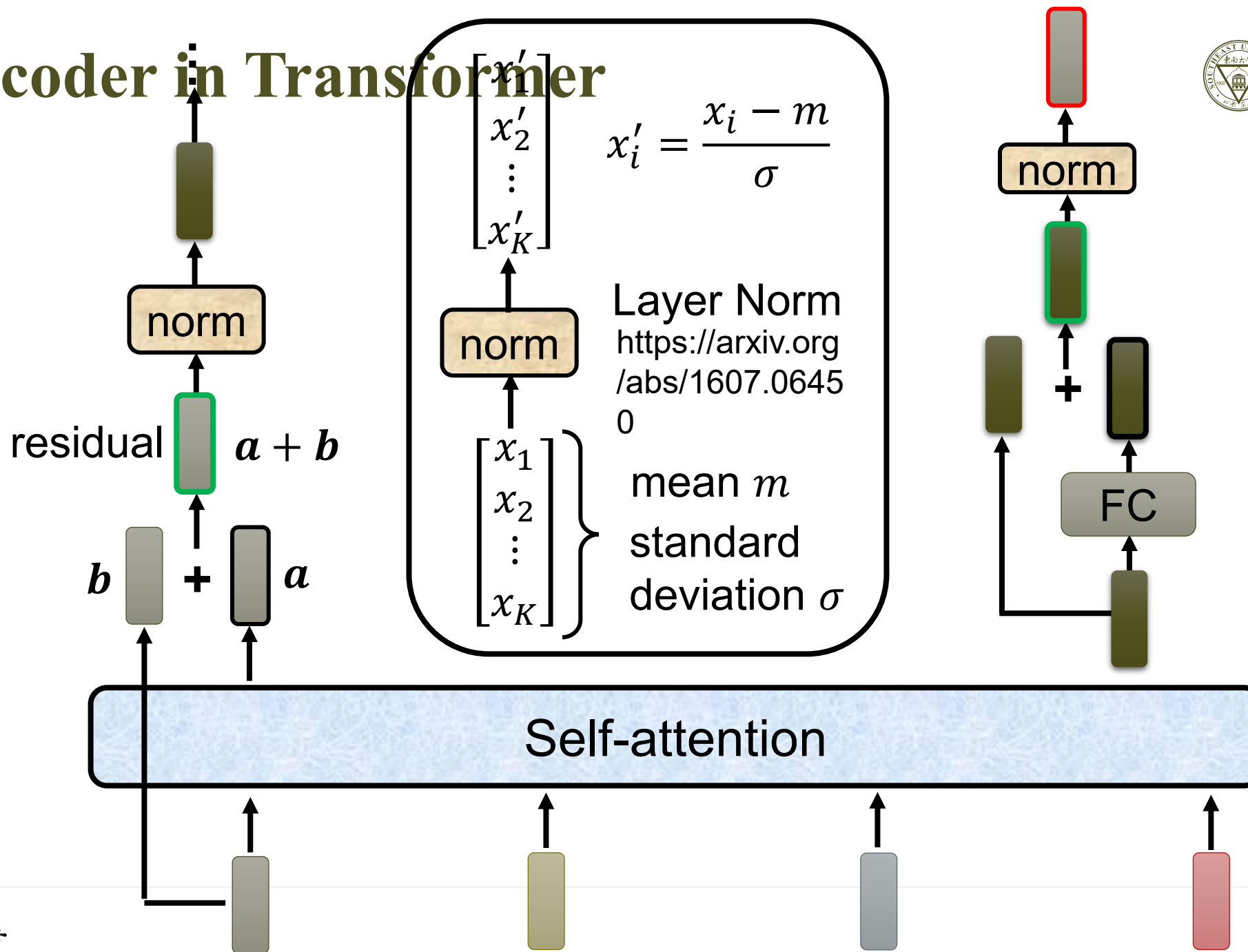
可以用 RNN 或者 CNN 实现



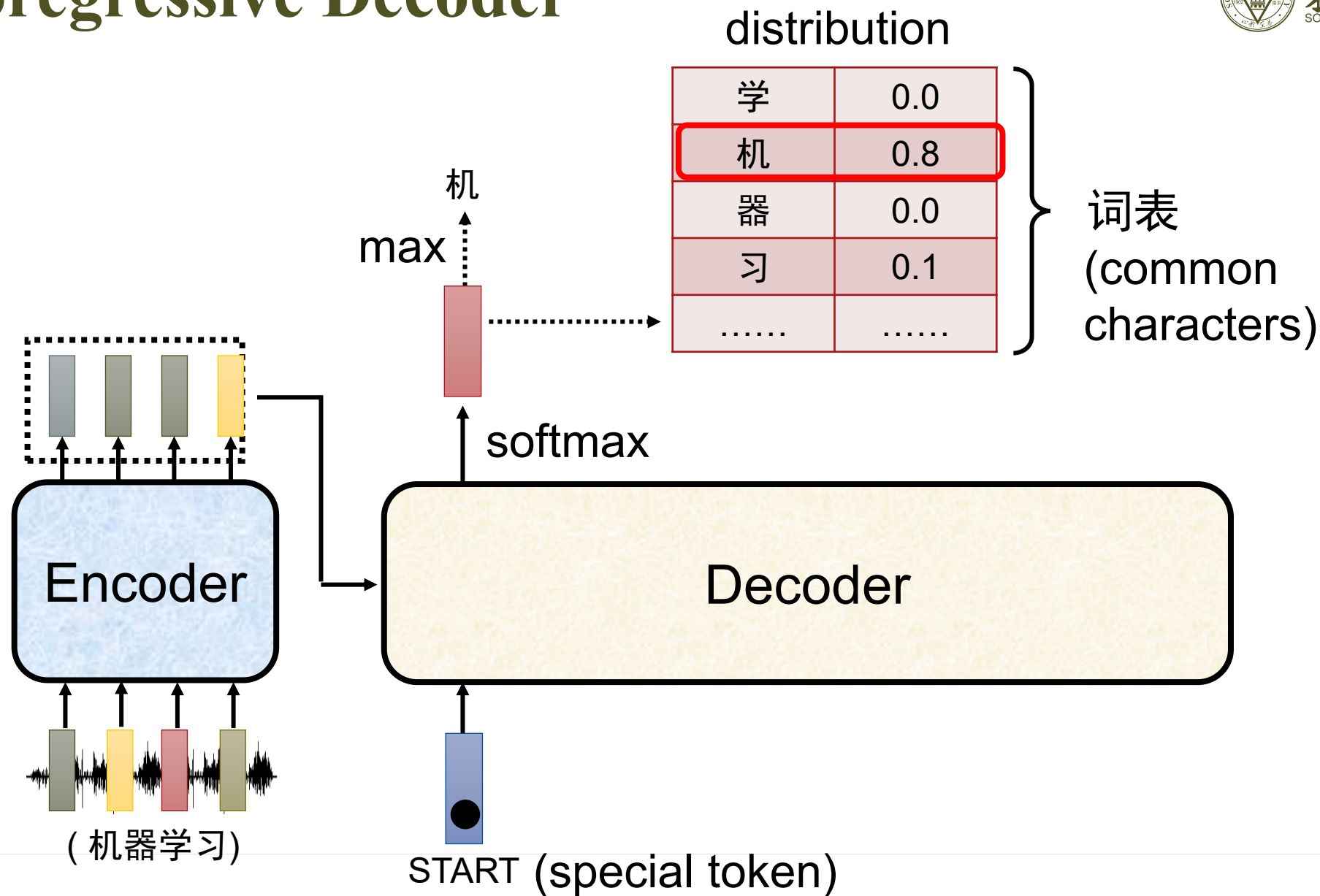
Encoder in parallel



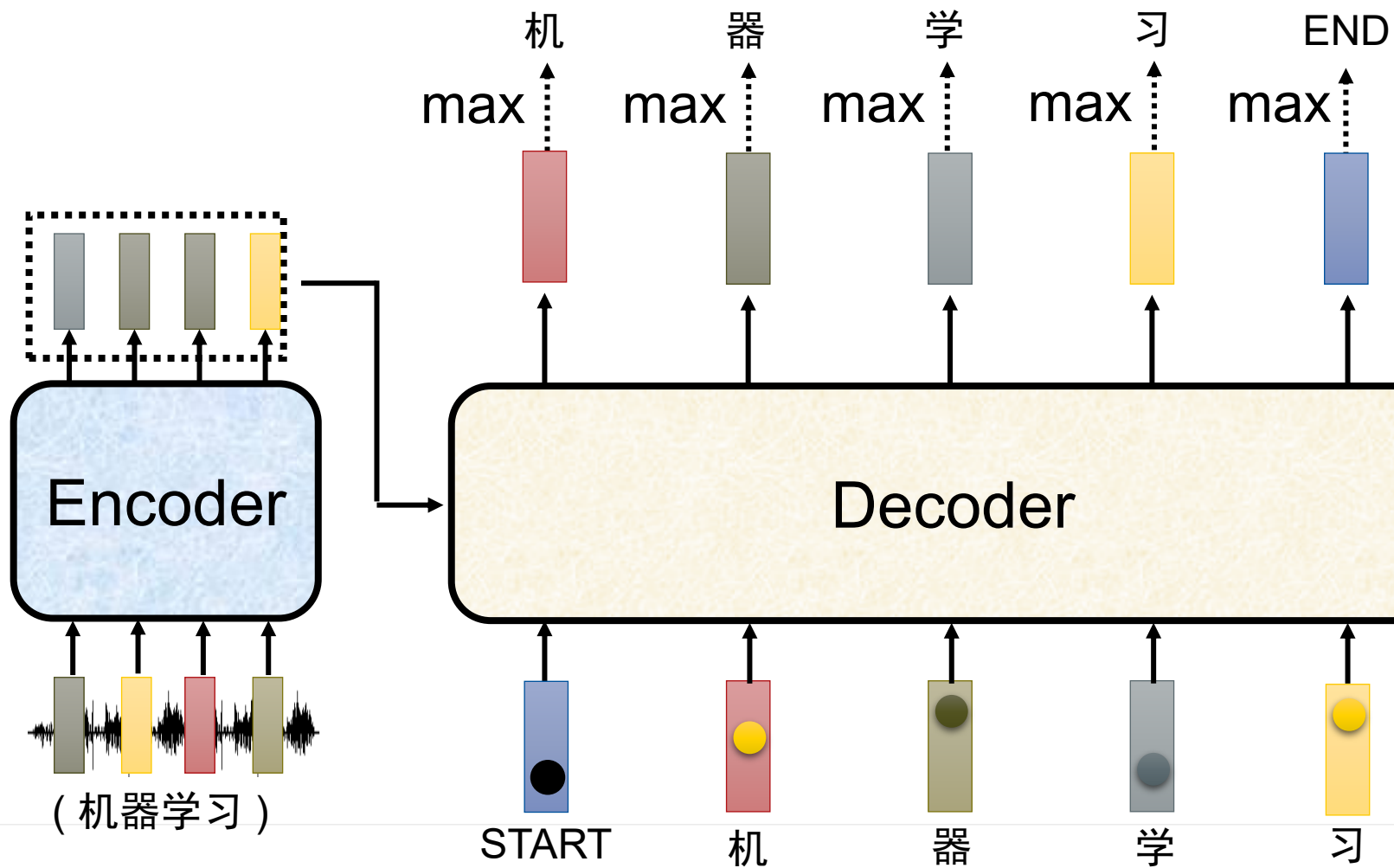
Encoder in Transformer



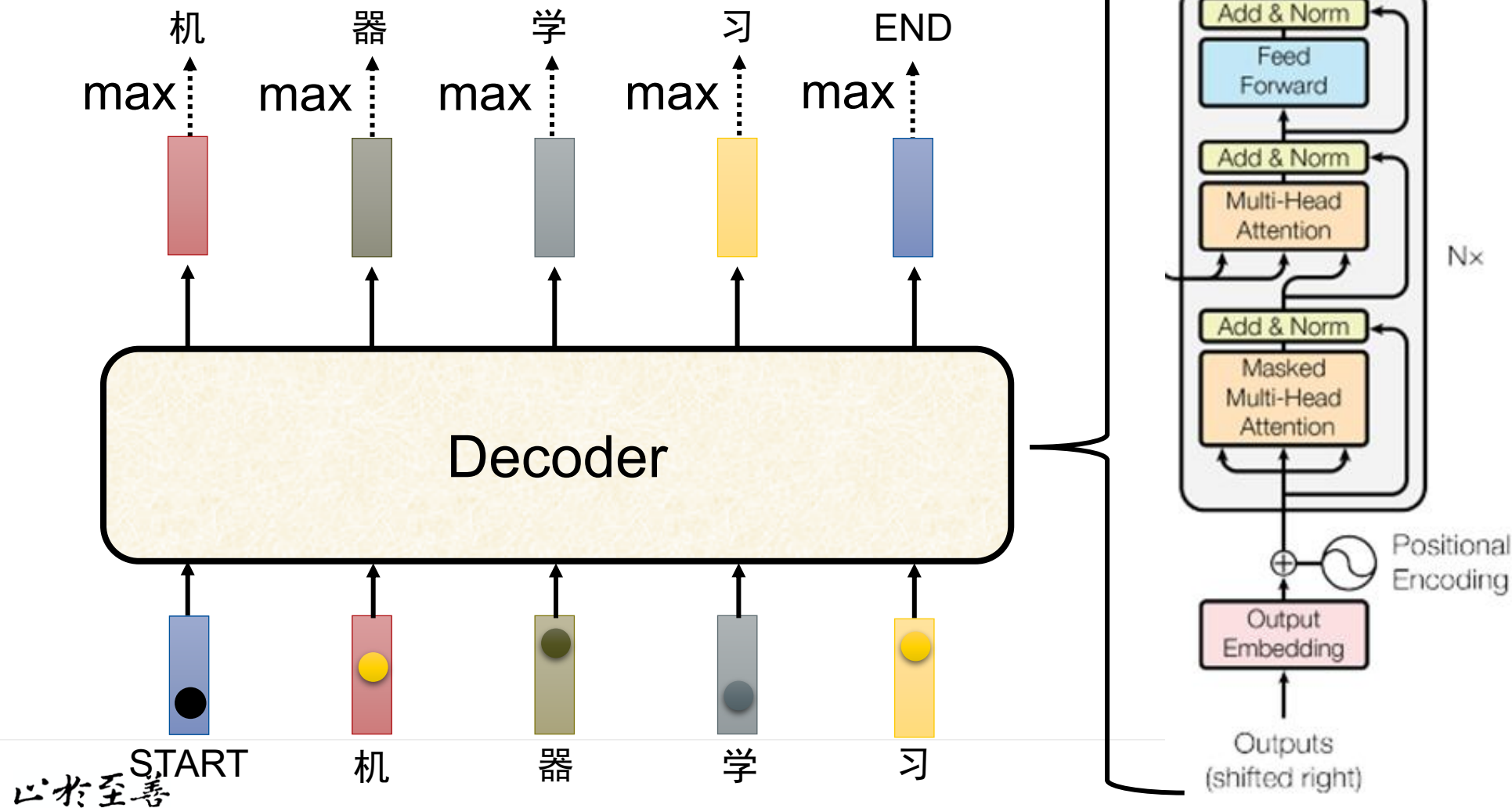
Autoregressive Decoder



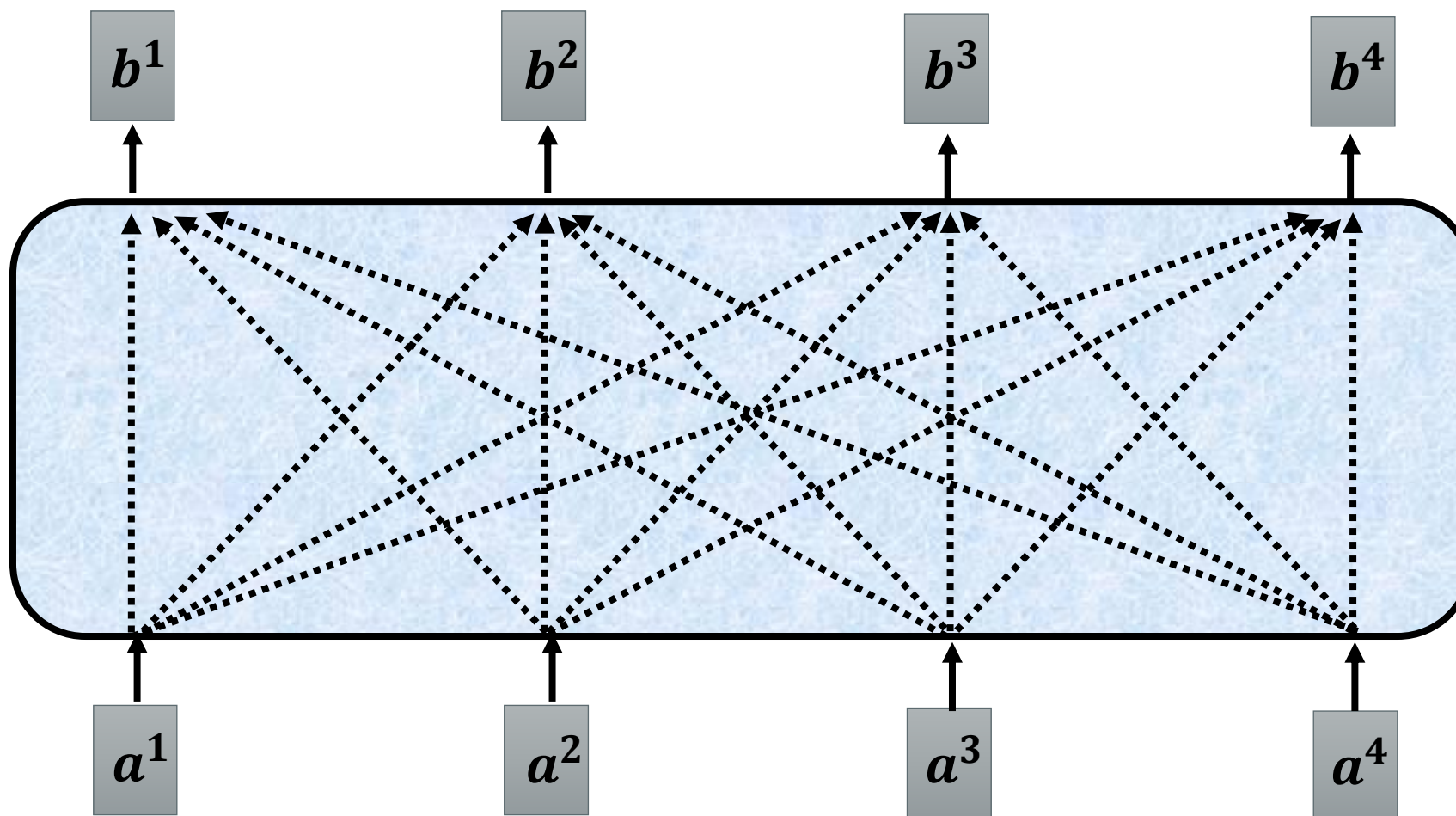
Autoregressive Decoder



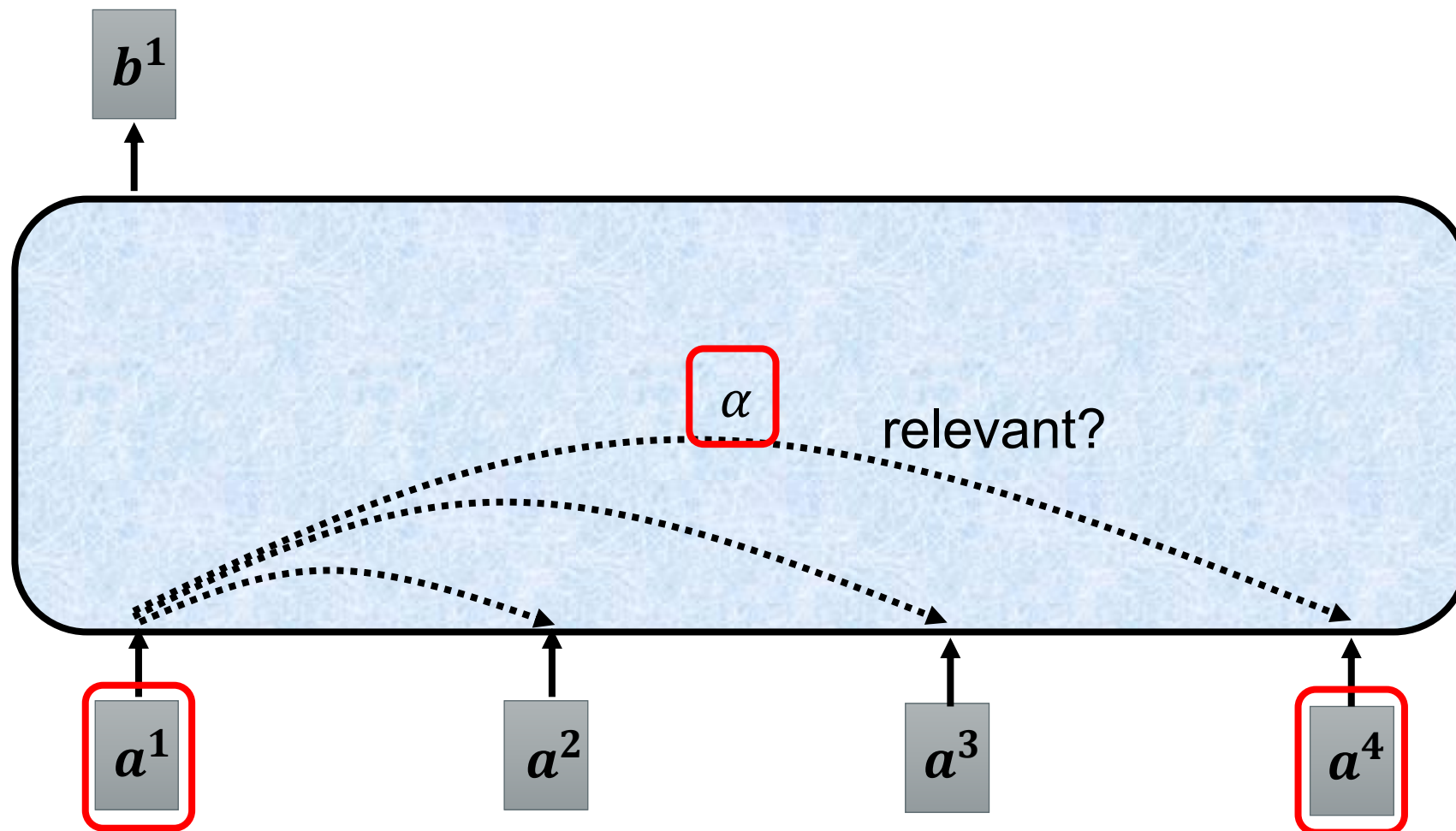
Autoregressive Decoder



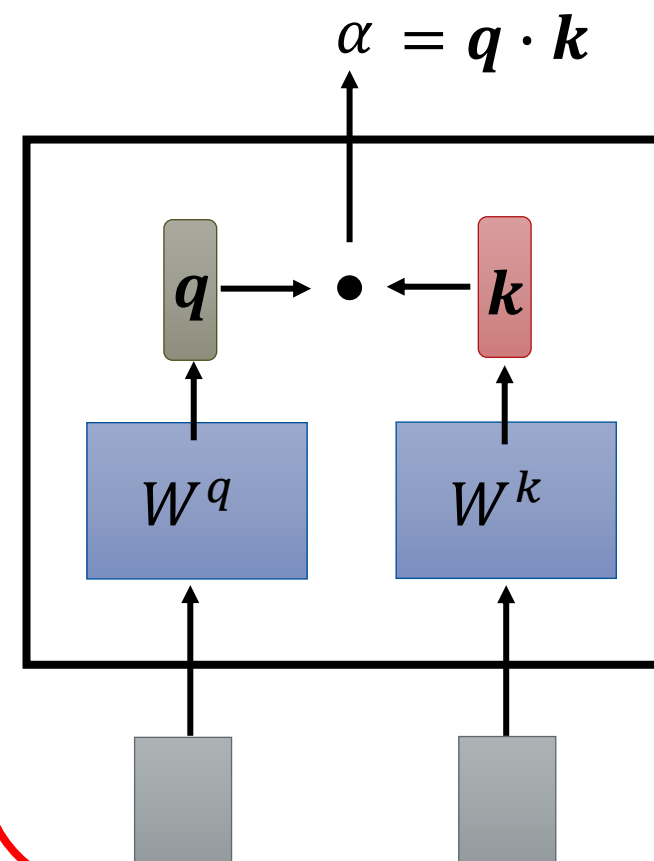
Self-attention



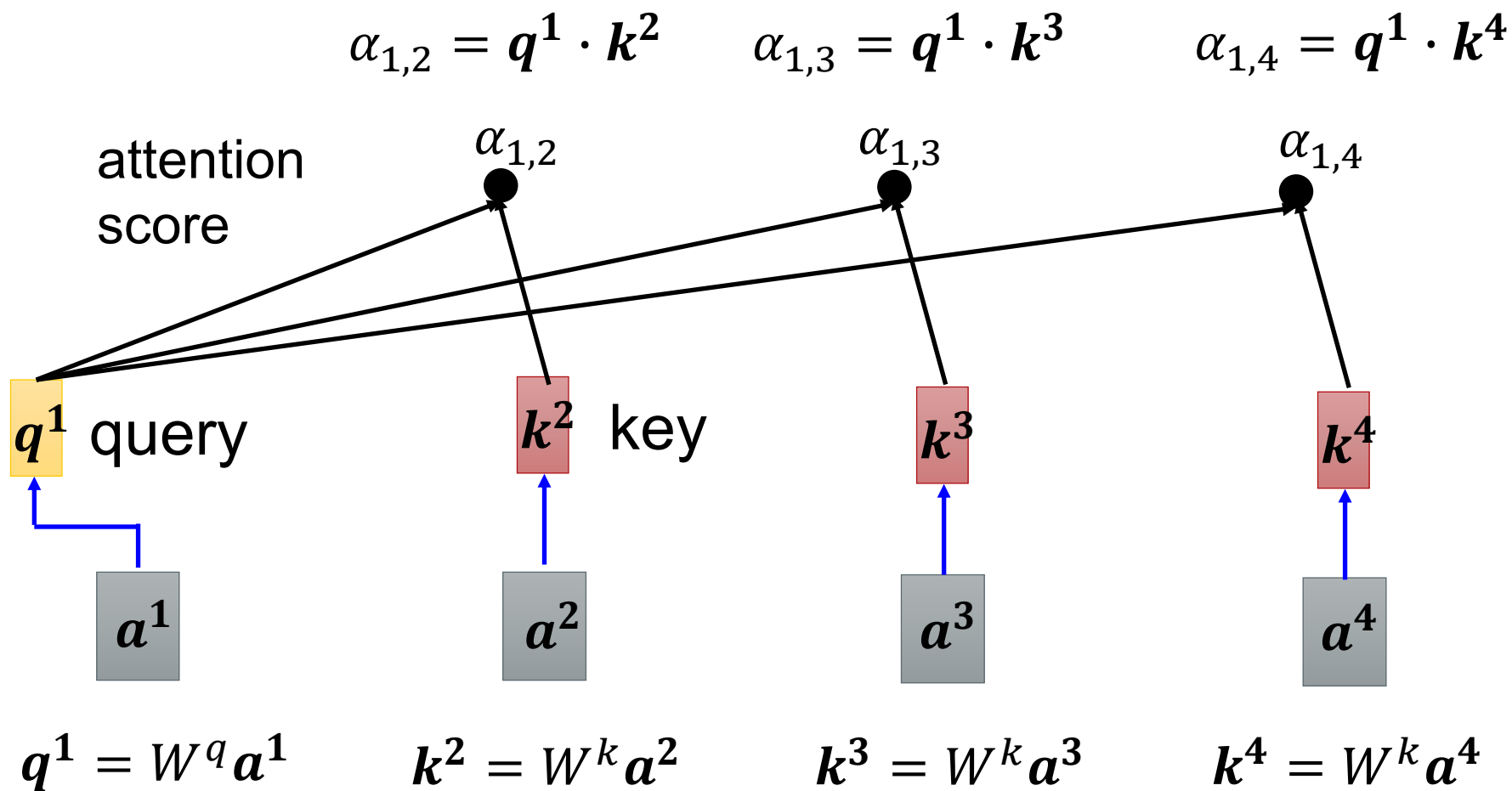
Self-attention



Dot-product



Self-attention

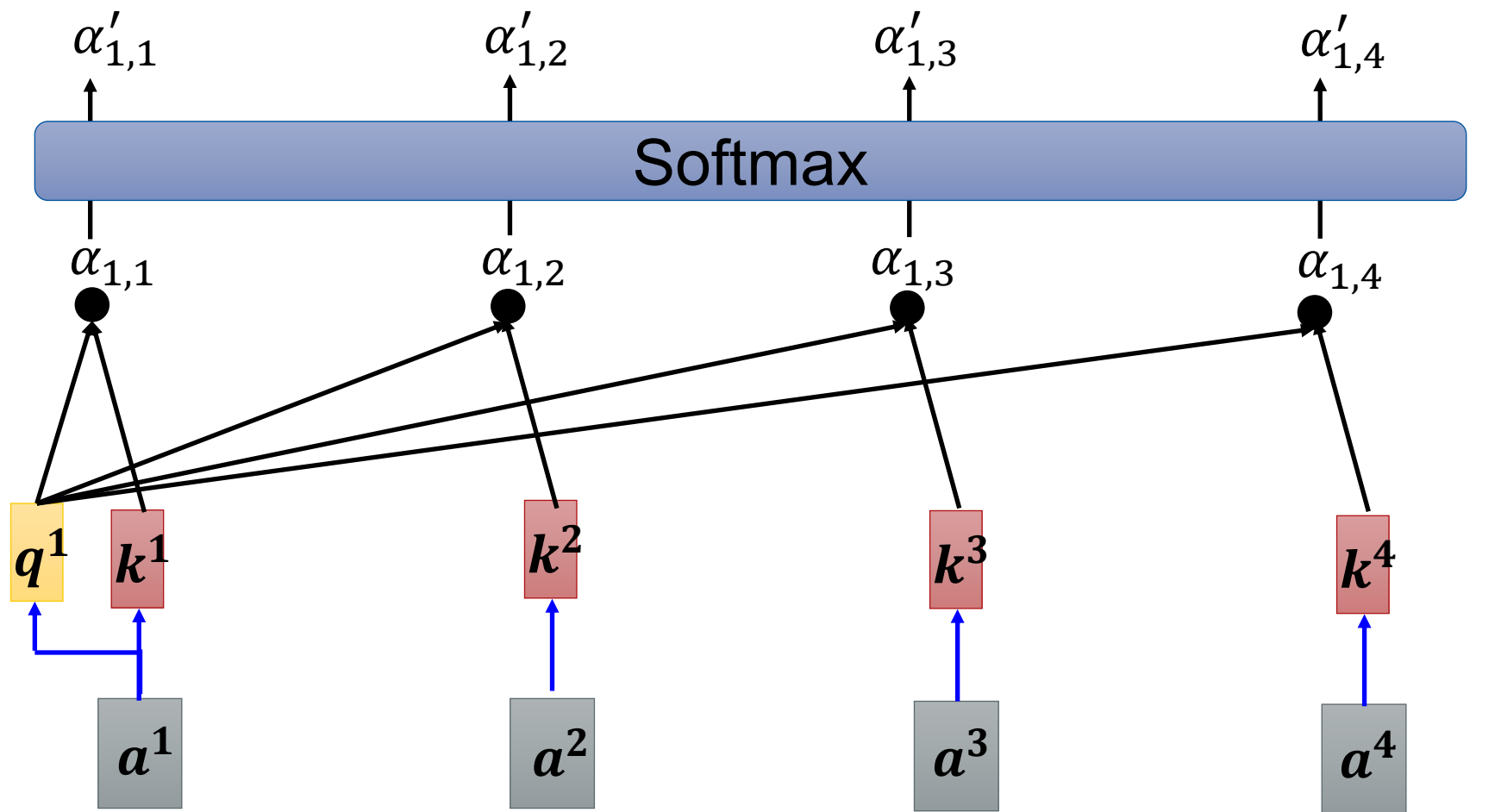


Self-attention

$$\alpha'_{1,i} = \exp(\alpha_{1,i}) / \sum_j \exp(\alpha_{1,j})$$



東南大學
SOUTHEAST UNIVERSITY



$$q^1 = W^q a^1$$

$$k^2 = W^k a^2$$

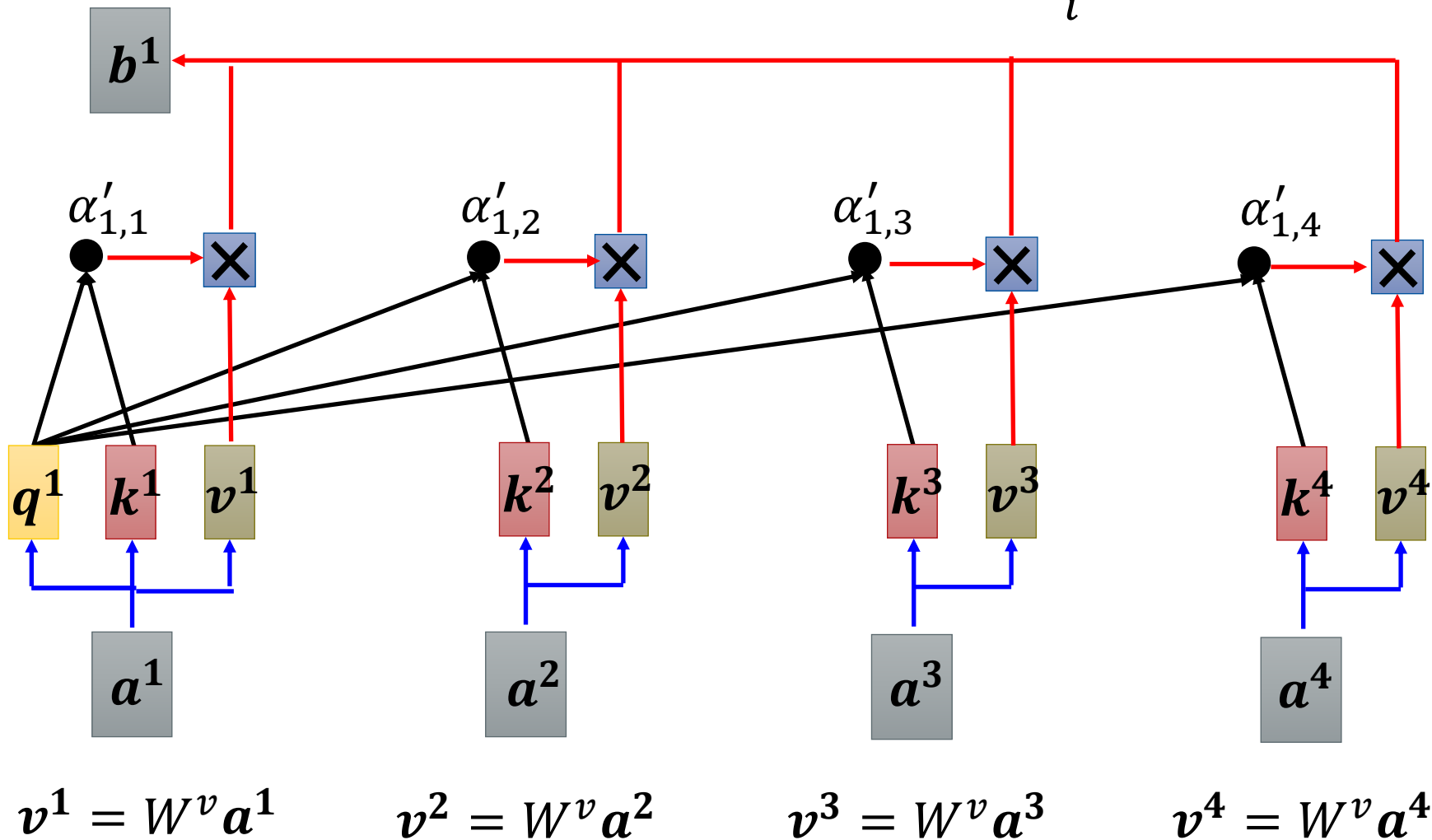
$$k^3 = W^k a^3$$

$$k^4 = W^k a^4$$

止於至善 $k^1 = W^k a^1$

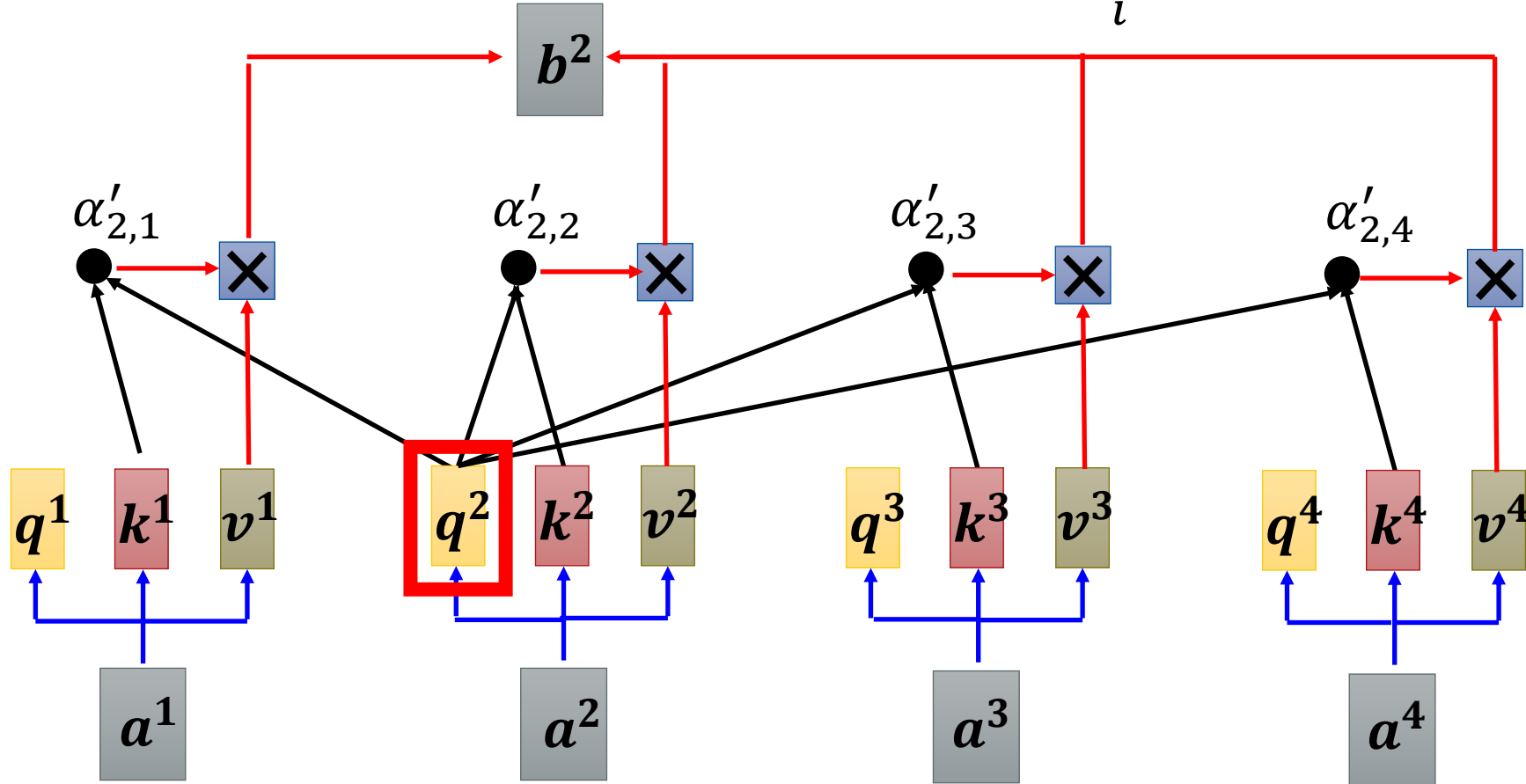
Self-attention

$$b^1 = \sum_i \alpha'_{1,i} v^i$$



Self-attention

$$b^2 = \sum_i \alpha'_{2,i} v^i$$

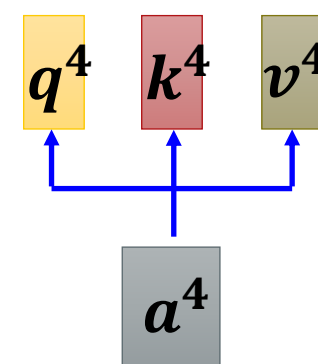
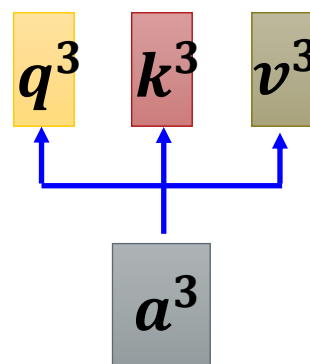
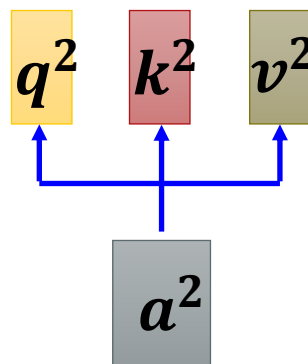
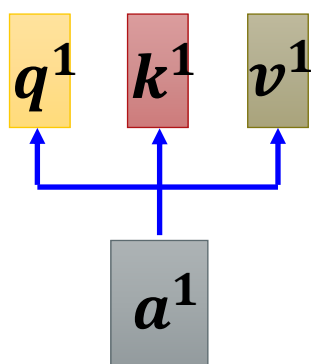


Self-attention

$$q^i = W^q a^i \quad \begin{matrix} q^1 & q^2 & q^3 & q^4 \\ \hline Q \end{matrix} = \begin{matrix} W^q & a^1 & a^2 & a^3 & a^4 \\ \hline I \end{matrix}$$

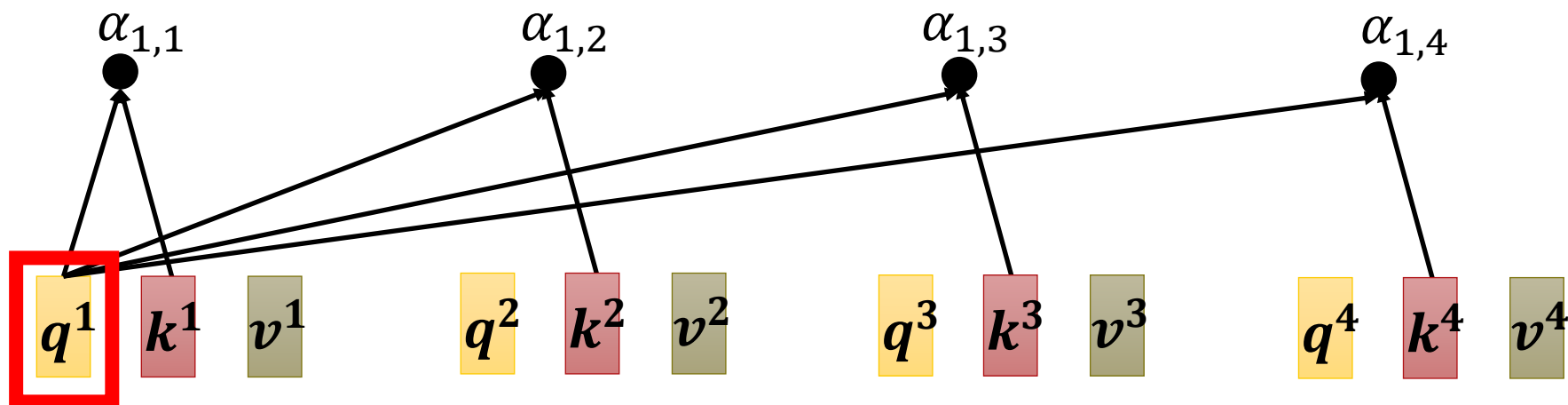
$$k^i = W^k a^i \quad \begin{matrix} k^1 & k^2 & k^3 & k^4 \\ \hline K \end{matrix} = \begin{matrix} W^k & a^1 & a^2 & a^3 & a^4 \\ \hline I \end{matrix}$$

$$v^i = W^v a^i \quad \begin{matrix} v^1 & v^2 & v^3 & v^4 \\ \hline V \end{matrix} = \begin{matrix} W^v & a^1 & a^2 & a^3 & a^4 \\ \hline I \end{matrix}$$



Self-attention

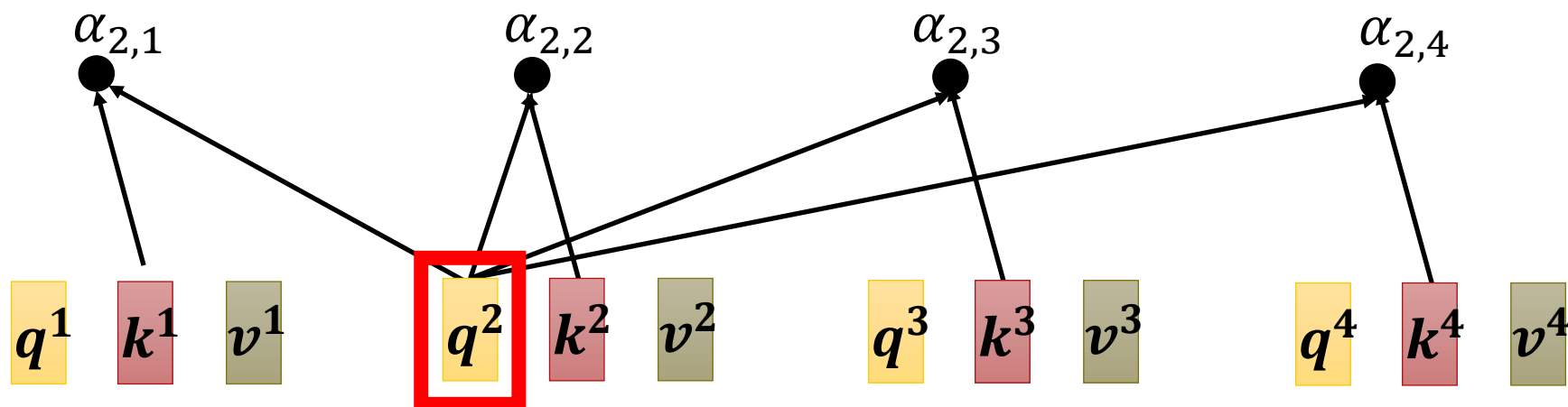
$$\begin{aligned}
 \alpha_{1,1} &= k^1 q^1 & \alpha_{1,2} &= k^2 q^1 \\
 \alpha_{1,3} &= k^3 q^1 & \alpha_{1,4} &= k^4 q^1
 \end{aligned}
 \quad
 \begin{matrix} \alpha_{1,1} \\ \alpha_{1,2} \\ \alpha_{1,3} \\ \alpha_{1,4} \end{matrix}
 =
 \begin{matrix} k^1 \\ k^2 \\ k^3 \\ k^4 \end{matrix}
 q^1$$



Self-attention

$$\begin{aligned}\alpha_{1,1} &= k^1 q^1 & \alpha_{1,2} &= k^2 q^1 \\ \alpha_{1,3} &= k^3 q^1 & \alpha_{1,4} &= k^4 q^1\end{aligned}$$

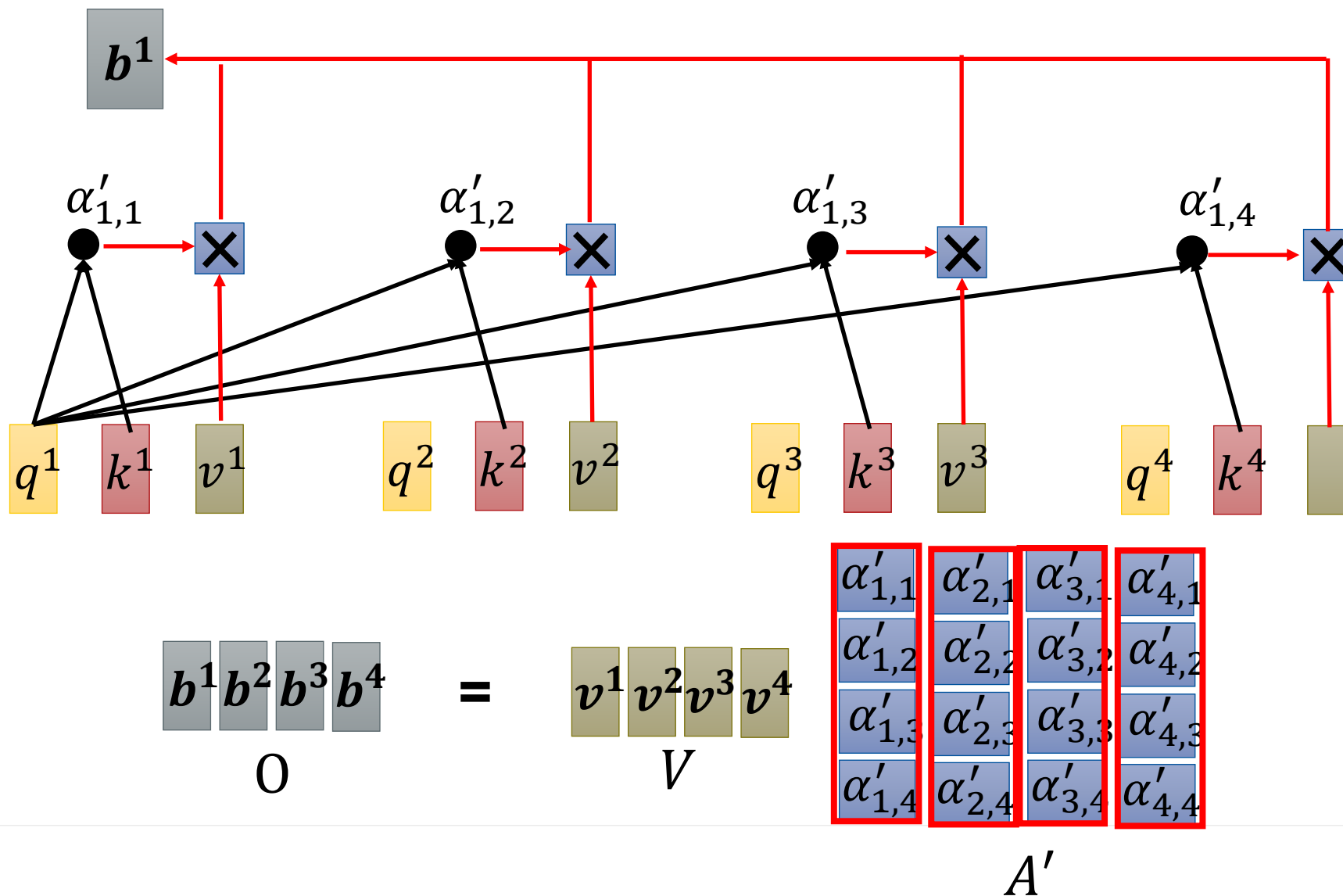
$$\begin{bmatrix} \alpha_{1,1} \\ \alpha_{1,2} \\ \alpha_{1,3} \\ \alpha_{1,4} \end{bmatrix} = \begin{bmatrix} k^1 \\ k^2 \\ k^3 \\ k^4 \end{bmatrix} q^1$$



$$\begin{bmatrix} \alpha'_{1,1} & \alpha'_{2,1} & \alpha'_{3,1} & \alpha'_{4,1} \\ \alpha'_{1,2} & \alpha'_{2,2} & \alpha'_{3,2} & \alpha'_{4,2} \\ \alpha'_{1,3} & \alpha'_{2,3} & \alpha'_{3,3} & \alpha'_{4,3} \\ \alpha'_{1,4} & \alpha'_{2,4} & \alpha'_{3,4} & \alpha'_{4,4} \end{bmatrix} \xleftarrow{\text{softmax}} \begin{bmatrix} \alpha_{1,1} & \alpha_{2,1} & \alpha_{3,1} & \alpha_{4,1} \\ \alpha_{1,2} & \alpha_{2,2} & \alpha_{3,2} & \alpha_{4,2} \\ \alpha_{1,3} & \alpha_{2,3} & \alpha_{3,3} & \alpha_{4,3} \\ \alpha_{1,4} & \alpha_{2,4} & \alpha_{3,4} & \alpha_{4,4} \end{bmatrix} = \begin{bmatrix} k^1 \\ k^2 \\ k^3 \\ k^4 \end{bmatrix} \begin{bmatrix} q^1 & q^2 & q^3 & q^4 \end{bmatrix} = K^T Q$$

A' softmax A K^T Q

Self-attention



Self-attention

$$\begin{aligned} Q &= W^q I \\ K &= W^k I \\ V &= W^v I \end{aligned}$$

要学习的参数

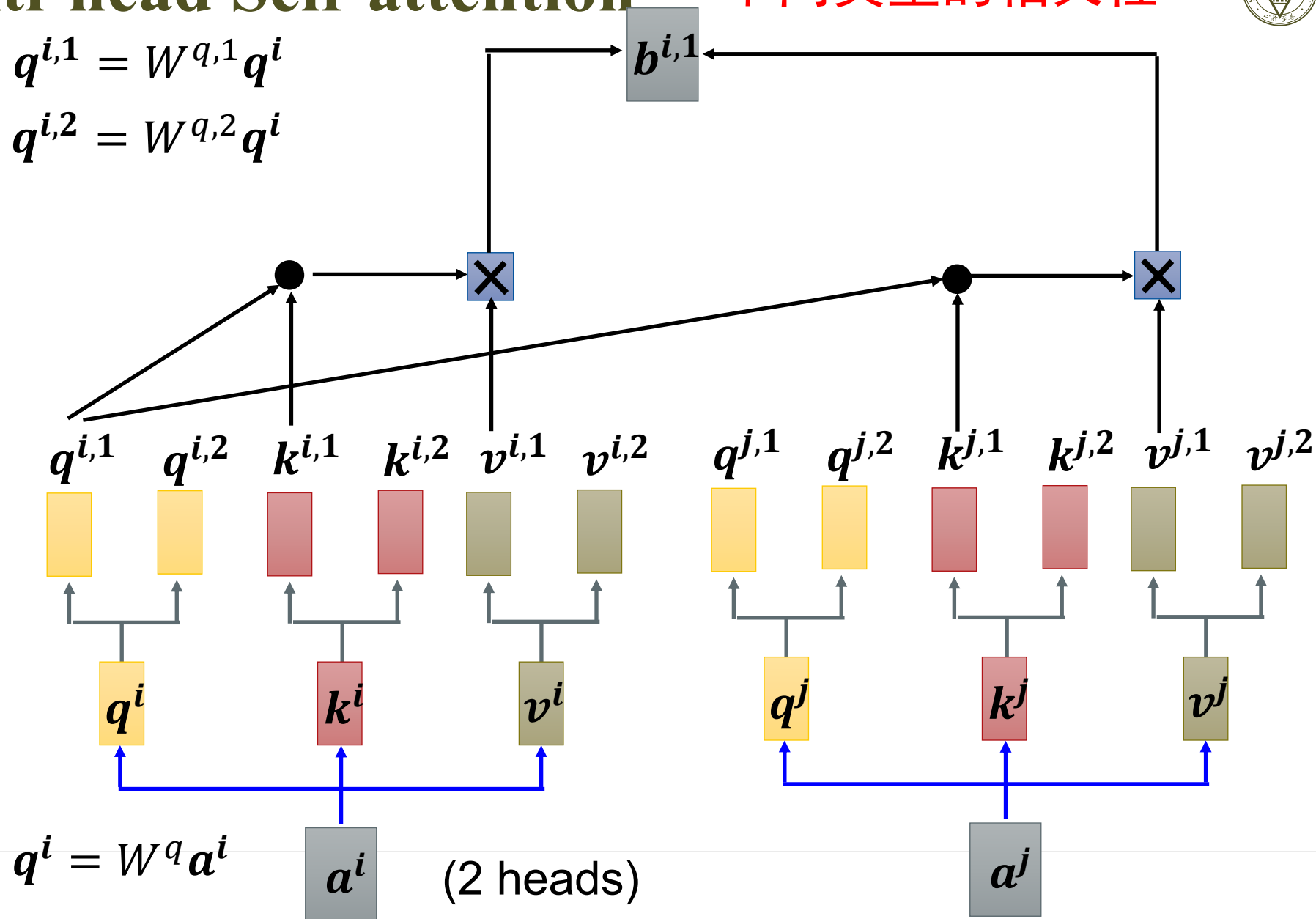
$$\begin{aligned} A' &\leftarrow A = K^T Q \\ 0 &= V A' \end{aligned}$$

Multi-head Self-attention

不同类型的相关性

$$q^{i,1} = W^{q,1} q^i$$

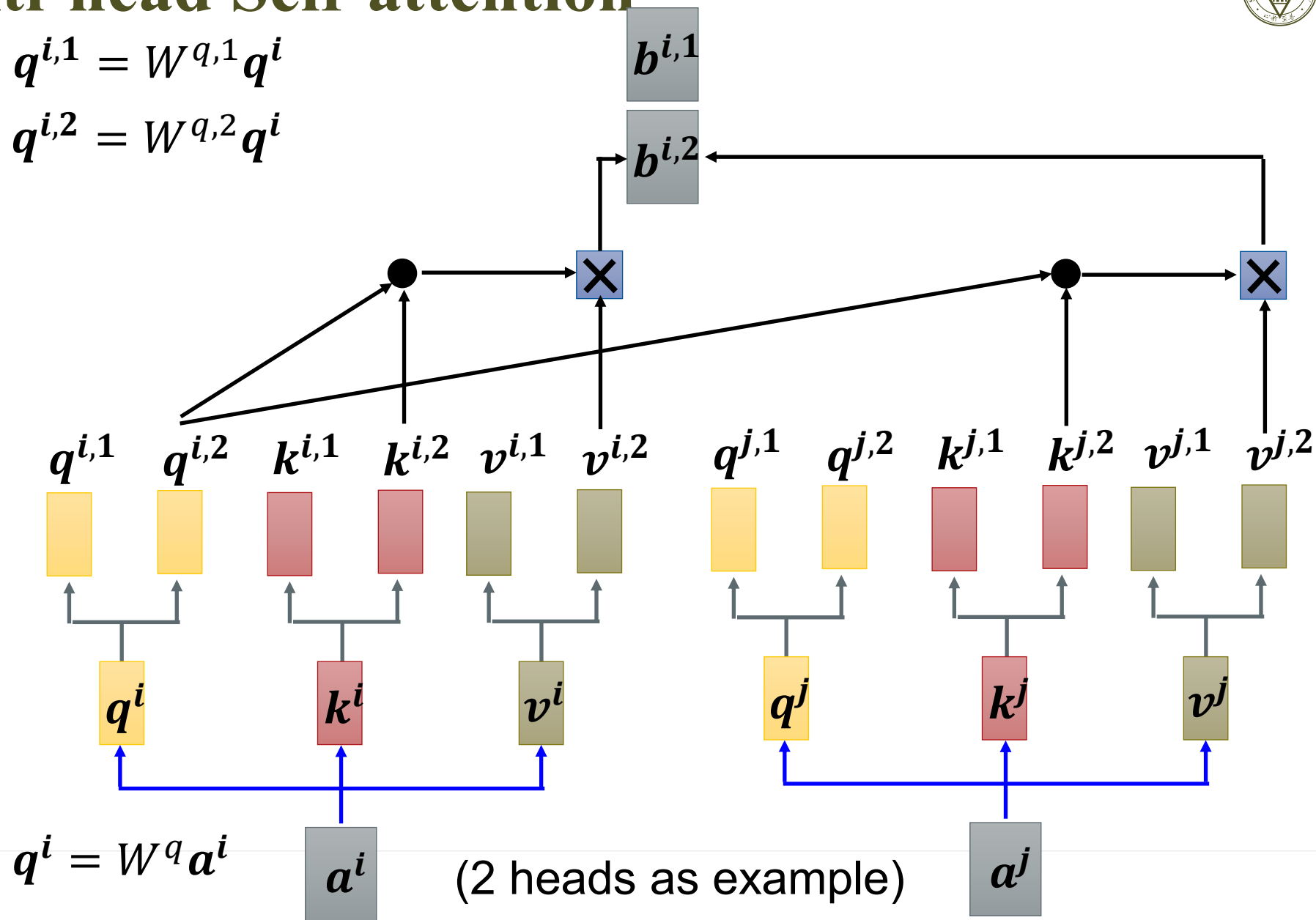
$$q^{i,2} = W^{q,2} q^i$$



Multi-head Self-attention

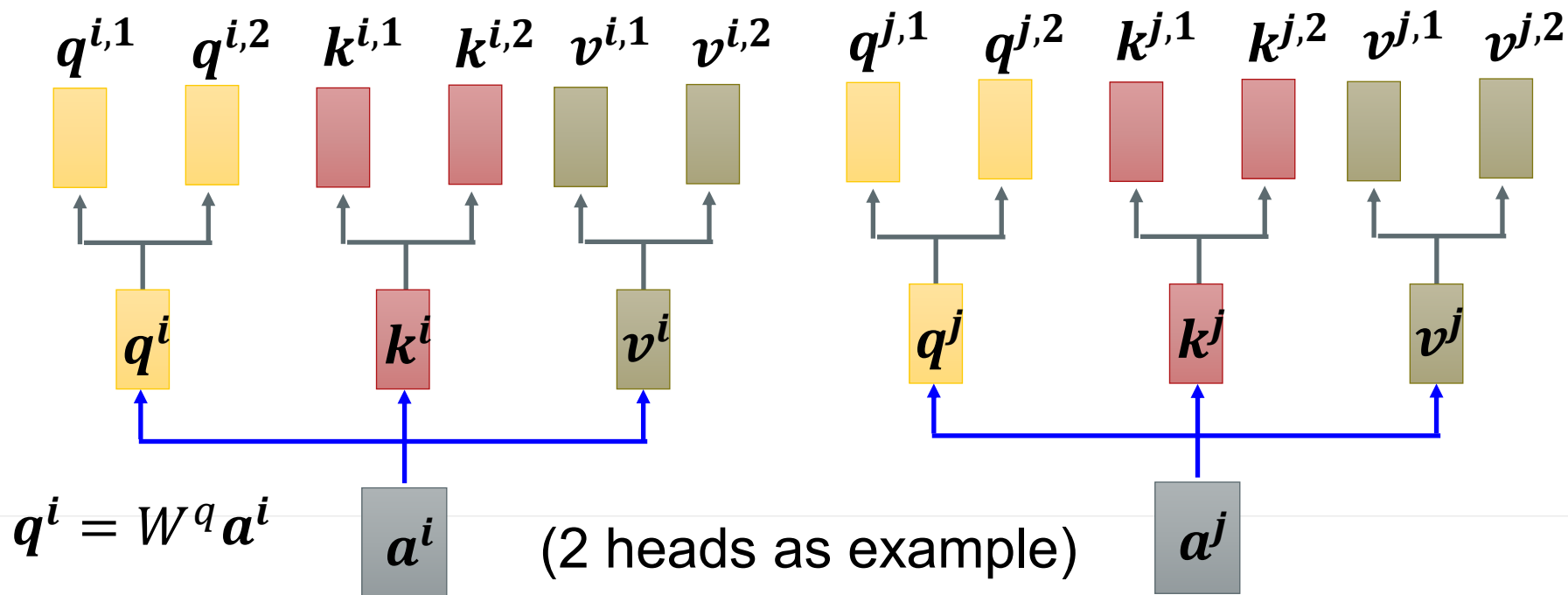
$$q^{i,1} = W^{q,1} q^i$$

$$q^{i,2} = W^{q,2} q^i$$



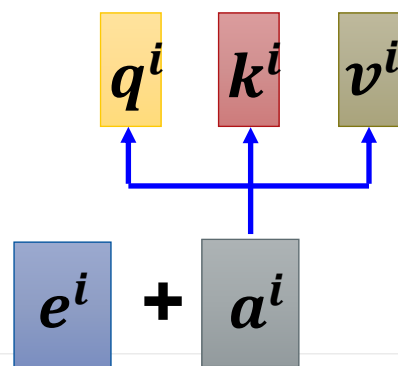
Multi-head Self-attention

$$b^i = W^O \begin{bmatrix} b^{i,1} \\ b^{i,2} \end{bmatrix}$$

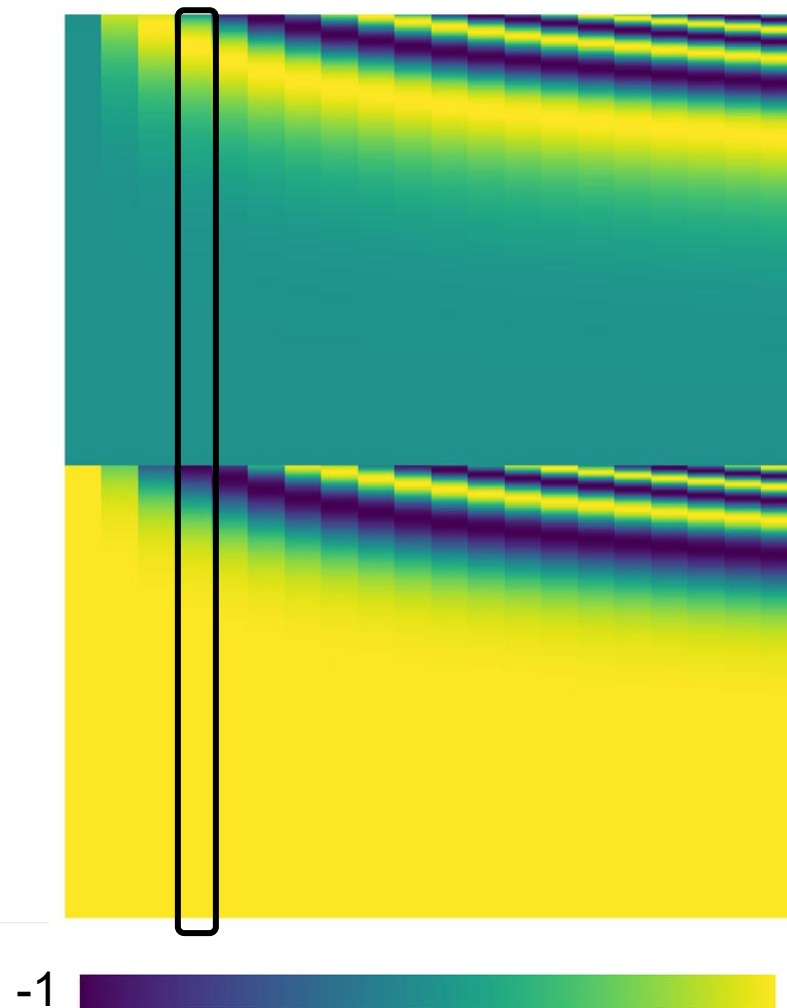


位置编码

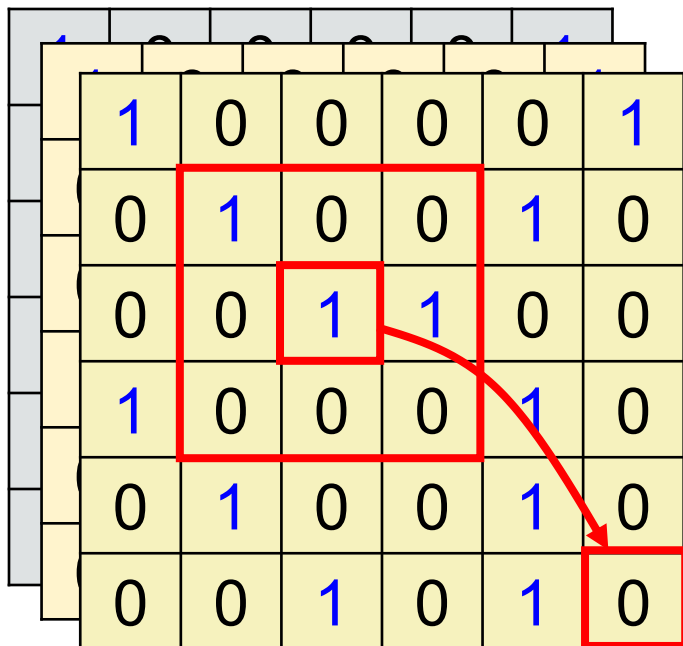
- 自注意力模块里没有位置信息
- 每个位置赋予一个特殊的向量 e^i
- 可以手工设置



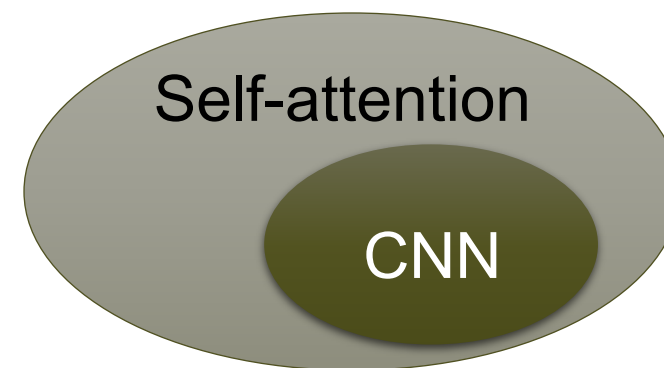
每一列表示一个位置向量 e^i



Self-attention v.s. CNN

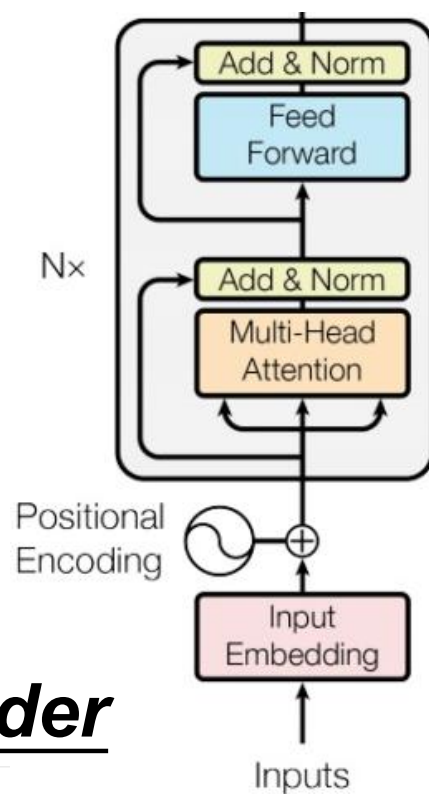


- CNN的自注意力机制只在感受野内进行，CNN是简化的self-attention
- Self-attention是具有可学习感受野的CNN

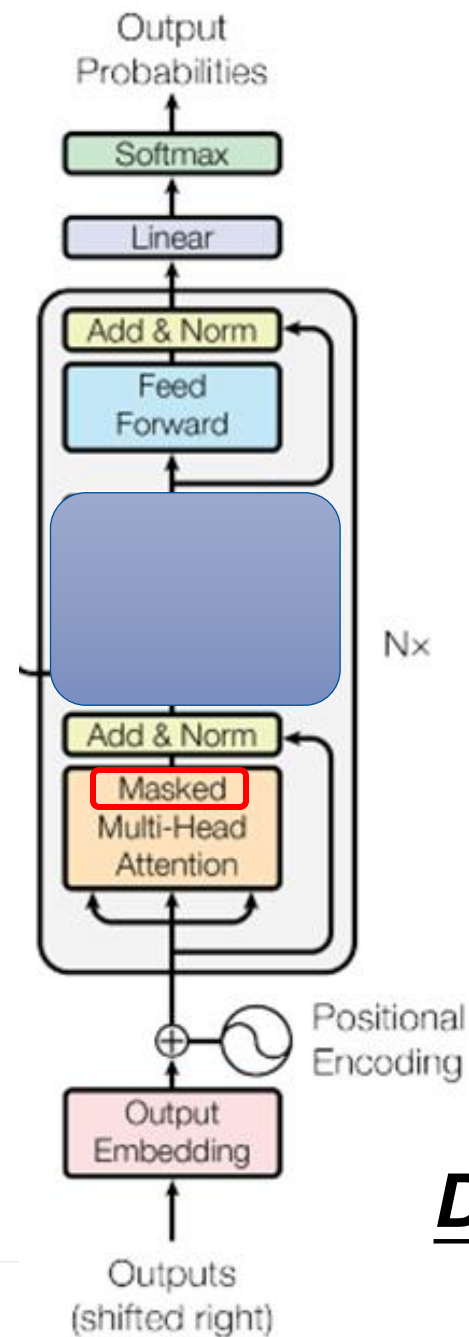


Transformer

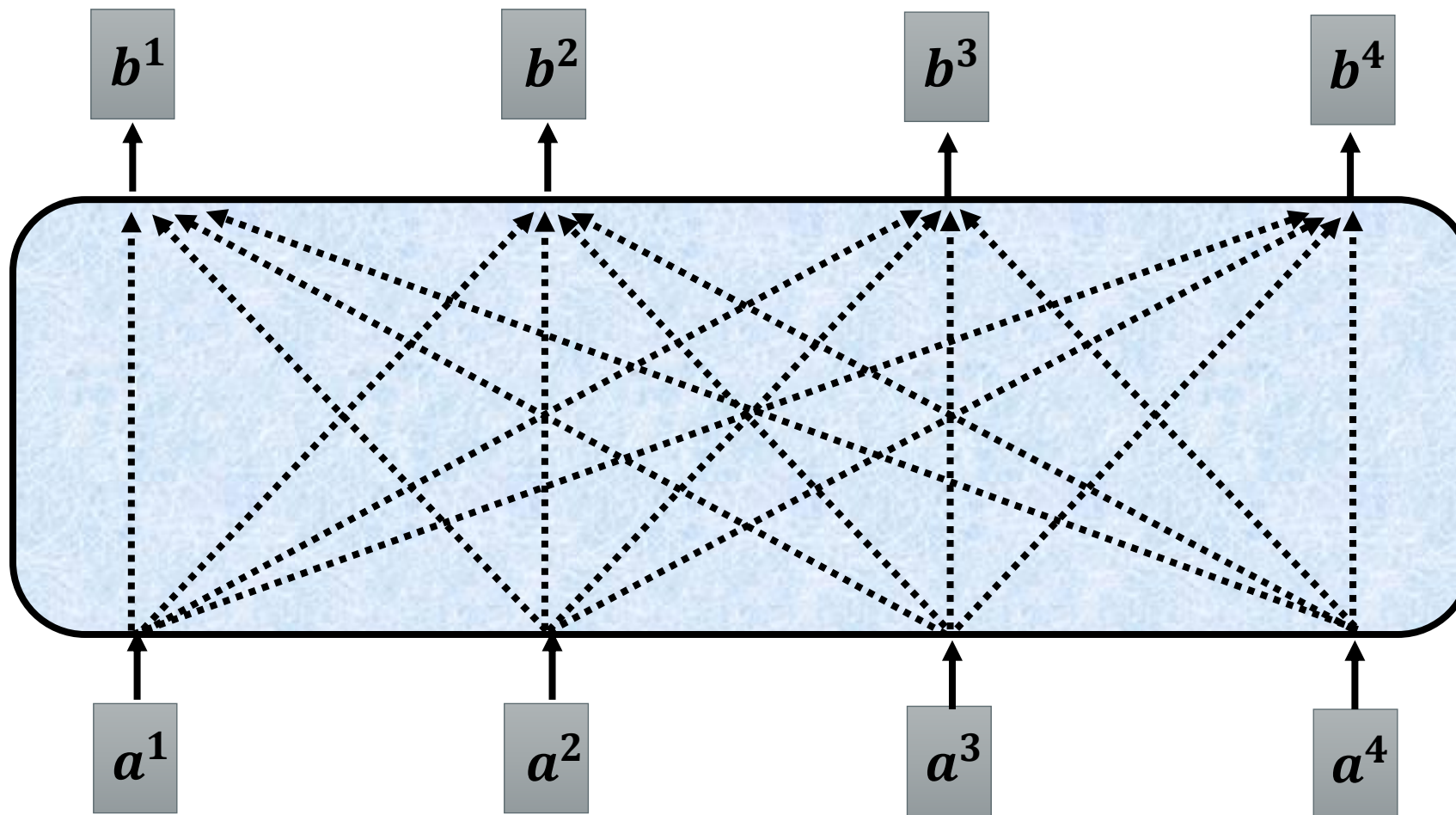
Encoder



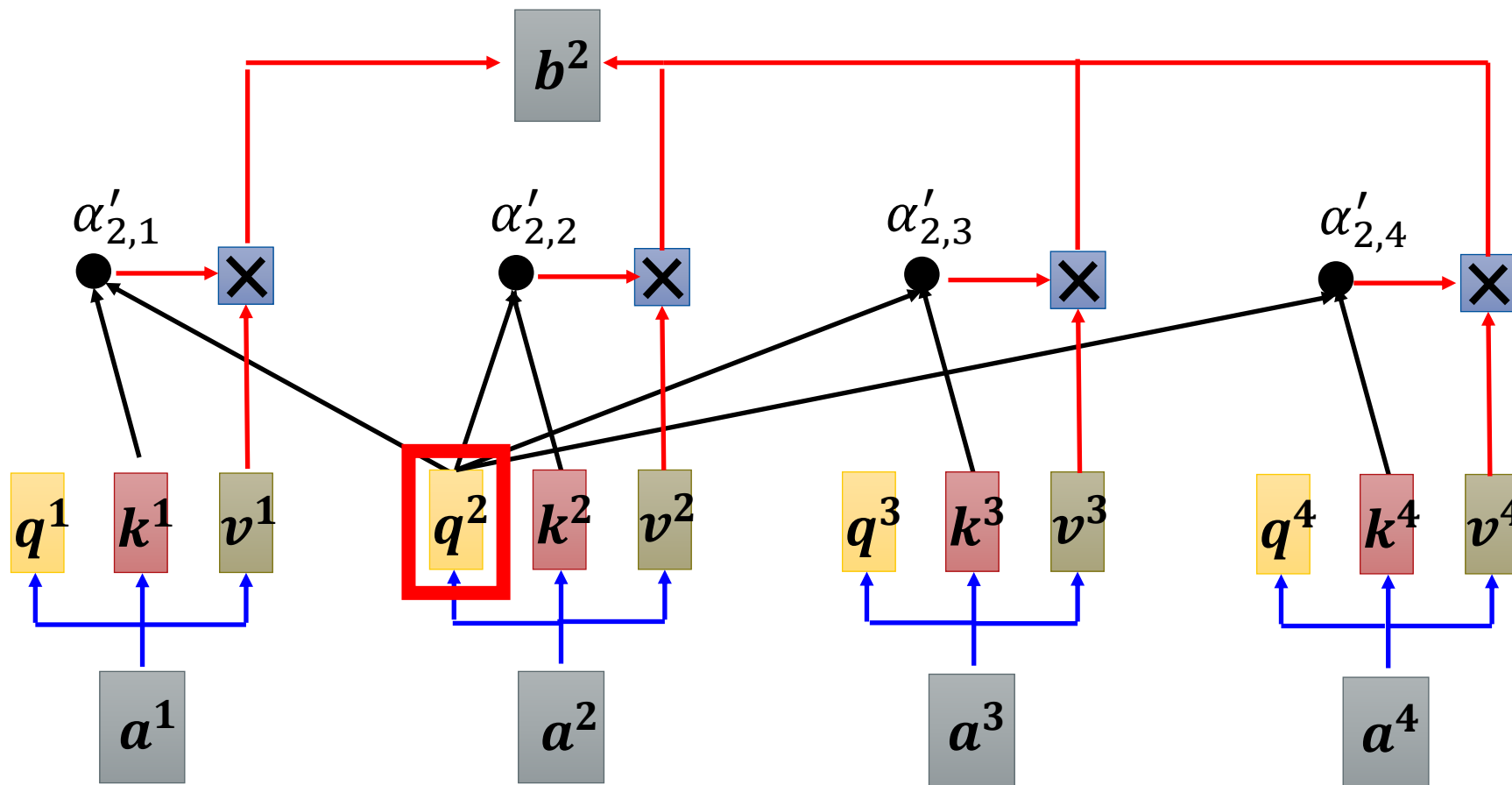
Decoder



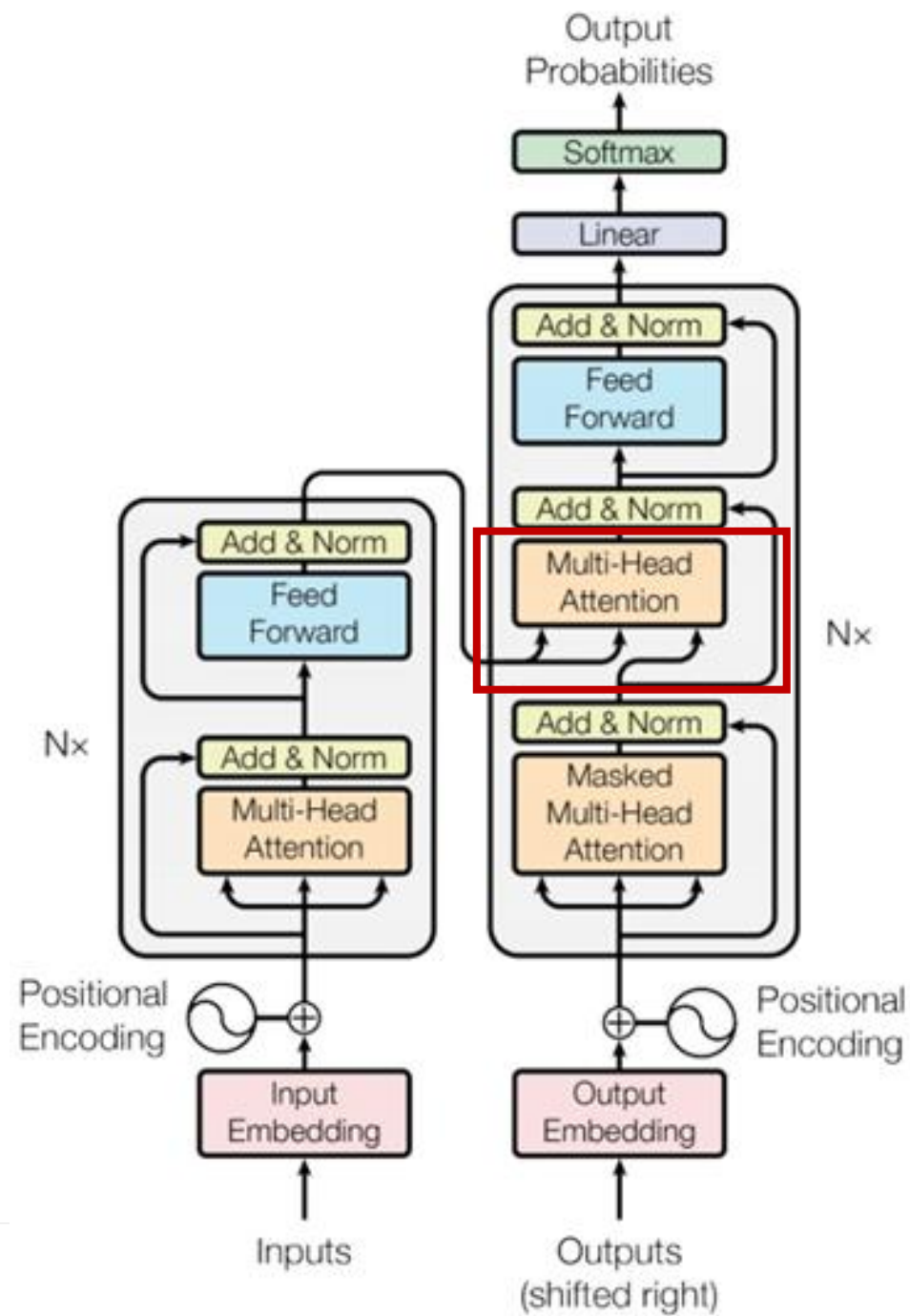
Masked Self-attention



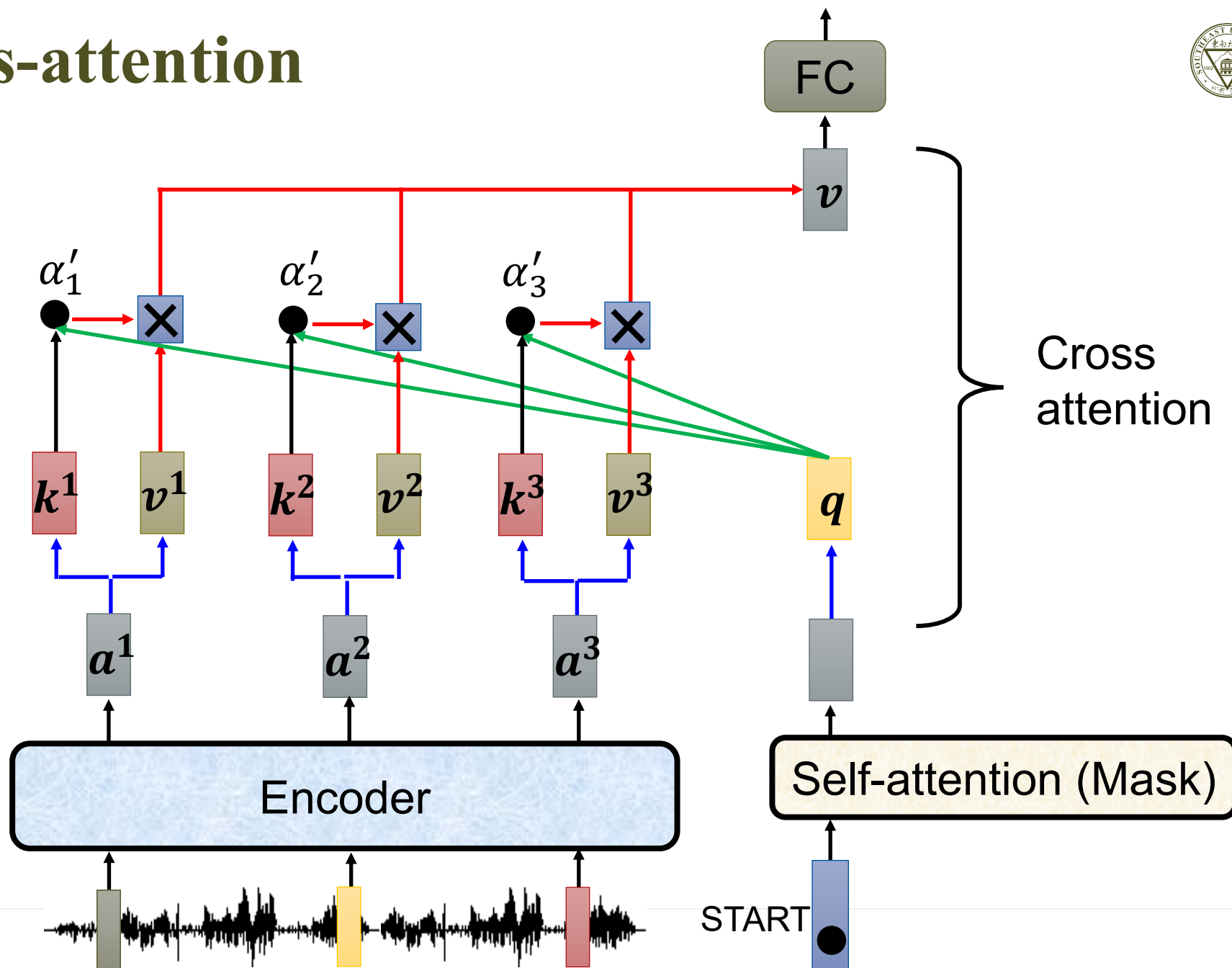
Masked Self-attention



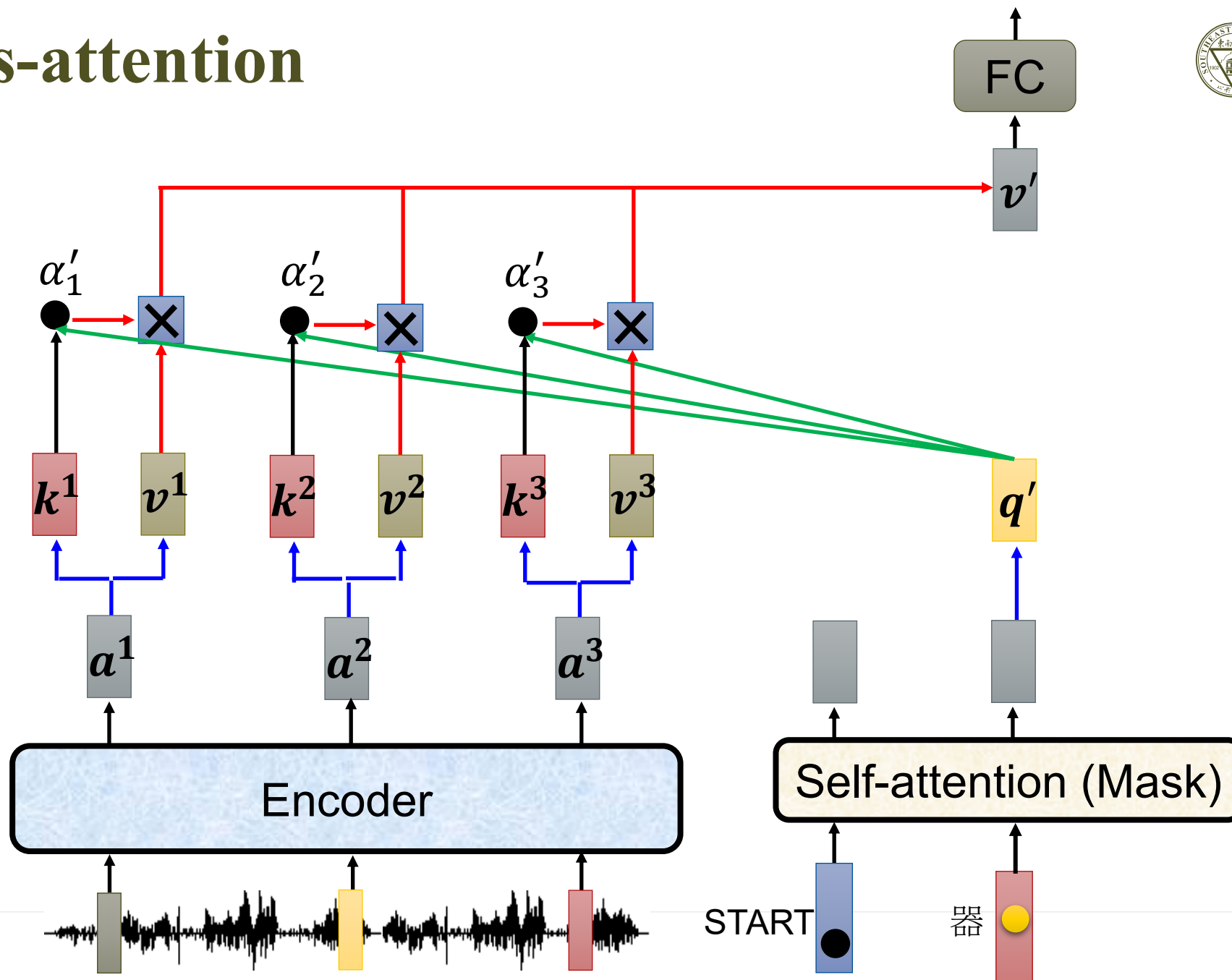
Transformer



Cross-attention



Cross-attention



Transformer

